VectorFFT

User Guide

v0.7 beta · October 2026

Set in CMU Serif and CMU Typewriter Text (Computer Modern Unicode 0.7.0, SIL Open Font License 1.1).
Mathematics set by KaTeX 0.16.21 (MIT licence) in the KaTeX fonts (SIL Open Font License 1.1).

Contents

How to use this guide 5
What the guide covers 5
What you need to know 5
Where to start 5
The programs 5
Commands and placeholders 6
Typography and references 6

Notation 7
Symbols 7
Conventions 7
Terms 8

Part I Getting started 9

1 Building and installing VectorFFT 10
1.1 Check what you need 10
1.2 Build and install the library 10
1.3 Build for other machines 12
1.4 Use the installed library from CMake 12
1.5 Build the library inside your project 12
1.6 Build a program with one command 13
1.7 Run your programs on Windows 13
1.8 Check the installation 14

2 Your first transform 16
2.1 Know what the two directions compute 16
2.2 Write the program 16
2.3 Build it with CMake 18
2.4 Run it 19
2.5 What goes wrong 19

Part II Using the API 20

3 Describing a transform 21
3.1 Start from a zeroed configuration 21
3.2 Add flags 22
3.3 Give the rank and the extents 22
3.4 Store a 2D or 3D array 23
3.5 See what the library serves 25

4 Plans: create, execute, destroy 27
4.1 Create a plan 27
4.2 Plan for the cost of the first create 27
4.3 Execute a plan 28
4.4 Execute a plan from one thread at a time 28
4.5 Destroy a plan 29

5 Buffers and placement 31
5.1 Allocate with `vfft_malloc()` 31
5.2 Size buffers from `vfft_info()` 32
5.3 Transform in place or out of place 33
5.4 Let an out-of-place plan overwrite its input 34

6 Complex transforms in one, two and three dimensions 36
6.1 Find the frequency of a bin 36
6.2 Leave the bins in the plan's own order 38
6.3 Transform a 2D or 3D array 38

7 Real transforms 41

7.1 Read the half spectrum 41
7.2 Filter real samples out of place 41
7.3 Give a C2R plan a half spectrum 43
7.4 Transform real samples in place 44
7.5 Transform a real array in 2D or 3D 45
7.6 Transform a real array in place with padded rows 45
7.7 Let the plans choose the row pitch 47

8 Batches 49
8.1 Choose where the transforms lie 49
8.2 Run a batch of complex transforms 50
8.3 Run a batch of real transforms 52
8.4 Run a batch of 2D or 3D arrays 52
8.5 Fix a refused batch 55

9 Threads 57
9.1 Run a plan on several threads 57
9.2 Compare results across thread counts 59
9.3 Call plans from threads of your own 59

10 Planning and wisdom 61
10.1 Know what a create costs 61
10.2 Find the wisdom of your CPU 63
10.3 Name a store of your own 64
10.4 Ship wisdom with an application 65
10.5 Measure a configuration again 65
10.6 Plan without measuring 65

11 Errors and messages 67
11.1 Read why a create was refused 67
11.2 Read why an execute was refused 69
11.3 Print the reasons as they happen 70
11.4 Read the lines the library prints unasked 70

Part III Worked examples 72

12 Solving a periodic Poisson equation 73
12.1 Build and run the examples 73
12.2 Create the two plans 74
12.3 Divide in the half spectrum 74
12.4 Adapt the program 75

13 Linear convolution of 3D data 77
13.1 Run the program 77
13.2 Pad both arrays to the length of the result 77
13.3 Transform both arrays, multiply, transform back 78
13.4 Adapt the program 78

14 Spectral derivatives in place 79
14.1 Run the program 79
14.2 Create the in-place plans and read the pitch 79
14.3 Differentiate along each axis 80
14.4 Adapt the program 81

15 Propagating a wave packet 82
15.1 Run the program 82
15.2 Create one plan for both directions 82
15.3 Filter the spectrum between the two transforms 83
15.4 Adapt the program 83

Part IV Moving a DFTI program 84

16 Switching a DFTI program to VectorFFT 85

16.1 Change the build 85
16.2 Build and run a DFTI program 86
16.3 Run a 3D real transform 88
16.4 Pass the values at DFTI's types 90
16.5 Keep MKL for BLAS and LAPACK 91

17 What the DFTI interface supports 93
17.1 Look up a function 93
17.2 Look up a configuration parameter 93
17.3 Describe complex data with strides and distances 97
17.4 Store real data 97
17.5 Compute on several threads 101
17.6 Read a status and its message 101
17.7 Know what DFTI does not reach 102

Part V Reference 103

18 `vfft.h` reference 104
18.1 What the header declares 104
18.2 Functions 104
18.3 The configuration 106
18.4 Transforms and directions 107
18.5 Layout, placement, order and batch 107
18.6 Flags 108
18.7 The facts of a plan 109
18.8 Names not part of this release 110

19 Environment variables 111
19.1 Set a variable for a program 111
19.2 `VFFT_WISDOM_DIR`: a store of your own 111
19.3 `VFFT_VERBOSE`: print every refusal 112
19.4 `VFFT_WISDOM_WRITE`: keep the store unchanged 112
19.5 `VFFT_THREAD_BLOCK_MS`: how long idle workers spin 112

A Notes for Windows and MinGW 113
A.1 Use one compiler for the library and your programs 113
A.2 Name the generator and the compiler to CMake 113
A.3 Give the program `libwinpthread-1.dll`, or link it statically 113
A.4 Keep build folders short 114
A.5 Build for the CPU that runs the program 114
A.6 Use `pkgconf` where `pkg-config` is missing 114

B Troubleshooting 115
B.1 The build fails 115
B.2 The program does not start 116
B.3 A create returns `NULL` 116
B.4 An execute does nothing, or the numbers look wrong 116
B.5 A create takes seconds, or measures on every run 117
B.6 A DFTI program 118

How to use this guide

This guide shows you how to build VectorFFT v0.7 beta, call it from your C and C++ programs, and move a program written for Intel MKL's DFTI interface to it. Each chapter of Parts I to IV starts from something you do and ends in a program that does it, with what the program printed.

What the guide covers

- **The API of `vfft.h`.** Complex and real transforms in one, two and three dimensions, in double precision, with complex data stored as interleaved pairs; batches of transforms; threads; the wisdom the library measures and keeps; and planning a complex transform without measuring (Section 10.6).
- **The DFTI interface** in `src/shims/dfti/`, which builds a program written for Intel MKL's DFTI interface against VectorFFT.
- **Four example programs** in `how-to-use/examples/`, each of which solves a problem on three-dimensional data.

`vfft.h` also declares a few names that are not part of this release; Section 18.8 lists them.

What you need to know

You write C or C++, and you know what a discrete Fourier transform is. The guide states the library's conventions where a call depends on them, and Notation collects them: the sign and scale of each direction, how arrays are stored, and which bin holds which frequency.

Where to start

The guide has five parts:

- Part I builds and installs the library (Chapter 1) and writes a first program (Chapter 2). Read it first.
- Part II takes one task per chapter, from describing a transform (Chapter 3) to reading what the library prints (Chapter 11). Read the chapter of your task.
- Part III walks through one of the four example programs per chapter. Read it for a complete program to start from.
- Part IV moves a program that calls DFTI to VectorFFT. Read Chapter 1, then Chapter 16.
- Part V is for looking things up: `vfft.h` (Chapter 18), the environment variables (Chapter 19), notes for Windows (Appendix A) and what to do when something does not work (Appendix B).

The programs

A listing headed by a file name is a complete program: save it under that name and build it as Chapter 2 shows, or, when it calls DFTI, as Chapter 16 shows. A listing headed From and a file name shows a part of a program, and an excerpt of an example names its file in `how-to-use/examples/`. A program that computes something checks its result against a stated reference and tolerance, ends its output with PASS or FAIL, and exits with status 0 only on PASS.

Each program was compiled against an installed VectorFFT and run before the guide quotes its output, which appears exactly as printed. The one exception is the line the library prints on standard error about its wisdom, where the path of the folder appears as `<the folder of this CPU>` (Section 10.2). Numbers at the level of rounding can differ in the last digits on your machine.

Commands and placeholders

Commands are for a POSIX shell: Git Bash on Windows, or a shell on Linux. Section 19.1 shows how to set an environment variable in Git Bash and in PowerShell, and Appendix A gives the notes for Windows. A word in angle brackets stands for a value of your own: `<prefix>` is the folder VectorFFT is installed in, `<VectorFFT>` the folder of its sources, and `<tag>` the name of a release.

Typography and references

- `vfft_create()`, `VFFT_FORWARD`, `n[0]`: names from the code, files and commands, in typewriter type and spelled exactly as in the code.
- $x[n]$, $X[k]$, N: mathematical symbols, in italics, as Notation lists them.
- Sections, equations, figures and tables are numbered by chapter, and every reference is a link: Section 2.3, Eq. (2.1), Figure 2.1, Table 1.1.
- A note gives something you need at that point; a warning, something that breaks a program or its results.

Notation

This page lists the symbols the guide uses and the names they have in the code (Table 0.1), the conventions every chapter relies on, and the terms the guide uses with one meaning each (Table 0.2).

Symbols

Table 0.1 The symbols of the guide and their names in the code.

Symbol	Meaning	In the code
j	the imaginary unit	`vfft.h` writes `i`; C99 code writes `I`
$W_N = e^{-j2\pi/N}$	the kernel of the forward transform	
$x[n]$, $0 \le n \le N-1$	the input of a 1D transform	`in`: $\mathcal{R}e\,x[n]$ at `in[2n]`, $\mathcal{I}m\,x[n]$ at `in[2n+1]`
N	the length	`n[0]`, with `rank` 0 or 1
$X[k] = \sum_n x[n]\,W_N^{kn}$	the forward transform	`VFFT_FORWARD`
$\sum_k X[k]\,W_N^{-kn} = N\,x[n]$	the backward transform: no $1/N$	`VFFT_BACKWARD`
N_1, N_2, N_3	the extents, N_1 slowest	`n[0]`, `n[1]`, `n[2]`; the last is contiguous
$x[n_1, n_2, n_3]$, $X[k_1, k_2, k_3]$	a 3D array and its transform	`rank = 3`
$\ell = (n_1 N_2 + n_2)N_3 + n_3$	where a sample is stored	row-major, the last axis contiguous
$((n))_N$	n modulo N	
$X[k] = X^*[((-k))_N]$	the symmetry of the transform of a real sequence	why `VFFT_R2C` keeps half
$\lfloor N/2 \rfloor + 1$	the bins of the half spectrum	`N/2 + 1` in C
K, t	the batch count; one transform of a batch	`howmany`; $0 \le t \le K-1$

Conventions

- **Sign and scale.** The forward kernel is $e^{-j2\pi kn/N}$, Eqs. (2.1). A forward and then a backward transform return the input times N in 1D, N_1N_2 in 2D and $N_1N_2N_3$ in 3D; a `VFFT_C2R` transform after a `VFFT_R2C` transform does the same. Your program divides, best inside a pass over the data it makes anyway.
- **Storage.** Arrays are row-major, the last axis contiguous: a 2D array holds $x[n_1, n_2]$ at $\ell = n_1N_2 + n_2$, and a 3D array holds $x[n_1, n_2, n_3]$ at $\ell = (n_1N_2 + n_2)N_3 + n_3$, Eq. (3.1). A complex sample takes two doubles, its real part at element 2ℓ and its imaginary part at element $2\ell + 1$ (Figure 2.1).
- **Frequencies.** Bin $k \le N/2$ holds frequency k, and bin $k > N/2$ holds frequency $k - N$. At an even N, bin $N/2$ is both $N/2$ and $-N/2$ (Section 6.1).
- **Index letters.** n counts samples, k counts bins, and t counts the transforms of a batch. Where `vfft.h` writes `e` for an element of a batch, the guide writes n.
- **Batches.** In a contiguous batch, element n of transform t is at `z[2(t N + n)]`; in a lane-major batch, at `z[2(n K + t)]`, Eqs. (8.1).

Terms

Table 0.2 The terms of the guide, each with one meaning.

Term	Meaning
configuration	what a `vfft_config_t` describes: the transform, its rank and extents, the batch, the layout, placement and order of the data, the threads and the flags
plan	what `vfft_create()` returns (`vfft_plan`): one configuration committed to one measured plan, ready to execute; it owns what it allocated until `vfft_destroy()`
create	the call `vfft_create()`: it checks the configuration, serves it from wisdom or measures it on a miss and saves the winner, then builds the plan
execute	the call `vfft_execute()`: it runs a plan on your buffers, and allocates, measures and decides nothing
wisdom	the record of the plan that won when a configuration was measured
wisdom folder	the folder of wisdom that comes with the library, with one folder in it for each CPU: `src/wisdom/` in the sources, `<prefix>/share/vectorfft/wisdom/` once installed
store	the directory a create reads wisdom from and saves winners into: the folder of this CPU in the library's wisdom folder, or a directory that `wisdom_dir` or `VFFT_WISDOM_DIR` names
miss	a create whose configuration has no wisdom in the store: it measures, then saves the winner
refusal	a configuration or call the library does not serve: `vfft_create()` returns `NULL`, and `vfft_error()` says why; nothing is converted and nothing falls back
natural order	`VFFT_ORDER_NATURAL`, the default: bin k at position k
half spectrum	what `VFFT_R2C` writes and `VFFT_C2R` reads: bins 0 to $N/2$ of the last axis
lane-major	`VFFT_BATCH_LANE_MAJOR`: the transforms of a batch side by side, sample by sample
pitch	the doubles from one row to the next of an in-place 2D or 3D real array: `vfft_info().stride`; `VFFT_FLAG_OWN_PITCH` lets the plan choose it

Part I **Getting started**

1 Building and installing VectorFFT

In this chapter you build VectorFFT from its sources and install it. Then you link it into a program of your own in one of three ways: from a CMake project that finds the installed library, from a CMake project that builds the library itself, or with one compiler command. The chapter ends with a short program that checks the installation.

1.1 Check what you need

Table 1.1 What building and using VectorFFT needs.

You need	Notes
an x86-64 CPU with AVX2 and FMA	the library runs on such a CPU only, and so does every program linked with it
GCC, Clang or Intel's ICX	on Windows, the MinGW-w64 build of GCC, or ICX; Microsoft's compilers (`cl`, `clang-cl`) are not supported
CMake 3.16 or newer	to build the library, and to build programs with it
Ninja	the build tool the commands of this guide name
Git	to get the sources

This guide was made on Windows 11 with MinGW-w64 GCC 15.2.0 (the WinLibs UCRT build), CMake 4.3.1 and Ninja 1.13.2. Its commands are for a POSIX shell: Git Bash on Windows, or a shell on Linux.

1.2 Build and install the library

Get the sources, then configure, build and install them with CMake. `<prefix>` is the folder to install into, an absolute path outside the folder of the sources:

```
git clone https://github.com/Tugbars/VectorFFT.git
cd VectorFFT
cmake -S . -B build -G Ninja -DCMAKE_C_COMPILER=gcc \
      -DCMAKE_INSTALL_PREFIX=<prefix> -DVFFT_BUILD_GAUNTLET=OFF
cmake --build build
cmake --install build
```

The configure command names three things:

- `-G Ninja` and `-DCMAKE_C_COMPILER=gcc` name the build tool and the compiler; for Clang, write `-DCMAKE_C_COMPILER=clang`, and for ICX `-DCMAKE_C_COMPILER=icx`, from the oneAPI command prompt that sets its paths. Name both: without them, CMake on a Windows machine with Visual Studio picks Microsoft's compiler.
- `-DCMAKE_INSTALL_PREFIX=<prefix>` names the folder `cmake --install` installs into.
- `-DVFFT_BUILD_GAUNTLET=OFF` leaves out the gauntlet, the repository's tool for testing the library on your own machine. You do not need it to build or use VectorFFT.

The build compiles about 1,600 files in Release mode. On the Intel Core i9-14900KF this guide was made on, with 32 threads, it took about four minutes. It also builds the four example programs of Part III, which Section 12.1 runs; `-DVFFT_BUILD_EXAMPLES=OFF` leaves them out. On Windows, keep the path

of the build folder short, as `build` inside the sources is: when CMake warns at configure that the path of an object file would pass 250 characters, the build can fail (Section A.4).

Table 1.2 lists what the install puts in `<prefix>`. A program linked with the library is about 35 MB.

Table 1.2 What `cmake --install` puts in `<prefix>`.

Path	What it holds
`include/vfft.h`	the header your programs include
`include/vfft_diagnostics.h`	a second header, not part of this release (Section 18.8)
`lib/libvfft.a`	the library: one static archive of about 41 MB
`lib/cmake/VectorFFT/`	the package that CMake's `find_package(VectorFFT)` reads
`lib/pkgconfig/vectorfft.pc`	the package file that `pkgconf` reads
`share/vectorfft/wisdom/`	the library's wisdom folder: one folder of wisdom for each CPU, which your programs read and save into (Section 10.2)
`share/vectorfft/frozen/`	tables the library reads at run time

The library reads `<prefix>/share/vectorfft/` at run time, and the path is built into it when you configure. Install it to the prefix you configured, and leave it there: `cmake --install` refuses another prefix. To install somewhere else, configure again with that prefix and build again. Your programs also save the wisdom they measure into `<prefix>/share/vectorfft/wisdom/`, in the folder of their CPU, so install to a prefix you can write: under a prefix that only an administrator can write, nothing is saved, and every run measures again what the folder lacks (Section 10.2). Where the library must stay under such a prefix, name a writable store seeded from the folder of this CPU (Section 10.3).

Figure 1.1 shows the whole route: the sources are configured, built and installed once, and a project of yours finds the installed package and links one target.

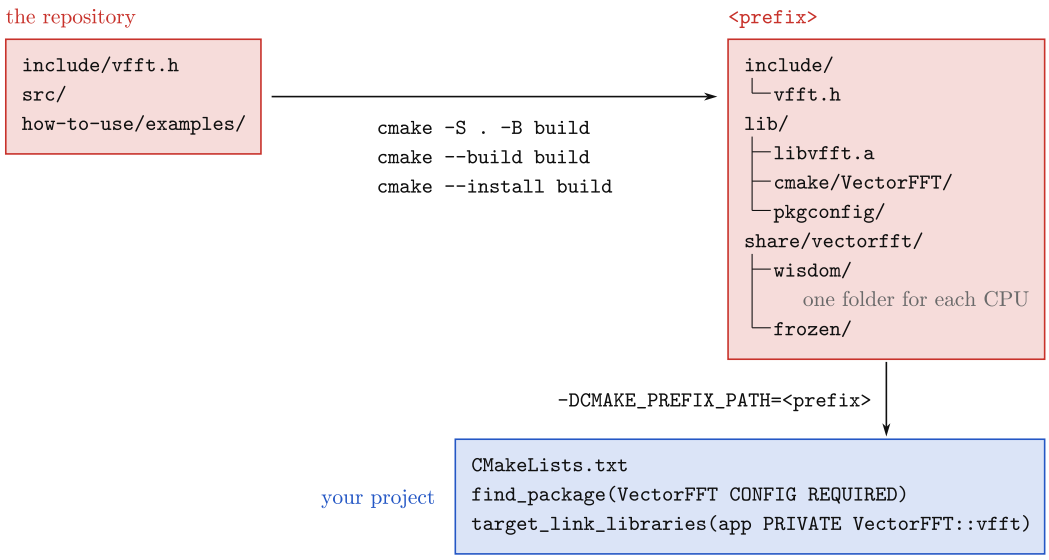


Figure 1.1 From the repository to your project. Three `cmake` commands build the library's pieces (red) and install them into `<prefix>`; your project (blue), told where they are with `-DCMAKE_PREFIX_PATH=<prefix>`, finds the package with `find_package()` and links the target `VectorFFT::vfft`.

1.3 Build for other machines

By default the library is compiled for the CPU of the machine that builds it, with GCC's `-march=native`, and so is the part of every program that comes from the library: such a program may not run on a computer with another CPU. To build programs that run on any x86-64 CPU with AVX2 and FMA, configure with `-DVFFT_NATIVE=OFF`:

```
cmake -S . -B build -G Ninja -DCMAKE_C_COMPILER=gcc \
      -DCMAKE_INSTALL_PREFIX=<prefix> -DVFFT_BUILD_GAUNTLET=OFF \
      -DVFFT_NATIVE=OFF
```

1.4 Use the installed library from CMake

Your project finds the installed library with `find_package()` and links `VectorFFT::vfft`, the two lines of Figure 1.1; the target brings the include folder, the library and the system libraries it needs:

```
cmake_minimum_required(VERSION 3.16)
project(app LANGUAGES C)
find_package(VectorFFT CONFIG REQUIRED)
add_executable(app app.c)
target_link_libraries(app PRIVATE VectorFFT::vfft)
```

Tell CMake where the library is installed with `CMAKE_PREFIX_PATH`, then build:

```
cmake -S . -B build -G Ninja -DCMAKE_C_COMPILER=gcc \
      -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=<prefix>
cmake --build build
```

A C++ program links the same way: write `project(app LANGUAGES CXX)`, and name the compiler with `-DCMAKE_CXX_COMPILER=g++`. `vfft.h` declares its functions for C++ as well, and Section 5.1.1 shows the two things a C++ program writes differently. Chapter 2 builds a first program with such a project.

> **Note.** Build your programs with the compiler that built the library: the same MinGW-w64 build of GCC, or ICX when ICX built it (Section A.1).

1.5 Build the library inside your project

Your CMake project can also build VectorFFT itself, from a copy of the sources you keep or from a copy CMake downloads, and link the same target. From a copy you keep, `<VectorFFT>` being its folder:

```
cmake_minimum_required(VERSION 3.16)
project(app LANGUAGES C)
add_subdirectory(<VectorFFT> vectorfft EXCLUDE_FROM_ALL)
add_executable(app app.c)
target_link_libraries(app PRIVATE VectorFFT::vfft)
```

The second argument of `add_subdirectory()`, `vectorfft`, names the folder of your build where the library is built; CMake needs it when the sources lie outside your project. `EXCLUDE_FROM_ALL` builds only what your targets use. From a copy CMake downloads, `<tag>` naming the release you want:

```
cmake_minimum_required(VERSION 3.16)
project(app LANGUAGES C)
include(FetchContent)
FetchContent_Declare(VectorFFT
  GIT_REPOSITORY https://github.com/Tugbars/VectorFFT.git
  GIT_TAG        <tag>
  GIT_SHALLOW    TRUE)
FetchContent_MakeAvailable(VectorFFT)
add_executable(app app.c)
target_link_libraries(app PRIVATE VectorFFT::vfft)
```

Either project configures and builds with no prefix:

```
cmake -S . -B build -G Ninja -DCMAKE_C_COMPILER=gcc -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Built inside your project, VectorFFT builds the library alone: no example programs, no test tools and no install rules. Your first build compiles its 1,600 files with your project's compiler. To build for other machines (Section 1.3), write `set(VFFT_NATIVE OFF)` before the library is added. The library's wisdom folder is then `src/wisdom/` in the copy of the sources, which your programs read and save into (Section 10.2).

1.6 Build a program with one command

One compiler command also builds a program against the installed library. `pkgconf` reads the flags from the package file:

```
export PKG_CONFIG_PATH=<prefix>/lib/pkgconfig
gcc -O3 app.c $(pkgconf --cflags --libs vectorfft) -o app
```

Or you name them yourself:

```
gcc -O3 app.c -I<prefix>/include -L<prefix>/lib -lvfft -lm -o app
```

On Linux, add `-pthread`. The MinGW-w64 build named in Section 1.1 provides the tool as `pkgconf`; where yours is called `pkg-config`, the same command works with that name.

1.7 Run your programs on Windows

A program built with MinGW-w64 GCC loads `libwinpthread-1.dll` when it starts. In the shell you build in, the `bin` folder of the compiler is on `PATH`, and Windows finds the DLL there. Where Windows finds no copy of the DLL, beside the program or on `PATH`, the program does not start: in PowerShell it prints nothing and its exit status is -1073741515 (0xC0000135), and in Git Bash it stops with `error while loading shared libraries` and exit status 127. Git Bash puts Git's own copy of the DLL on its `PATH`, so a program can start there and still fail elsewhere (Section A.3). You have two fixes:

- link the program with `-static`, which puts the thread library into the program;
- or copy `libwinpthread-1.dll` from the `bin` folder of the compiler beside the program.

With one command:

```
gcc -O3 app.c -I<prefix>/include -L<prefix>/lib -lvfft -lm -static -o app
```

With CMake, add one line after `target_link_libraries()`:

```
target_link_options(app PRIVATE -static)
```

Appendix A gives more notes for Windows.

1.8 Check the installation

This program makes a plan for 8 complex points and transforms the impulse, $x[0] = 1$ and $x[n] = 0$ for $n \neq 0$, whose transform is 1 in every bin. Save it as `ch01_check_install.c`:

ch01_check_install.c

```
/* ch01_check_install.c: checks a VectorFFT installation. It builds a plan
 * for 8 complex points and transforms the impulse x = (1, 0, ..., 0),
 * whose transform is 1 in every bin.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

enum { N = 8 };
static const double TOL = 1e-12;

int main(void)
{
    vfft_config_t cfg = {0};        /* zeroed: one 1D complex transform */
    cfg.n[0] = N;
    vfft_plan plan = vfft_create(&cfg);
    if (!plan)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }

    vfft_info_t info;
    vfft_info(plan, &info);
    double *x = vfft_malloc(info.in_doubles * sizeof(double));
    double *X = vfft_malloc(info.out_doubles * sizeof(double));
    if (!x || !X)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (size_t i = 0; i < info.in_doubles; i++)
        x[i] = 0.0;
    x[0] = 1.0;                     /* the real part of x[0] */

    vfft_execute(plan, VFFT_FORWARD, x, X);

    double err = 0.0;
    for (int k = 0; k < N; k++)     /* every bin must be 1 + 0j */
    {
        double d = hypot(X[2 * k] - 1.0, X[2 * k + 1]);
        if (d > err || isnan(d))
            err = d;
    }
    int pass = err <= TOL;          /* false for a NaN */
```

```
    printf("impulse of %d points: max |X[k] - 1| = %.1e, tolerance %.0e: %s\n",
           N, err, TOL, pass ? "PASS" : "FAIL");

    vfft_free(X);
    vfft_free(x);
    vfft_destroy(plan);
    return pass ? 0 : 1;
}
```

Build it with one command from the folder you saved it in, and run it:

```
gcc -O3 ch01_check_install.c -I<prefix>/include -L<prefix>/lib \
    -lvfft -lm -o check_install
./check_install
```

It prints:

```
impulse of 8 points: max |X[k] - 1| = 0.0e+00, tolerance 1e-12: PASS
```

If the wisdom of your CPU lacks this configuration, the first run takes a few seconds while the library measures it; Section 2.4 explains why, and Chapter 2 writes a first program step by step.

2 Your first transform

In this chapter you write a program that transforms 64 complex samples forward and back, build it with CMake against the VectorFFT you installed in Chapter 1, and run it. The program checks its own results and prints PASS or FAIL.

2.1 Know what the two directions compute

A plan for complex data, `VFFT_C2C`, runs in two directions. For N complex samples $x[n]$, run with `VFFT_FORWARD` it computes Eq. (2.1a), and run with `VFFT_BACKWARD` on a spectrum $X[k]$ it computes Eq. (2.1b):

$$\text{Forward:} \qquad X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{kn}, \qquad 0 \le k \le N-1, \tag{2.1a}$$

$$\text{Backward:} \qquad y[n] = \sum_{k=0}^{N-1} X[k]\, W_N^{-kn}, \qquad 0 \le n \le N-1, \tag{2.1b}$$

where $W_N = e^{-j2\pi/N}$ and j is the imaginary unit. The forward transform has the minus sign in its exponent. The backward transform has no factor $1/N$: run on the output of the forward transform, it returns the input times N, $y[n] = N\,x[n]$. Your program divides by N, best in a pass it makes over the data anyway.

The samples are stored interleaved, as Figure 2.1 shows: the real part of $x[n]$ at element $2n$ of the array and its imaginary part at element $2n+1$. A transform of N points reads $2N$ doubles and writes $2N$ doubles. An array of C99 `double complex` has the same layout.

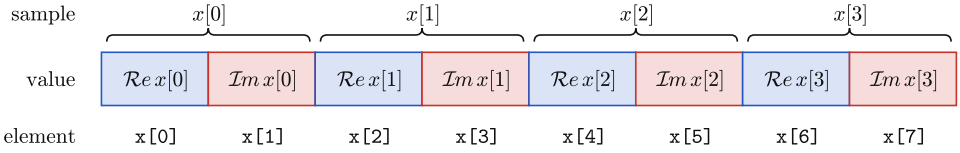


Figure 2.1 An array of $N = 4$ complex samples, interleaved. Sample $x[n]$ takes two adjacent doubles: its real part (blue) at element $2n$ and its imaginary part (red) at element $2n+1$.

2.2 Write the program

The program makes one plan for $N = 64$ points, runs it forward and backward, and checks both results: the forward one against the direct sum of Eq. (2.1a), the backward one, divided by N, against the input.

ch02_first_transform.c

```c
/* ch02_first_transform.c: a first program. It transforms N = 64 complex
 * samples forward, compares the result with the direct sum of the forward
 * transform, transforms it back and compares that with N times the input.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

#define PI 3.14159265358979323846

enum { N = 64 };
static const double TOL = 1e-12;

int main(void)
{
    /* 1. Describe the transform and create its plan. */
```

```
vfft_config_t cfg = {0};        /* zeroed: one 1D complex transform */
cfg.n[0] = N;                   /* of N points */
vfft_plan plan = vfft_create(&cfg);
if (!plan)
{
    fprintf(stderr, "vfft_create: %s\n", vfft_error());
    return 1;
}

/* 2. Allocate the buffers, sized by the plan. */
vfft_info_t info;
vfft_info(plan, &info);         /* in_doubles = out_doubles = 2 N */
double *x = vfft_malloc(info.in_doubles * sizeof(double));
double *X = vfft_malloc(info.out_doubles * sizeof(double));
double *y = vfft_malloc(info.in_doubles * sizeof(double));
if (!x || !X || !y)
{
    fprintf(stderr, "out of memory\n");
    return 1;
}

/* 3. Fill the input: Re x[n] at x[2n], Im x[n] at x[2n + 1]. */
for (int n = 0; n < N; n++)
{
    x[2 * n] = cos(0.1 * n);
    x[2 * n + 1] = sin(0.3 * n);
}

/* 4. Run the plan forward, then backward. */
vfft_execute(plan, VFFT_FORWARD, x, X);     /* X[k] */
vfft_execute(plan, VFFT_BACKWARD, X, y);    /* y[n] = N x[n] */

/* 5. Check X[k] against the direct sum over n of x[n] e^(-j2pi kn/N). */
double ferr = 0.0;
for (int k = 0; k < N; k++)
{
    double re = 0.0, im = 0.0;
    for (int n = 0; n < N; n++)
    {
        double a = -2.0 * PI * ((k * n) % N) / N;
        re += x[2 * n] * cos(a) - x[2 * n + 1] * sin(a);
        im += x[2 * n] * sin(a) + x[2 * n + 1] * cos(a);
    }
    double d = hypot(X[2 * k] - re, X[2 * k + 1] - im);
    if (d > ferr || isnan(d))
        ferr = d;
}

/* 6. Divide by N, and check y[n] / N against x[n]. */
double berr = 0.0;
for (size_t i = 0; i < info.in_doubles; i++)
{
    double d = fabs(y[i] / N - x[i]);
    if (d > berr || isnan(d))
        berr = d;
}

int pass = ferr <= TOL && berr <= TOL;      /* false for a NaN */
printf("forward:  max |X[k] - direct sum| = %.1e\n", ferr);
printf("backward: max |y[n]/N - x[n]|     = %.1e\n", berr);
printf("1D C2C of %d points, tolerance %.0e: %s\n",
       N, TOL, pass ? "PASS" : "FAIL");

/* 7. Release the buffers and the plan. */
```

```
    vfft_free(y);
    vfft_free(X);
    vfft_free(x);
    vfft_destroy(plan);
    return pass ? 0 : 1;
}
```

The numbered comments in the program mark its steps:

1. A zeroed `vfft_config_t` with `n[0]` set describes one transform of `n[0]` complex points: `VFFT_C2C`, interleaved, out of place, with its bins in natural order, on one thread. Chapter 3 describes the other fields. `vfft_create()` turns the configuration into a plan. When the library does not serve a configuration, `vfft_create()` returns `NULL` and `vfft_error()` says why; test every plan before you use it.
2. `vfft_info()` reports how many doubles the plan reads, `in_doubles`, and writes, `out_doubles`: here $2N$ each. `vfft_malloc()` allocates memory aligned the way the plans were measured. It returns `NULL` when memory runs out.
3. The program fills the input as Figure 2.1 shows.
4. `vfft_execute()` runs the plan in the direction you name, from the input buffer to the output buffer. It allocates nothing and measures nothing, so you call it as often as you need, on any buffers of the plan's size. Out of place, the input keeps its values.
5. The forward result is compared with the direct sum of Eq. (2.1a), bin by bin.
6. The backward result is divided by N and compared with the input.
7. `vfft_free()` releases what `vfft_malloc()` allocated, and `vfft_destroy()` releases the plan and everything it allocated.

Warning. Release memory from `vfft_malloc()` with `vfft_free()`, never with `free()`, and never pass memory from `malloc()` to `vfft_free()`.

2.3 Build it with CMake

Save the program as `ch02_first_transform.c` in a folder of its own, beside this `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.16)
project(first_transform LANGUAGES C)
find_package(VectorFFT CONFIG REQUIRED)
add_executable(first_transform ch02_first_transform.c)
target_link_libraries(first_transform PRIVATE VectorFFT::vfft m)
```

`VectorFFT::vfft` gives the program the header `vfft.h` and the library; `m` is the C math library, for the program's own `cos()` and `sin()`. Configure and build from that folder, with `<prefix>` the folder VectorFFT is installed in (Section 1.2):

```
cmake -S . -B build -G Ninja -DCMAKE_C_COMPILER=gcc \
      -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=<prefix>
cmake --build build
```

Chapter 1 gives the other ways to build a program against VectorFFT: inside your own CMake project, or with one compiler command.

2.4 Run it

Run the program from the folder you built it in:

```
./build/first_transform
```

In PowerShell:

```
.\build\first_transform.exe
```

The program prints:

```
forward:  max |X[k] - direct sum| = 1.1e-14
backward: max |y[n]/N - x[n]|     = 4.4e-16
1D C2C of 64 points, tolerance 1e-12: PASS
```

Both differences are rounding errors, far inside the tolerance; on your machine they can differ in the last digits.

When `vfft_create()` has no wisdom for a configuration, it measures candidate plans on your machine and keeps the fastest; the record of the winner is the configuration's wisdom. The library comes with a wisdom folder that holds one folder of wisdom for each CPU, and it picks the folder of your CPU by itself. On a CPU without shipped wisdom, the first run of this program measures for a few seconds and saves the winner into the folder of that CPU before `vfft_create()` returns; every later run is served from it at once. On a CPU whose folder holds the configuration, the first run is served at once too. A program that creates its plans at startup and reuses them pays the cost of measuring once per configuration. Chapter 10 describes planning and the wisdom folder in full.

2.5 What goes wrong

- **The result is N times too large.** The backward transform has no $1/N$, Eqs. (2.1). Divide by N in 1D, by N_1N_2 in 2D and by $N_1N_2N_3$ in 3D.
- **The bins seem to be in the wrong place.** Bin k holds frequency k for $k \le N/2$, and frequency $k - N$ for $k > N/2$; Notation lists the conventions and Section 6.1 shows them at work.
- **`vfft_create()` returns `NULL`.** The library does not serve the configuration, and `vfft_error()` says why. Section 11.1 lists the common reasons.
- **The program does not start on Windows.** It cannot find `libwinpthread-1.dll`; Section 1.7 gives the two fixes.
- **The first run is slow.** It measures what the wisdom of your CPU lacks, once per configuration (Section 2.4).

Part II **Using the API**

3 Describing a transform

You tell the library which transform to run by filling a `vfft_config_t` and passing it to `vfft_create()`. This chapter shows how to fill it: start from a zeroed configuration, set the fields that differ from their defaults, add flags, give the rank and the extents, and store a two- or three-dimensional array the way the plan reads it.

3.1 Start from a zeroed configuration

Zero every field, then set only the fields you need. A zeroed configuration with `n[0]` set describes, in the words of `vfft.h`, "one 1D C2C transform of n[0] points, interleaved, out of place, natural order, one thread, served from the library's wisdom store". In C you zero it with `= {0}`. In C++ you write `= {}`, because C++ does not convert `0` to the enumeration that is the first field (Section 5.1.1).

Table 3.1 lists the fields, what a zero means in each, and the other values, with the part of the guide that shows them in use.

Table 3.1 The fields of `vfft_config_t`.

Field	Zero means	Other values
`transform`	`VFFT_C2C`: complex in, complex out	`VFFT_R2C`: real in, half spectrum out; `VFFT_C2R`: half spectrum in, real out (Chapter 7)
`rank`	one axis; 1 means the same	2 or 3 axes (Section 3.3)
`n[4]`	no extent: always set it	the extents, `n[0]` the slowest axis (Section 3.3)
`howmany`	one transform; 1 means the same	K transforms run by one execute (Chapter 8)
`layout`	`VFFT_LAYOUT_INTERLEAVED`: a complex number is two adjacent doubles, the real part first (Notation)	none: leave it at zero
`placement`	`VFFT_PLACEMENT_OUT`: separate input and output buffers	`VFFT_PLACEMENT_IN`: one buffer, the output replacing the input (Chapter 5)
`order`	`VFFT_ORDER_NATURAL`: bin k at position k	`VFFT_ORDER_SCRAMBLED`, for `VFFT_C2C` only: the plan's own order (Section 6.2)
`batch`	`VFFT_BATCH_DEFAULT`: the default layout of a batch	`VFFT_BATCH_CONTIGUOUS`, `VFFT_BATCH_LANE_MAJOR` (Chapter 8)
`nthreads`	the calling thread only; 1 means the same	the threads the plan runs on, the caller's included (Chapter 9)
`flags`	no flags	`VFFT_FLAG_*` bits (Table 3.2)
`wisdom_dir`	`NULL`: the folder of this CPU in the library's wisdom folder (Section 10.2)	a store you name, for the cases of Section 10.3

The program `ch03_defaults.c` describes one transform twice: once zeroed with `n[0]` set, and once with every field written out at its default value. This is the second configuration:

From ch03_defaults.c

```
vfft_config_t full = {0};
full.transform = VFFT_C2C;              /* complex in, complex out */
full.rank = 1;                          /* one axis */
full.n[0] = N;                          /* its extent */
full.howmany = 1;                       /* one transform, no batch */
full.layout = VFFT_LAYOUT_INTERLEAVED;  /* (re, im) pairs */
full.placement = VFFT_PLACEMENT_OUT;    /* separate in and out */
full.order = VFFT_ORDER_NATURAL;        /* bin k at position k */
full.batch = VFFT_BATCH_DEFAULT;        /* no batch geometry named */
full.nthreads = 1;                      /* the calling thread only */
full.flags = VFFT_FLAG_PATIENT;         /* names the default planning */
full.wisdom_dir = NULL;                 /* the library's wisdom folder */
```

The program creates a plan from each configuration and runs both on the same input. The two plans report the same sizes, the forward transform agrees with the sum evaluated directly, and the two outputs are equal bit for bit:

```
in_doubles 128, out_doubles 128, nthreads 1: the same for both plans
max |X[k] - direct sum| = 8.2e-15, tolerance 1e-12
the outputs of the two plans are equal bit for bit
zeroed and written-out configurations: PASS
```

3.2 Add flags

`flags` holds `VFFT_FLAG_*` bits; combine several with the `|` operator. Table 3.2 lists them. A bit that is not one of these makes the create refuse the configuration.

Table 3.2 The flags of a configuration.

Flag	Effect
`VFFT_FLAG_PATIENT`	names the default: every create without `VFFT_FLAG_ESTIMATE` plans PATIENT, measuring a wide set of candidates on a miss, so setting the flag changes nothing (Chapter 10)
`VFFT_FLAG_RECALIBRATE`	measures the configuration again, even when the store holds its wisdom or this process has just measured it, and saves the winner (Chapter 10)
`VFFT_FLAG_DESTROY_INPUT`	lets an out-of-place plan overwrite its input (Section 5.4)
`VFFT_FLAG_OWN_PITCH`	lets an in-place 2D or 3D real plan choose its row pitch, which you then read with `vfft_info()` (Chapter 7)
`VFFT_FLAG_VERBOSE`	prints the refusals and warnings of this create to standard error as well (Chapter 11)
`VFFT_FLAG_ESTIMATE`	plans a `VFFT_C2C` transform by formulas, measuring nothing and reading no wisdom: one transform, and in 2D and 3D natural order on one thread (Section 10.6)

3.3 Give the rank and the extents

`rank` is the number of axes: 0 or 1 for a 1D transform, 2 for a 2D transform, 3 for a 3D transform. The extents go in the first `rank` entries of `n[]`. A 1D transform of N points has N in `n[0]`. A 3D transform has N_1, N_2 and N_3 in `n[0]`, `n[1]` and `n[2]`, with `n[0]` the slowest axis and the last axis contiguous in memory (Section 3.4). Leave the entries past the rank at zero.

Give every extent a value of at least 2. Lengths that are not powers of two are transformed as well. At very large lengths that are not a power of two, a create can refuse a length; `vfft_error()` then says why. An extent of zero or less is refused at once (Section 11.1 shows the message).

3.4 Store a 2D or 3D array

The plan reads a multidimensional array in row-major order, the order in which C stores an array declared `x[N1][N2][N3]`: the last axis is contiguous, and `n[0]` is the slowest. The sample $x[n_1, n_2, n_3]$ is the complex number at index

Index:
$$\ell = (n_1 N_2 + n_2)\, N_3 + n_3, \qquad 0 \le n_i \le N_i - 1, \tag{3.1}$$

of the buffer, so its real part is the double at offset 2ℓ and its imaginary part the double at offset $2\ell + 1$. A 2D array has no third axis: $\ell = n_1 N_2 + n_2$. Figure 3.1 shows a 2D array of 3×4 numbers and the order in which it is stored.

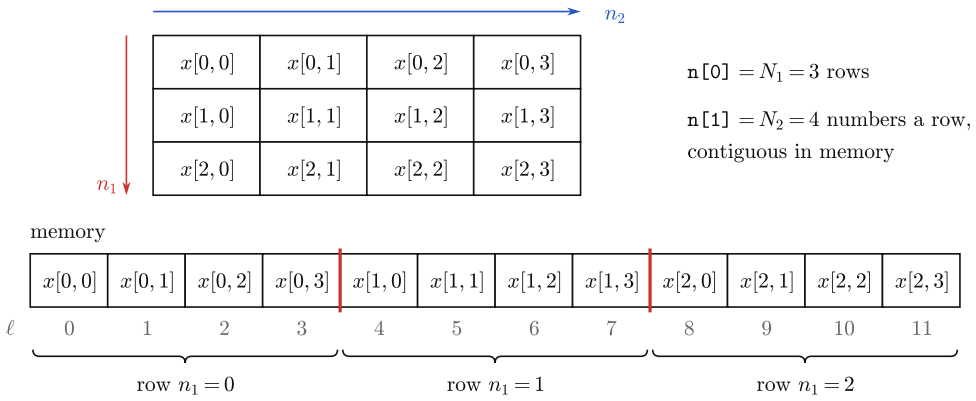


Figure 3.1 The storage of a 2D array of $N_1 \times N_2 = 3 \times 4$ complex numbers. The rows lie one after another, and inside a row n_2 runs along memory. The number $x[n_1, n_2]$ is at index $\ell = n_1 N_2 + n_2$, and each index holds two doubles, the real part first.

A complex transform in natural order writes its output the same way: bin $X[k_1, k_2, k_3]$ is at the index of Eq. (3.1) with k in place of n. A real transform halves the last axis of its spectrum (Chapter 7).

The program `ch03_index.c` fills an array of $4 \times 8 \times 16$ points through Eq. (3.1), runs a forward transform, and compares every bin, read through the same index, with the transform summed directly:

ch03_index.c

```
/* ch03_index.c: a 3D complex array of N1 x N2 x N3 points, stored row-major
 * with n[0] the slowest axis, transformed forward and checked bin by bin
 * against the transform summed directly.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

enum { N1 = 4, N2 = 8, N3 = 16 };
static const double TOL = 1e-12;

/* where element (i1, i2, i3) of the array is stored, in complex numbers */
static size_t idx(int i1, int i2, int i3)
{
    return ((size_t)i1 * N2 + i2) * N3 + i3;
}

/* bin (k1, k2, k3) of the forward transform, summed directly:
```

```
 * x[n1,n2,n3] times e^(-j 2 pi (k1 n1 / N1 + k2 n2 / N2 + k3 n3 / N3)),
 * each product k n taken modulo its N */
static void direct(const double *x, int k1, int k2, int k3,
                   double *re, double *im)
{
    const double pi = 3.14159265358979323846;
    double sr = 0.0, si = 0.0;
    for (int n1 = 0; n1 < N1; n1++)
        for (int n2 = 0; n2 < N2; n2++)
            for (int n3 = 0; n3 < N3; n3++)
            {
                double t = -2.0 * pi * ((double)(k1 * n1 % N1) / N1
                                        + (double)(k2 * n2 % N2) / N2
                                        + (double)(k3 * n3 % N3) / N3);
                size_t l = idx(n1, n2, n3);
                sr += x[2 * l] * cos(t) - x[2 * l + 1] * sin(t);
                si += x[2 * l] * sin(t) + x[2 * l + 1] * cos(t);
            }
    *re = sr;
    *im = si;
}

int main(void)
{
    vfft_config_t cfg = {0};
    cfg.rank = 3;
    cfg.n[0] = N1;          /* the slowest axis */
    cfg.n[1] = N2;
    cfg.n[2] = N3;          /* the contiguous axis */
    vfft_plan plan = vfft_create(&cfg);
    if (!plan)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }

    vfft_info_t info;
    vfft_info(plan, &info);
    double *x = vfft_malloc(info.in_doubles * sizeof(double));
    double *X = vfft_malloc(info.out_doubles * sizeof(double));
    if (!x || !X)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }

    for (int n1 = 0; n1 < N1; n1++)
        for (int n2 = 0; n2 < N2; n2++)
            for (int n3 = 0; n3 < N3; n3++)
            {
                size_t l = idx(n1, n2, n3);
                /* the real part at 2 l, the imaginary part at 2 l + 1 */
                x[2 * l] = cos(1.7 * n1 + 0.6 * n2 * n2 + 0.3 * n3 * n3);
                x[2 * l + 1] = sin(2.3 * n1 * n2 + 0.9 * n3);
            }

    vfft_execute(plan, VFFT_FORWARD, x, X);

    double err = 0.0;
    for (int k1 = 0; k1 < N1; k1++)
        for (int k2 = 0; k2 < N2; k2++)
            for (int k3 = 0; k3 < N3; k3++)
            {
                double re, im;
```

```
                direct(x, k1, k2, k3, &re, &im);
                size_t m = idx(k1, k2, k3);       /* the same index map */
                double d = hypot(X[2 * m] - re, X[2 * m + 1] - im);
                if (d > err || isnan(d))
                    err = d;
            }
    int pass = err <= TOL;
    printf("3D transform, n[0] = %d, n[1] = %d, n[2] = %d\n", N1, N2, N3);
    printf("max |X[k1,k2,k3] - direct sum| = %.1e, tolerance %.0e: %s\n",
           err, TOL, pass ? "PASS" : "FAIL");

    vfft_free(X);
    vfft_free(x);
    vfft_destroy(plan);
    return pass ? 0 : 1;
}
```

It prints:

```
3D transform, n[0] = 4, n[1] = 8, n[2] = 16
max |X[k1,k2,k3] - direct sum| = 1.3e-13, tolerance 1e-12: PASS
```

If you give the extents in the opposite order, with the contiguous axis in `n[0]`, the plan reads your doubles as an array of another shape, and the output is not the transform of your array. An array stored column-major, as Fortran stores one, has its first index contiguous: give its extents from the last index to the first, and read its bin $[k_1, k_2, k_3]$ as bin $[k_3, k_2, k_1]$ of the plan.

3.5 See what the library serves

Before you fill a configuration, see whether this release serves it. Figure 3.2 shows the nine transforms of this release, `VFFT_C2C`, `VFFT_R2C` and `VFFT_C2R` in one, two and three dimensions, and for each what you can ask of it: in place or out of place, the scrambled order, a contiguous or a lane-major batch, and more than one thread. A configuration the figure refuses makes `vfft_create()` return `NULL`, and `vfft_error()` says why (Chapter 11); the chapters named in Table 3.1 show each served configuration at work.

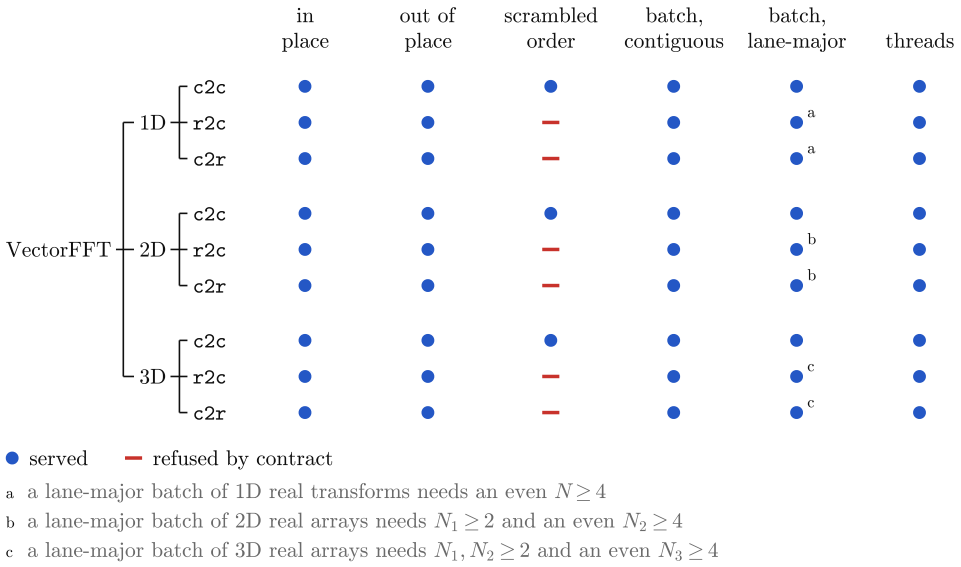


Figure 3.2 What this release serves, by transform and rank: `VFFT_C2C` (c2c), `VFFT_R2C` (r2c) and `VFFT_C2R` (c2r) in one, two and three dimensions, each with the two placements, `VFFT_ORDER_SCRAMBLED`, the two batch geometries and threads. A blue dot is a configuration `vfft_create()` serves; a red dash is one it refuses by contract. A real transform is natural by construction, so the scrambled order is for `VFFT_C2C` only. A letter beside a dot names a condition written under the tree.

4 Plans: create, execute, destroy

A plan does the work. You create it once from a configuration, execute it as often as you need, and destroy it when you are done. This chapter shows the three calls and what the first create of a configuration costs.

4.1 Create a plan

`vfft_create()` takes your configuration and returns a `vfft_plan`, a handle to a plan that is ready to execute; Figure 4.1 shows the life of a plan from this call to `vfft_destroy()`. In that one call the library

1. checks the configuration, and refuses a configuration it does not serve;
2. looks the configuration up in the store, and on a miss measures the candidates and saves the winner;
3. builds the plan: its tables, its scratch memory and its threads are all set up here.

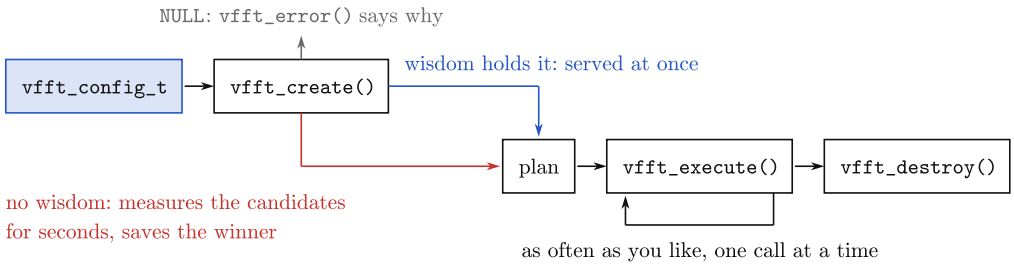


Figure 4.1 The life of a plan. A create is served at once when the store holds wisdom for the configuration (blue), and otherwise measures the candidates for seconds and saves the winner (red); the plan then serves any number of executes, one call at a time, until `vfft_destroy()`. A refused create returns `NULL`, and `vfft_error()` says why.

The plan owns what it allocated until you destroy it. Every create can be refused, so test the plan before you use it:

From ch04_reuse.c

```
    vfft_config_t cfg = {0};
    cfg.n[0] = N;
    vfft_plan plan = vfft_create(&cfg);     /* once, before any data */
    if (!plan)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }
```

A refused create returns `NULL`, and `vfft_error()` says why: read the reason at once, on the thread that made the call. Section 11.1 shows the reasons you meet most often.

A create also turns on flush-to-zero and denormals-are-zero on the calling thread and leaves them on: from then on, floating-point arithmetic on that thread, your program's own included, treats subnormal numbers, those smaller in magnitude than about 2.2×10^{-308}, as zero.

4.2 Plan for the cost of the first create

A create whose configuration the store holds is served at once; one that misses measures the candidates first, for seconds, up to about a minute for some configurations, and saves the winner in the store before it returns, so that every later

create of that configuration is served, in the same run and in later ones (Figure 4.1; Section 10.1 gives the times). So create your plans once, at startup, and keep them for as long as you transform; never create a plan inside a loop that has to be fast; and install the library to a prefix your programs can write, so that the winner one run saves in the library's wisdom folder serves the next (Section 10.2).

4.3 Execute a plan

`vfft_execute()` runs a plan on your buffers. It takes the plan, the direction and two buffers, the input `in` and the output `out`. The direction is `VFFT_FORWARD` or `VFFT_BACKWARD`: a `VFFT_C2C` plan runs both, a `VFFT_R2C` plan runs `VFFT_FORWARD` only, and a `VFFT_C2R` plan runs `VFFT_BACKWARD` only. The backward transform is not normalized: a forward transform followed by a backward one returns the input times N, or times N_1N_2 and $N_1N_2N_3$ at rank 2 and 3, and you divide (Chapter 2).

An execute allocates nothing, measures nothing and decides nothing, because the create settled everything. It is the call for your time-critical loop. One plan serves any number of executes, on any buffers of the sizes `vfft_info()` gives (Chapter 5). The program `ch04_reuse.c` creates one plan and runs it eight times, forward and back on four blocks of a signal, each block in buffers of its own:

From ch04_reuse.c

```
    double err = 0.0;
    for (int b = 0; b < BLOCKS; b++)
    {
        vfft_execute(plan, VFFT_FORWARD, x[b], X[b]);
        vfft_execute(plan, VFFT_BACKWARD, X[b], y[b]);  /* N x[b] */
        double e = 0.0;
        for (size_t i = 0; i < info.in_doubles; i++)
        {
            double d = fabs(y[b][i] / N - x[b][i]);
            if (d > e || isnan(d))
                e = d;
        }
        printf("block %d: max |y/N - x| = %.1e\n", b, e);
        if (e > err || isnan(e))
            err = e;
    }
```

```
block 0: max |y/N - x| = 5.6e-16
block 1: max |y/N - x| = 6.7e-16
block 2: max |y/N - x| = 4.4e-16
block 3: max |y/N - x| = 6.7e-16
one plan, 8 executes, tolerance 1e-12: PASS
```

Make every call fit its plan: a plan from a successful create, buffers sized from `vfft_info()`, a direction the transform runs, and the buffers the placement names (Section 5.3). The library refuses a call with a `NULL` plan or buffer, a direction the transform does not run, or two buffers for an in-place real plan: such a call computes nothing and leaves the output as it was (Section 11.2). Nothing checks the size of a buffer, so make each one the size `vfft_info()` gives (Section 5.2).

4.4 Execute a plan from one thread at a time

Never execute the same plan from two threads at once: give each thread that transforms a plan of its own. A plan with `nthreads` greater than 1 runs on several threads by itself, from one call. Chapter 9 shows both, and gives the rules for calling the library from threads of your own.

4.5 Destroy a plan

`vfft_destroy()` frees a plan and everything it allocated. It leaves your buffers alone: release those with `vfft_free()` (Chapter 5). Destroy a plan only when no thread is executing it, and do not use it afterwards. `vfft_destroy()` accepts `NULL` and then does nothing, so a plan whose create was refused needs no special case.

This is the whole of `ch04_reuse.c`, from the create to the destroy:

ch04_reuse.c

```
/* ch04_reuse.c: one plan, created once, transforms four blocks of a signal
 * forward and back, each block in buffers of its own, and is destroyed at
 * the end. Every block must come back as itself times N.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

enum { N = 256, BLOCKS = 4 };
static const double TOL = 1e-12;

int main(void)
{
    vfft_config_t cfg = {0};
    cfg.n[0] = N;
    vfft_plan plan = vfft_create(&cfg);     /* once, before any data */
    if (!plan)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }

    vfft_info_t info;
    vfft_info(plan, &info);
    double *x[BLOCKS], *X[BLOCKS], *y[BLOCKS];
    for (int b = 0; b < BLOCKS; b++)
    {
        x[b] = vfft_malloc(info.in_doubles * sizeof(double));
        X[b] = vfft_malloc(info.out_doubles * sizeof(double));
        y[b] = vfft_malloc(info.in_doubles * sizeof(double));
        if (!x[b] || !X[b] || !y[b])
        {
            fprintf(stderr, "out of memory\n");
            return 1;
        }
        for (int n = 0; n < N; n++)         /* block b of the signal */
        {
            double t = (double)(b * N + n);
            x[b][2 * n] = cos(0.01 * t) + 0.2 * cos(0.9 * t);
            x[b][2 * n + 1] = sin(0.01 * t);
        }
    }

    double err = 0.0;
    for (int b = 0; b < BLOCKS; b++)
    {
        vfft_execute(plan, VFFT_FORWARD, x[b], X[b]);
        vfft_execute(plan, VFFT_BACKWARD, X[b], y[b]);  /* N x[b] */
        double e = 0.0;
        for (size_t i = 0; i < info.in_doubles; i++)
        {
            double d = fabs(y[b][i] / N - x[b][i]);
            if (d > e || isnan(d))
                e = d;
```

```
        }
        printf("block %d: max |y/N - x| = %.1e\n", b, e);
        if (e > err || isnan(e))
            err = e;
    }
    int pass = err <= TOL;
    printf("one plan, %d executes, tolerance %.0e: %s\n",
           2 * BLOCKS, TOL, pass ? "PASS" : "FAIL");

    vfft_destroy(plan);                     /* the plan and its memory */
    for (int b = 0; b < BLOCKS; b++)        /* the buffers stay yours */
    {
        vfft_free(y[b]);
        vfft_free(X[b]);
        vfft_free(x[b]);
    }
    return pass ? 0 : 1;
}
```

5 Buffers and placement

The buffers a plan reads and writes are yours: you allocate them, fill them and free them. This chapter shows how to allocate them with `vfft_malloc()`, how large to make them, and how to pass them to a plan that works in place or out of place.

5.1 Allocate with `vfft_malloc()`

`vfft_malloc()` takes a size in bytes and returns a block of memory aligned to `vfft_info().alignment` bytes, 64 in this release: the alignment every plan was measured on. Allocate the buffers you pass to `vfft_execute()` with it. The block is not initialized, so fill it before a transform reads it. Test the pointer: `vfft_malloc()` returns `NULL` when memory runs out. Release the block with `vfft_free()` only, never with `free()`; `vfft_free()` accepts `NULL`.

Every program in this guide allocates its buffers the same way: it creates the plan, asks `vfft_info()` for the sizes, and allocates them. These are the lines of `ch05_sizes.c` that do it for a `VFFT_R2C` plan:

From ch05_sizes.c

```
vfft_config_t cfg = {0};
cfg.transform = VFFT_R2C;
cfg.n[0] = N;
vfft_plan plan = vfft_create(&cfg);
if (!plan)
{
    fprintf(stderr, "vfft_create: %s\n", vfft_error());
    return 1;
}
vfft_info_t info;
vfft_info(plan, &info);
double *x = vfft_malloc(info.in_doubles * sizeof(double));
double *X = vfft_malloc(info.out_doubles * sizeof(double));
if (!x || !X)
{
    fprintf(stderr, "out of memory\n");
    return 1;
}
```

`vfft_info()` called with a `NULL` plan fills in the library's own facts, `alignment` among them. The program checks its two blocks against it:

From ch05_sizes.c

```
vfft_info_t lib;
vfft_info(NULL, &lib);                  /* the library's facts */
int aligned = (uintptr_t)x % lib.alignment == 0
              && (uintptr_t)X % lib.alignment == 0;
```

5.1.1 From C++

`vfft.h` declares its functions for C++ as well, and two things differ from C: you zero the configuration with `= {}`, and you cast the `void *` that `vfft_malloc()` returns. The start of `ch05_cxx.cpp` shows both:

From ch05_cxx.cpp

```
vfft_config_t cfg = {};                 /* zeroed: C++ rejects {0} */
cfg.n[0] = N;
vfft_plan plan = vfft_create(&cfg);
if (!plan)
{
```

```
        std::fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }
    vfft_info_t info;
    vfft_info(plan, &info);
    double *x = static_cast<double *>(
        vfft_malloc(info.in_doubles * sizeof(double)));
    double *X = static_cast<double *>(
        vfft_malloc(info.out_doubles * sizeof(double)));
    double *y = static_cast<double *>(
        vfft_malloc(info.in_doubles * sizeof(double)));
```

The program then runs the same forward and backward transform as a C program would:

```
C++: roundtrip of 64 points: max |y/N - x| = 5.6e-16, tolerance 1e-12: PASS
```

5.2 Size buffers from `vfft_info()`

`in_doubles` is the number of doubles an execute reads, and `out_doubles` the number it writes. They count doubles, two for each complex number. Table 5.1 gives them for one transform. In it, P is the number of points, N, N_1N_2 or $N_1N_2N_3$, and H is the number of complex bins a real transform keeps, $N/2+1$, $N_1(N_2/2+1)$ or $N_1N_2(N_3/2+1)$, in C integer division (Chapter 7).

Table 5.1 The doubles one execute reads and writes.

Plan	`in_doubles`	`out_doubles`
`VFFT_C2C`	$2P$	$2P$
`VFFT_R2C`, out of place	P	$2H$
`VFFT_C2R`, out of place	$2H$	P
`VFFT_R2C` or `VFFT_C2R`, in place, 1D	$2H$	$2H$
`VFFT_R2C` or `VFFT_C2R`, in place, 2D or 3D	rows times `stride`	rows times `stride`

An in-place 2D or 3D real plan pads each row of the array; `vfft_info()` gives the doubles from one row to the next as `stride`, and the rows are N_1 in 2D and N_1N_2 in 3D (Chapter 7). A batch of K transforms reads and writes K times as much as one (Chapter 8).

The program `ch05_sizes.c` prints the sizes of eight plans and checks each against its row of Table 5.1:

```
vfft_malloc() blocks start on 64-byte boundaries: yes

plan                         in_doubles  out_doubles
1D C2C, N = 64                      128          128
1D R2C, N = 64                       64           66
1D C2R, N = 64                       66           64
1D R2C in place, N = 64              66           66
2D C2C, 16 x 32                    1024         1024
2D R2C, 16 x 32                     512          544
3D C2C, 4 x 8 x 16                 1024         1024
3D R2C, 4 x 8 x 16                  512          576

every size as its formula gives: PASS
```

Read the sizes from `vfft_info()` rather than computing them: the padding of an in-place 2D or 3D real plan is the plan's, and with `VFFT_FLAG_OWN_PITCH` the plan chooses it (Chapter 7). A buffer smaller than the size `vfft_info()` gives makes an execute read or write past its end.

5.3 Transform in place or out of place

`placement` decides whether the output replaces the input:

- `VFFT_PLACEMENT_OUT`, the default: you pass two separate buffers, `in` and `out`. The plan writes `out` and leaves `in` as it was.
- `VFFT_PLACEMENT_IN`: you pass one buffer twice, as `in` and as `out`. The output replaces the input, and the transform needs one buffer instead of two. An in-place plan reports equal `in_doubles` and `out_doubles`; for a real transform that buffer is larger than the real data (Chapter 7).

Figure 5.1 shows both placements, and the out-of-place plan that may overwrite its input (Section 5.4).

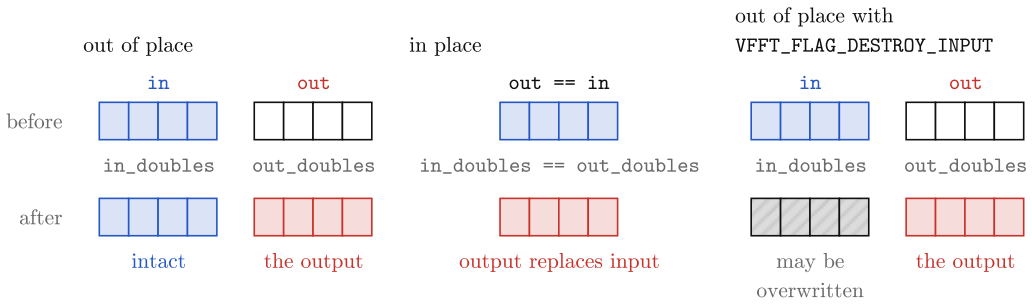


Figure 5.1 The three placements, before and after the call. Out of place, the plan reads the input buffer (blue), writes the output buffer (red) and leaves the input intact. In place, one buffer holds the input before the call and the output after it. Out of place with `VFFT_FLAG_DESTROY_INPUT`, the plan may also overwrite the input.

The program `ch05_placement.c` creates the same transform both ways:

From ch05_placement.c

```
    vfft_config_t cfg = {0};
    cfg.n[0] = N;                           /* VFFT_PLACEMENT_OUT */
    vfft_plan outp = vfft_create(&cfg);
    if (!outp)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }
    cfg.placement = VFFT_PLACEMENT_IN;
    vfft_plan inp = vfft_create(&cfg);
    if (!inp)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }
```

It runs each plan on a copy of one signal, the out-of-place plan with two buffers and the in-place plan with one:

From ch05_placement.c

```
    vfft_execute(outp, VFFT_FORWARD, x, X);    /* two buffers */
    vfft_execute(inp, VFFT_FORWARD, z, z);     /* one buffer, twice */
```

The out-of-place plan kept its input, and the two plans computed the same spectrum:

```
out of place: the input is unchanged
in place: the spectrum replaced the signal, max |z[k] - X[k]| = 0.0e+00
tolerance 1e-12: PASS
```

Pass exactly the buffers the placement names: out of place, two blocks that do not overlap; in place, the same pointer twice.

5.4 Let an out-of-place plan overwrite its input

An out-of-place plan keeps its input as it was, which can cost it memory or time. When your program does not read the input again after the call, set `VFFT_FLAG_DESTROY_INPUT`: the plan may then overwrite its input. A typical case is a `VFFT_C2R` plan that turns a spectrum you no longer need back into data. The program `ch05_destroy_input.c` transforms a 3D real array forward and back, the backward plan with the flag set:

ch05_destroy_input.c

```
/* ch05_destroy_input.c: a 3D real array transformed forward out of place,
 * then back with a VFFT_C2R plan that may overwrite its input; the result
 * must be the array times N1 N2 N3.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

enum { N1 = 4, N2 = 8, N3 = 16 };
static const double TOL = 1e-12;

int main(void)
{
    vfft_config_t cfg = {0};
    cfg.transform = VFFT_R2C;
    cfg.rank = 3;
    cfg.n[0] = N1;
    cfg.n[1] = N2;
    cfg.n[2] = N3;
    vfft_plan r2c = vfft_create(&cfg);
    if (!r2c)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }
    cfg.transform = VFFT_C2R;
    cfg.flags = VFFT_FLAG_DESTROY_INPUT;    /* X is not read again */
    vfft_plan c2r = vfft_create(&cfg);
    if (!c2r)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }

    vfft_info_t fwd, bwd;
    vfft_info(r2c, &fwd);
    vfft_info(c2r, &bwd);
    double *x = vfft_malloc(fwd.in_doubles * sizeof(double));
    double *X = vfft_malloc(fwd.out_doubles * sizeof(double));
    double *y = vfft_malloc(bwd.out_doubles * sizeof(double));
    if (!x || !X || !y)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (size_t l = 0; l < fwd.in_doubles; l++)
        x[l] = sin(0.37 * l) + 0.5 * cos(0.05 * l * l);

    vfft_execute(r2c, VFFT_FORWARD, x, X);
```

```
    vfft_execute(c2r, VFFT_BACKWARD, X, y);    /* X may be overwritten */

    const double scale = 1.0 / ((double)N1 * N2 * N3);
    double err = 0.0;
    for (size_t l = 0; l < bwd.out_doubles; l++)
    {
        double d = fabs(y[l] * scale - x[l]);
        if (d > err || isnan(d))
            err = d;
    }
    int pass = err <= TOL;
    printf("3D real roundtrip of %d x %d x %d points, "
           "VFFT_FLAG_DESTROY_INPUT set\n", N1, N2, N3);
    printf("max |y/(N1 N2 N3) - x| = %.1e, tolerance %.0e: %s\n",
           err, TOL, pass ? "PASS" : "FAIL");

    vfft_free(y);
    vfft_free(X);
    vfft_free(x);
    vfft_destroy(c2r);
    vfft_destroy(r2c);
    return pass ? 0 : 1;
}
```

It prints:

```
3D real roundtrip of 4 x 8 x 16 points, VFFT_FLAG_DESTROY_INPUT set
max |y/(N1 N2 N3) - x| = 5.0e-16, tolerance 1e-12: PASS
```

After such an execute, the input buffer holds nothing you can use; fill it again before a transform reads it. The flag changes where the plan may work, not what it computes.

6 Complex transforms in one, two and three dimensions

You transform complex data with a plan whose `transform` is `VFFT_C2C`, the value a zeroed configuration has. Executed with `VFFT_FORWARD`, the plan computes Eq. (2.1a); executed with `VFFT_BACKWARD`, it computes Eq. (2.1b), which has no factor $1/N$. In this chapter you find the frequency each bin holds, let a plan keep the bins in an order of its own, and transform arrays of two and three dimensions.

6.1 Find the frequency of a bin

A plan writes the bins in natural order, `VFFT_ORDER_NATURAL`, the value a zeroed configuration has: bin k at position k, for k from 0 to $N-1$. A complex exponential $e^{j2\pi fn/N}$ of frequency f transforms to N in one bin and 0 in the others, and the bins above $N/2$ hold the negative frequencies (Figure 6.1):

- bin $k \le N/2$ holds frequency k;
- bin $k > N/2$ holds frequency $k - N$;
- at an even N, bin $N/2$ holds both $N/2$ and $-N/2$.

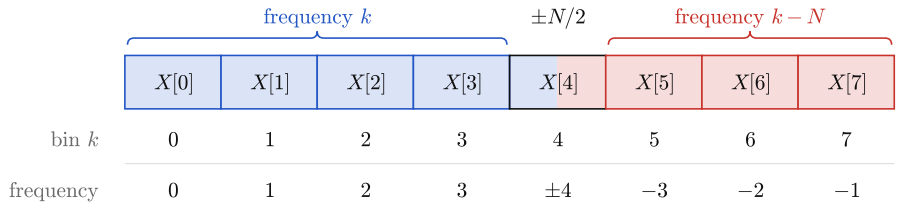


Figure 6.1 The bins of a `VFFT_C2C` spectrum of $N = 8$ points in natural order. Bins 0 to 3 (blue) hold the frequencies 0 to 3, bins 5 to 7 (red) the frequencies -3 to -1, and bin 4 (half of each) holds both 4 and -4.

The program `ch06_frequencies.c` transforms the sum of two exponentials of $N = 64$ samples, frequency 5 with amplitude 1 and frequency -3 with amplitude $1/2$. It checks that bin 5 holds N, bin $64 - 3 = 61$ holds $N/2$ and every other bin 0, then runs the backward transform and checks that it returns N times the input:

ch06_frequencies.c

```
/* ch06_frequencies.c: a complex signal of frequencies +5 and -3 through a 1D
 * VFFT_C2C plan of N = 64 points, forward and back. Each frequency lands in
 * the bin that frequency() names, at N times its amplitude.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

#define PI 3.14159265358979323846

enum { N = 64 };
static const double TOL = 1e-12;

/* the frequency that bin k of an n-point spectrum holds */
static int frequency(int k, int n)
{
    return k <= n / 2 ? k : k - n;
}

int main(void)
{
```

```
    vfft_config_t cfg = {0};        /* 1D VFFT_C2C, out of place, natural */
    cfg.n[0] = N;
    vfft_plan plan = vfft_create(&cfg);
    if (!plan)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }

    vfft_info_t info;
    vfft_info(plan, &info);
    double *x = vfft_malloc(info.in_doubles * sizeof(double));
    double *X = vfft_malloc(info.out_doubles * sizeof(double));
    double *y = vfft_malloc(info.in_doubles * sizeof(double));
    if (!x || !X || !y)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }

    /* x[n] = e^(j 2 pi 5 n / N) + 0.5 e^(-j 2 pi 3 n / N) */
    for (int n = 0; n < N; n++)
    {
        double a = 2 * PI * 5 * n / N, b = -2 * PI * 3 * n / N;
        x[2 * n] = cos(a) + 0.5 * cos(b);           /* Re x[n] */
        x[2 * n + 1] = sin(a) + 0.5 * sin(b);       /* Im x[n] */
    }

    vfft_execute(plan, VFFT_FORWARD, x, X);

    /* expected: N at frequency +5, N/2 at frequency -3, 0 elsewhere */
    double err = 0.0;
    for (int k = 0; k < N; k++)
    {
        int f = frequency(k, N);
        double want = f == 5 ? N : f == -3 ? 0.5 * N : 0.0;
        double d = hypot(X[2 * k] - want, X[2 * k + 1]);
        if (d > err || isnan(d))
            err = d;
        if (want != 0.0)
            printf("bin %d holds frequency %+d: Re X[%d] = %.3f\n",
                   k, f, k, X[2 * k]);
    }

    vfft_execute(plan, VFFT_BACKWARD, X, y);        /* y = N x */
    double back = 0.0;
    for (size_t i = 0; i < info.in_doubles; i++)
    {
        double d = fabs(y[i] / N - x[i]);
        if (d > back || isnan(d))
            back = d;
    }

    int pass = err <= TOL && back <= TOL;           /* false for a NaN */
    printf("spectrum: max |X[k] - expected| = %.1e\n", err);
    printf("roundtrip: max |y[n]/N - x[n]| = %.1e\n", back);
    printf("tolerance %.0e: %s\n", TOL, pass ? "PASS" : "FAIL");

    vfft_free(y);
    vfft_free(X);
    vfft_free(x);
    vfft_destroy(plan);
    return pass ? 0 : 1;
}
```

It prints:

```
bin 5 holds frequency +5: Re X[5] = 64.000
bin 61 holds frequency -3: Re X[61] = 32.000
spectrum: max |X[k] - expected| = 4.3e-14
roundtrip: max |y[n]/N - x[n]| = 4.4e-16
tolerance 1e-12: PASS
```

The function `frequency()` is the rule of the list in code. Use it wherever your program treats a bin by its frequency, as a filter or a derivative does. Two mistakes give wrong frequencies. Reading bin $k > N/2$ as frequency k turns every negative frequency into a high positive one. And a formula from a text that puts $e^{+j2\pi kn/N}$ in the forward transform expects frequency f in bin $N - f$; with Eq. (2.1a), frequency 5 is in bin 5, not in bin 59.

6.2 Leave the bins in the plan's own order

With `order` set to `VFFT_ORDER_SCRAMBLED`, a `VFFT_C2C` plan writes the bins in an order of its own. It can be faster than natural order. The order is not documented, and only the backward transform of the same plan reads it back. Ask for it only when nothing your program does between the forward and the backward transform depends on where a bin is, such as a product of two spectra of the same plan, bin by bin. A `VFFT_R2C` or `VFFT_C2R` plan refuses it: a real spectrum is in natural order (Chapter 7).

6.3 Transform a 2D or 3D array

Set `rank` to 2 or 3 and the extents in `n[0]`, `n[1]` and `n[2]`, `n[0]` the slowest axis and the last extent the contiguous one, and store the array row-major, as the C array `double complex x[N1][N2][N3]` is: sample $x[n_1, n_2, n_3]$ is element ℓ of Eq. (3.1) (Section 3.4). A plan of rank 3 runs Eq. (2.1a) along every axis:

$$\text{Forward:} \qquad X[k_1, k_2, k_3] = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} \sum_{n_3=0}^{N_3-1} x[n_1, n_2, n_3]\, W_{N_1}^{k_1 n_1} W_{N_2}^{k_2 n_2} W_{N_3}^{k_3 n_3}, \tag{6.1}$$

and writes bin $X[k_1, k_2, k_3]$ to element $(k_1 N_2 + k_2)N_3 + k_3$, the place Eq. (3.1) gives with k for n. Each axis follows the rule of Section 6.1 on its own: bin $k_i \le N_i/2$ holds frequency k_i, and bin $k_i > N_i/2$ holds $k_i - N_i$. A plan of rank 2 computes the same sum without n_3 and k_3. The backward transform uses the conjugate kernels, $W_{N_i}^{-k_i n_i}$, and returns the input times $N_1 N_2 N_3$, or $N_1 N_2$ in 2D (Table 6.1).

Table 6.1 A `VFFT_C2C` plan at each rank: the extents in `n[]`, the doubles in each buffer, and the factor a forward and a backward transform multiply the input by.

`rank`	`n[]`	`in_doubles`, `out_doubles`	**Factor**
0 or 1	N	$2N$	N
2	N_1, N_2	$2N_1N_2$	N_1N_2
3	N_1, N_2, N_3	$2N_1N_2N_3$	$N_1N_2N_3$

6.3.1 A 2D array out of place

The program `ch06_array_2d.c` transforms an array of 16×32 samples out of place, compares every bin with the sum of Eq. (6.1) in 2D, and checks the roundtrip. Its plan names the rank and the two extents:

From ch06_array_2d.c

```
vfft_config_t cfg = {0};        /* VFFT_C2C, out of place, natural */
cfg.rank = 2;
cfg.n[0] = N1;                  /* N1 rows: the slow axis */
cfg.n[1] = N2;                  /* N2 columns: the contiguous axis */
vfft_plan plan = vfft_create(&cfg);
if (!plan)
{
    fprintf(stderr, "vfft_create: %s\n", vfft_error());
    return 1;
}
```

and it fills the input row by row, at the element Eq. (3.1) gives:

From ch06_array_2d.c

```
for (int n1 = 0; n1 < N1; n1++)
    for (int n2 = 0; n2 < N2; n2++)
    {
        size_t l = (size_t)n1 * N2 + n2;        /* row-major */
        x[2 * l] = cos(0.3 * n1 - 0.1 * n2 * n2);     /* Re x[n1,n2] */
        x[2 * l + 1] = sin(0.2 * n1 * n2 + 1.0);      /* Im x[n1,n2] */
    }
```

It prints:

```
2D VFFT_C2C of 16 x 32 points: in_doubles 1024, out_doubles 1024
forward: max |X - direct sum| = 7.1e-14
roundtrip: max |y/(N1 N2) - x| = 5.6e-16
tolerance 1e-12: PASS
```

6.3.2 A 3D array in place

In place, you set `placement` to `VFFT_PLACEMENT_IN`, allocate one buffer of `in_doubles` doubles, and pass it to `vfft_execute()` as both input and output; the spectrum replaces the samples. The program `ch06_volume_in_place.c` transforms the plane wave $e^{j2\pi(f_1 n_1/N_1 + f_2 n_2/N_2 + f_3 n_3/N_3)}$ of the frequencies $(f_1, f_2, f_3) = (3, -5, 7)$ on an $8 \times 16 \times 32$ array in place:

From ch06_volume_in_place.c

```
vfft_config_t cfg = {0};
cfg.rank = 3;
cfg.n[0] = N1;                  /* the slowest axis */
cfg.n[1] = N2;
cfg.n[2] = N3;                  /* the contiguous axis */
cfg.placement = VFFT_PLACEMENT_IN;
vfft_plan plan = vfft_create(&cfg);
if (!plan)
{
    fprintf(stderr, "vfft_create: %s\n", vfft_error());
    return 1;
}
```

It stores sample $[n_1, n_2, n_3]$ at the element of Eq. (3.1),

From ch06_volume_in_place.c

```
/* where x[n1,n2,n3] is stored: element l, doubles 2 l and 2 l + 1 */
static size_t element(int n1, int n2, int n3)
{
    return ((size_t)n1 * N2 + n2) * N3 + n3;
}
```

runs the forward transform in the buffer,

From ch06_volume_in_place.c

```
    vfft_execute(plan, VFFT_FORWARD, z, z);         /* the spectrum over x */
```

and finds the wave in the bin of its frequencies, the negative one wrapped to $16 - 5 = 11$:

From ch06_volume_in_place.c

```
    /* frequency f of an axis of n points is bin f, or n + f when f < 0 */
    int k1 = F1 < 0 ? N1 + F1 : F1;
    int k2 = F2 < 0 ? N2 + F2 : F2;
    int k3 = F3 < 0 ? N3 + F3 : F3;
    size_t peak = element(k1, k2, k3);
```

The bin holds $N_1N_2N_3 = 4096$, every other bin 0, and the backward transform in the same buffer returns $N_1N_2N_3$ times the wave:

```
3D VFFT_C2C in place, 8 x 16 x 32 points: one buffer of 8192 doubles
frequency (3, -5, 7) is bin [3, 11, 7], element 1895: Re X = 4096.000
spectrum: max |X/(N1 N2 N3) - expected| = 5.4e-16
roundtrip: max |y/(N1 N2 N3) - x| = 7.2e-16
tolerance 1e-12: PASS
```

Two mistakes show at once in such a check. Extents given in the wrong order, the contiguous one first, describe another array: on an array that is not a cube, the wave then spreads over many bins. And a roundtrip that comes back $N_1N_2N_3$ times too large is the missing division of Table 6.1. Chapter 15 propagates a wave packet with a plan of this kind.

7 Real transforms

When your samples are real, a `VFFT_R2C` plan computes their spectrum and keeps only the half that the other half repeats, and a `VFFT_C2R` plan turns such a half spectrum back into real samples. In this chapter you read the half spectrum, run the pair out of place and in place, and transform real arrays of two and three dimensions.

7.1 Read the half spectrum

For a real sequence $x[n]$, the bins of Eq. (2.1a) above $N/2$ are the conjugates of the bins below it:

$$X[N-k] = X^*[k], \qquad 1 \le k \le N-1. \tag{7.1}$$

A `VFFT_R2C` plan therefore stores bins 0 to $\lfloor N/2 \rfloor$ only: $\lfloor N/2 \rfloor + 1$ bins, `N/2 + 1` in C, written as $2(\lfloor N/2 \rfloor + 1)$ doubles in (re, im) pairs, as a complex spectrum is. Figure 7.1 shows which bins are stored for an even and an odd N, and the frequency each holds:

- bin k holds frequency k, for $0 \le k \le \lfloor N/2 \rfloor$;
- the negative frequency $-k$ sits in bin $N-k$, which is not stored: by Eq. (7.1) its value is the conjugate of bin k;
- bin 0 is real, and at an even N so is bin $N/2$, which holds both $N/2$ and $-N/2$.



Figure 7.1 The bins of the transform of a real sequence of $N = 8$ and of $N = 7$ samples. A `VFFT_R2C` plan stores bins 0 to $\lfloor N/2 \rfloor$ (blue); each bin above $\lfloor N/2 \rfloor$ (red outline) is the conjugate of a stored bin and is not stored.

7.2 Filter real samples out of place

You create two plans from one configuration: `VFFT_R2C` for the forward transform and `VFFT_C2R` for the backward one. A `VFFT_R2C` plan runs with `VFFT_FORWARD` only and a `VFFT_C2R` plan with `VFFT_BACKWARD` only; the other direction is refused and writes nothing. The `VFFT_C2R` plan computes Eq. (2.1b) over the whole spectrum, the bins it does not read taken from Eq. (7.1), so the pair returns your input times N, as a complex pair does.

Out of place, a `VFFT_R2C` plan reads the reals and writes the bins, two doubles each, and a `VFFT_C2R` plan reads the bins and writes the reals: `vfft_info()` reports these sizes in doubles as `in_doubles` and `out_doubles`. Table 7.1 gives them at each rank.

Table 7.1 The reals and the bins of a real transform at each rank, and the doubles of the one buffer of an in-place pair; s is the row pitch of Section 7.6.

rank	Reals	Bins	In place
0 or 1	N	$\lfloor N/2 \rfloor + 1$	$2(\lfloor N/2 \rfloor + 1)$
2	$N_1 N_2$	$N_1(\lfloor N_2/2 \rfloor + 1)$	$N_1\, s$
3	$N_1 N_2 N_3$	$N_1 N_2(\lfloor N_3/2 \rfloor + 1)$	$N_1 N_2\, s$

The program `ch07_lowpass.c` removes the fast part of $N = 64$ real samples. It transforms them, multiplies every stored bin by a real factor, $1/N$ for the bins below a cut and 0 above it, and transforms back; the $1/N$ of the pair rides on the same multiplication:

ch07_lowpass.c

```
/* ch07_lowpass.c: low-pass filter N = 64 real samples with a VFFT_R2C plan,
 * a cut in the half spectrum and a VFFT_C2R plan, out of place. The signal
 * is a slow part (frequencies 0, 3, 5) plus a fast part (20 and N/2); the
 * filter keeps bins 0 to CUT - 1, so the result is the slow part.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

#define PI 3.14159265358979323846

enum { N = 64, CUT = 8 };
static const double TOL = 1e-12;

static double slow(int n)
{
    return 1.0 + cos(2 * PI * 3 * n / N) - 0.5 * sin(2 * PI * 5 * n / N);
}

static double fast(int n)
{
    return 0.25 * cos(2 * PI * 20 * n / N) + 0.125 * (n % 2 ? -1.0 : 1.0);
}

int main(void)
{
    vfft_config_t cfg = {0};        /* 1D, out of place */
    cfg.transform = VFFT_R2C;
    cfg.n[0] = N;
    vfft_plan r2c = vfft_create(&cfg);
    if (!r2c)
    {
        fprintf(stderr, "vfft_create(VFFT_R2C): %s\n", vfft_error());
        return 1;
    }
    cfg.transform = VFFT_C2R;       /* the same configuration otherwise */
    vfft_plan c2r = vfft_create(&cfg);
    if (!c2r)
    {
        fprintf(stderr, "vfft_create(VFFT_C2R): %s\n", vfft_error());
        return 1;
    }

    vfft_info_t fi, bi;
    vfft_info(r2c, &fi);
    vfft_info(c2r, &bi);
    printf("VFFT_R2C: in_doubles %zu, out_doubles %zu (%zu bins)\n",
           fi.in_doubles, fi.out_doubles, fi.out_doubles / 2);
```

```
    printf("VFFT_C2R: in_doubles %zu, out_doubles %zu\n",
           bi.in_doubles, bi.out_doubles);
    double *x = vfft_malloc(fi.in_doubles * sizeof(double));   /* N reals */
    double *X = vfft_malloc(fi.out_doubles * sizeof(double));  /* bins */
    double *y = vfft_malloc(bi.out_doubles * sizeof(double));  /* N reals */
    if (!x || !X || !y)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (int n = 0; n < N; n++)
        x[n] = slow(n) + fast(n);

    vfft_execute(r2c, VFFT_FORWARD, x, X);         /* bins 0 to N/2 */
    for (int k = 0; k <= N / 2; k++)
    {
        double g = k < CUT ? 1.0 / N : 0.0;         /* the cut and the 1/N */
        X[2 * k] *= g;                              /* Re X[k] */
        X[2 * k + 1] *= g;                          /* Im X[k] */
    }
    vfft_execute(c2r, VFFT_BACKWARD, X, y);        /* N reals */

    double err = 0.0;
    for (int n = 0; n < N; n++)
    {
        double d = fabs(y[n] - slow(n));
        if (d > err || isnan(d))
            err = d;
    }
    int pass = err <= TOL;
    printf("kept bins 0 to %d of 0 to %d: max |y[n] - slow(n)| = %.1e\n",
           CUT - 1, N / 2, err);
    printf("tolerance %.0e: %s\n", TOL, pass ? "PASS" : "FAIL");

    vfft_free(y);
    vfft_free(X);
    vfft_free(x);
    vfft_destroy(c2r);
    vfft_destroy(r2c);
    return pass ? 0 : 1;
}
```

The slow part comes back to rounding, with the sizes of Table 7.1:

```
VFFT_R2C: in_doubles 64, out_doubles 66 (33 bins)
VFFT_C2R: in_doubles 66, out_doubles 64
kept bins 0 to 7 of 0 to 32: max |y[n] - slow(n)| = 1.8e-15
tolerance 1e-12: PASS
```

7.3 Give a C2R plan a half spectrum

A `VFFT_C2R` plan reads its bins as the half spectrum of a real sequence. Bin 0 of a real sequence is real, and so is bin $N/2$ at an even N: when you build a half spectrum yourself, set the imaginary part of bin 0, and at an even N that of bin $N/2$, to 0. The plan defines no result for other values there.

At rank 2 and 3 the same holds on the bins whose last index is 0 or, at an even last extent, $N_3/2$: there bin $[k_1, k_2, k_3]$ must be the conjugate of bin $[-k_1, -k_2, k_3]$ (in 2D, bin $[k_1, k_2]$ of bin $[-k_1, k_2]$), the indices wrapped as in Section 6.1. A spectrum a `VFFT_R2C` plan wrote has this form, and keeps it when you multiply it bin by bin by another such spectrum, as Chapter 13 does, or by a real factor that has the same value at a frequency and at its negative, as the cut of `ch07_lowpass.c` and the factor

of Chapter 12 do. An imaginary factor that changes sign with the frequency, as the factor of a derivative does, keeps it everywhere except on the bins at index $N/2$ of its axis, which Chapter 14 sets to 0.

7.4 Transform real samples in place

With `placement` set to `VFFT_PLACEMENT_IN` in both plans, the samples and the bins share one buffer of $2(\lfloor N/2 \rfloor + 1)$ doubles: $N+2$ at an even N and $N+1$ at an odd N. You write the N reals at its start; the doubles after them are pad and need no value. The forward transform writes the bins over the whole buffer, and the backward transform writes N times the reals back at its start (Figure 7.2).

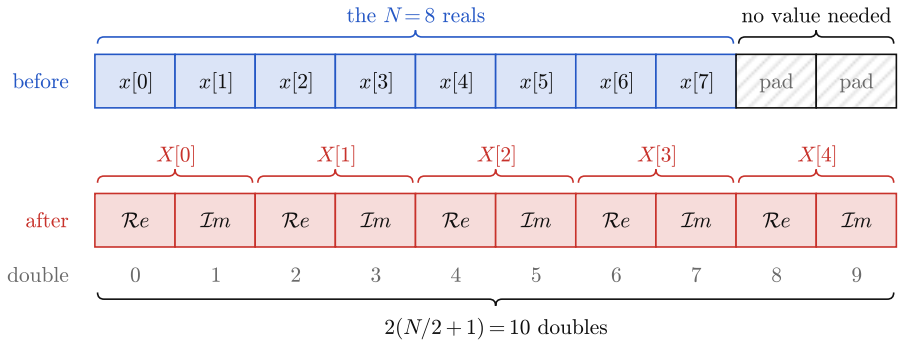


Figure 7.2 The buffer of an in-place pair of $N = 8$ points: the 8 reals (blue) and 2 doubles of pad (grey) before the forward transform, the 5 bins (red) after it.

The program `ch07_in_place_1d.c` creates the pair at $N = 63$, sizes its one buffer from `vfft_info()`,

From ch07_in_place_1d.c

```
    vfft_info_t info;
    vfft_info(r2c, &info);          /* in_doubles = out_doubles */
    double *buf = vfft_malloc(info.in_doubles * sizeof(double));
    if (!buf)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (int n = 0; n < N; n++)
        buf[n] = sample(n);     /* the reals first; the pad needs no value */
```

and passes it to each execute as both the input and the output:

From ch07_in_place_1d.c

```
    vfft_execute(r2c, VFFT_FORWARD, buf, buf);  /* bin k: buf[2k], buf[2k+1] */
```

It checks every bin against the sum of Eq. (2.1a), then runs the backward transform in the same buffer:

```
in place, N = 63: one buffer of 64 doubles
bins 0 to 31: max |X[k] - direct sum| = 8.6e-15
roundtrip: max |y[n]/N - x[n]| = 6.7e-16
tolerance 1e-12: PASS
```

An in-place real plan given two different pointers refuses the call and writes nothing. A buffer of N doubles is too short: the bins need the pad.

7.5 Transform a real array in 2D or 3D

You give `rank` and `n[]` as for a complex array (Section 3.3): `n[0]` is the slowest axis and the last extent the contiguous one. The real transforms halve the last axis only. The spectrum of $N_1 \times N_2 \times N_3$ reals is $N_1 \times N_2 \times (\lfloor N_3/2 \rfloor + 1)$ bins, row-major:

$$\begin{gathered} X[k_1, k_2, k_3] \text{ is element } (k_1 N_2 + k_2)\,(\lfloor N_3/2 \rfloor + 1) + k_3, \\ 0 \le k_3 \le \lfloor N_3/2 \rfloor, \end{gathered} \tag{7.2}$$

and element b is the doubles $2b$ and $2b+1$, as in Section 3.4. In 2D, bin $[k_1, k_2]$ is element $k_1(\lfloor N_2/2 \rfloor + 1) + k_2$. The other axes keep all their bins, the negative frequencies wrapped as in Section 6.1. A forward and a backward transform return the input times $N_1 N_2 N_3$, or $N_1 N_2$ in 2D.

The program `ch07_image_2d.c` transforms 256×64 reals that hold a constant and a wave of the frequencies $(-3, 5)$ out of place. It creates both plans from a configuration that differs from the 1D one in `rank` and `n[]` only:

From ch07_image_2d.c

```
vfft_config_t cfg = {0};
cfg.transform = VFFT_R2C;
cfg.rank = 2;
cfg.n[0] = N1;
cfg.n[1] = N2;                  /* the last axis: the one halved */
```

The wave lands in one bin, at the element Eq. (7.2) gives, its negative first frequency wrapped to $256 - 3 = 253$:

From ch07_image_2d.c

```
/* bin [k1, k2] is element k1 H2 + k2: doubles 2 (k1 H2 + k2) and + 1 */
int k1 = F1 < 0 ? N1 + F1 : F1;     /* all N1 bins: -3 is N1 - 3 */
int k2 = F2;                        /* 0 <= F2 <= N2/2: stored as is */
size_t dc = 0, wave = (size_t)k1 * H2 + k2;
```

It prints:

```
2D VFFT_R2C of 256 x 64 reals: 256 x 33 bins, out_doubles 16896
bin [0, 0] = 16384.000, bin [253, 5] = 8192.000
spectrum: max |X/(N1 N2) - expected| = 1.0e-16
roundtrip: max |y/(N1 N2) - x| = 1.1e-15
tolerance 1e-12: PASS
```

Chapter 12 and Chapter 13 run the same pair on volumes, out of place.

7.6 Transform a real array in place with padded rows

In place, each row of N_3 reals (of N_2 in 2D) is padded to the row pitch s, the doubles from one row to the next. You read s from `vfft_info().stride` of the plan and never compute it; it is at least $2(\lfloor N_3/2 \rfloor + 1)$. The reals of a row come first and the rest of the row is pad, which needs no value. After the forward transform each row holds its $\lfloor N_3/2 \rfloor + 1$ bins:

In place:

$$\begin{gathered} x[n_1, n_2, n_3] \text{ at double } (n_1 N_2 + n_2)\, s + n_3, \\ \mathcal{R}e\, X[k_1, k_2, k_3] \text{ at double } (k_1 N_2 + k_2)\, s + 2k_3, \end{gathered} \tag{7.3}$$

and $\mathcal{I}m\, X[k_1, k_2, k_3]$ at the double after it. In 2D, row n_1 starts at double $n_1 s$ (Figure 7.3). The buffer holds $N_1 N_2\, s$ doubles, $N_1 s$ in 2D: `in_doubles`.

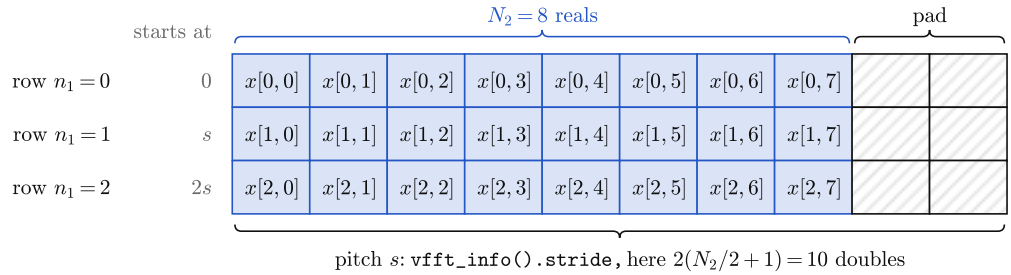


Figure 7.3 An in-place 2D array of 3 rows of 8 reals (blue) at the row pitch $s = 10$, the pad hatched grey. Row n_1 starts at double $n_1 s$; after the forward transform it holds its 5 bins over the same s doubles.

The program `ch07_volume_in_place.c` transforms an $8 \times 8 \times 16$ volume in place. It reads the pitch and the size from both plans, which agree:

From ch07_volume_in_place.c

```
vfft_info_t fi, bi;
vfft_info(r2c, &fi);
vfft_info(c2r, &bi);
if (fi.stride != bi.stride || fi.in_doubles != bi.in_doubles)
{
    fprintf(stderr, "the two plans want different buffers\n");
    return 1;
}
const size_t s = fi.stride;     /* doubles from one row to the next */
double *v = vfft_malloc(fi.in_doubles * sizeof(double));
if (!v)
{
    fprintf(stderr, "out of memory\n");
    return 1;
}
```

It writes each row's reals at the row's start,

From ch07_volume_in_place.c

```
for (int n1 = 0; n1 < N1; n1++)
    for (int n2 = 0; n2 < N2; n2++)
    {
        double *row = v + ((size_t)n1 * N2 + n2) * s;
        for (int n3 = 0; n3 < N3; n3++)
            row[n3] = field(n1, n2, n3);    /* row[N3] to row[s-1]: pad */
    }
```

runs the forward transform in the volume,

From ch07_volume_in_place.c

```
vfft_execute(r2c, VFFT_FORWARD, v, v);          /* bins over the reals */
```

and reads bin $[k_1, k_2, k_3]$ at Eq. (7.3):

From ch07_volume_in_place.c

```
/* bin [k1,k2,k3]: doubles 2 k3 and 2 k3 + 1 of row (k1, k2) */
int w1 = F1 < 0 ? N1 + F1 : F1, w2 = F2 < 0 ? N2 + F2 : F2, w3 = F3;
const double S = (double)N1 * N2 * N3;
double err = 0.0;                   /* relative to N1 N2 N3 */
for (int k1 = 0; k1 < N1; k1++)
    for (int k2 = 0; k2 < N2; k2++)
    {
        const double *row = v + ((size_t)k1 * N2 + k2) * s;
```

```
            for (int k3 = 0; k3 < H3; k3++)
            {
                double want = 0.0;
                if (k1 == 0 && k2 == 0 && k3 == 0)
                    want = 2.0;
                else if (k1 == w1 && k2 == w2 && k3 == w3)
                    want = 0.5;
                double d = hypot(row[2 * k3] / S - want, row[2 * k3 + 1] / S);
                if (d > err || isnan(d))
                    err = d;
            }
        }
```

It prints:

```
8 x 8 x 16 in place: rows of 18 doubles, 1152 doubles in all
bin [1, 6, 3] = 512.000, half of N1 N2 N3
spectrum: max |X/(N1 N2 N3) - expected| = 1.4e-16
roundtrip: max |y/(N1 N2 N3) - x| = 8.9e-16
tolerance 1e-12: PASS
```

After the backward transform each row holds $N_1N_2N_3$ times its reals at its start; the pad holds nothing you need. Chapter 14 differentiates a 64^3 volume this way.

7.7 Let the plans choose the row pitch

With `VFFT_FLAG_OWN_PITCH` in `flags`, an in-place `VFFT_R2C` or `VFFT_C2R` plan of rank 2 or 3 may choose a pitch wider than a tight row, at the row lengths where a wider row runs faster. A program that reads the pitch, as `ch07_volume_in_place.c` does, needs nothing else; give the flag to both plans of a pair. A create refuses the flag for any other configuration.

The program `ch07_own_pitch.c` creates such a pair for 8×512 reals, both plans from this configuration:

From ch07_own_pitch.c

```
    vfft_config_t cfg = {0};
    cfg.transform = VFFT_R2C;
    cfg.rank = 2;
    cfg.n[0] = N1;
    cfg.n[1] = N2;
    cfg.placement = VFFT_PLACEMENT_IN;
    cfg.flags = VFFT_FLAG_OWN_PITCH;    /* the plans may widen the rows */
```

and reads the pitch they chose:

From ch07_own_pitch.c

```
    vfft_info_t fi, bi;
    vfft_info(r2c, &fi);
    vfft_info(c2r, &bi);
    if (fi.stride != bi.stride || fi.in_doubles != bi.in_doubles)
    {
        fprintf(stderr, "the two plans want different buffers\n");
        return 1;
    }
    const size_t s = fi.stride;     /* read it: never 2 H2 by assumption */
    printf("rows of %d reals: pitch %zu doubles (a tight row is %d), "
           "%zu doubles in all\n", N2, s, 2 * H2, fi.in_doubles);
```

The plans chose 530 doubles, 16 more than a tight row:

```
rows of 512 reals: pitch 530 doubles (a tight row is 514), 4240 doubles in all
spectrum: max |X/(N1 N2) - expected| = 6.2e-15
roundtrip: max |y/(N1 N2) - x| = 6.7e-16
tolerance 1e-12: PASS
```

A program that computes the pitch as $2(\lfloor N_2/2 \rfloor + 1)$ instead of reading it puts every row after the first in the wrong place: here row 1 would start at double 514, not 530.

8 Batches

You transform many arrays of the same shape with one plan: set `howmany` to their number, and one call of `vfft_execute()` runs them all. This chapter shows the two ways the transforms of a batch can lie in memory, how to batch complex and real transforms in one, two and three dimensions, and which batches the library serves.

8.1 Choose where the transforms lie

A batch is K transforms of one configuration; `howmany` holds K, and 0 or 1 is a single transform. `batch` names the geometry: where in your buffer each transform's elements lie. Write $x_t[n]$ for element n of transform t, $0 \le t \le K-1$. For K complex transforms of N points, the two geometries put it at

$$\text{Contiguous:} \qquad x_t[n] \text{ at } z[2(tN+n)], \tag{8.1a}$$

$$\text{Lane-major:} \qquad x_t[n] \text{ at } z[2(nK+t)], \tag{8.1b}$$

for $0 \le n \le N-1$, with its imaginary part in the double that follows. `VFFT_BATCH_CONTIGUOUS`, Eq. (8.1a), gives each transform a block of its own, one block after another. `VFFT_BATCH_LANE_MAJOR`, Eq. (8.1b), puts the transforms side by side, sample by sample: sample 0 of every transform, then sample 1 of every transform, and so on. Figure 8.1 shows both for three transforms of four points. The output follows the same rule, with bin k in place of sample n.

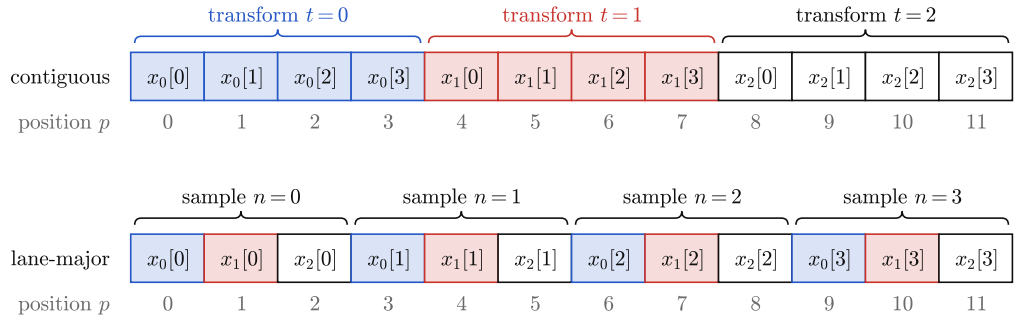


Figure 8.1 The two geometries of a batch of $K = 3$ transforms of $N = 4$ complex elements. Each box is one complex element; the number under it is its position p, so its real part is the double `z[2p]` and its imaginary part `z[2p + 1]`. Contiguous: transform t fills positions tN to $tN + N - 1$. Lane-major: sample n of transform t is at position $nK + t$, so the elements of one transform (blue: transform 0; red: transform 1) lie K positions apart.

For a 1D batch, `VFFT_BATCH_DEFAULT`, the zero value, means contiguous for `VFFT_C2C` and lane-major for `VFFT_R2C` and `VFFT_C2R`; Table 8.2 gives it for 2D and 3D batches. Name the geometry anyway: your configuration then states the layout your program fills.

A real batch counts its samples in doubles and its bins in complex pairs. Each real transform has N samples, and its half spectrum has $N/2+1$ bins (Chapter 7); Table 8.1 gives where each lies.

Table 8.1 Where sample n and bin k of real transform t lie, out of place.

Data	`VFFT_BATCH_CONTIGUOUS`	`VFFT_BATCH_LANE_MAJOR`
sample n: the input of `VFFT_R2C`, the output of `VFFT_C2R`	$x[tN+n]$	$x[nK+t]$
bin k, $0 \le k \le N/2$: the output of `VFFT_R2C`, the input of `VFFT_C2R`	$z[2(t(N/2+1)+k)]$	$z[2(kK+t)]$

For arrays of rank 2 and 3, n is the storage index ℓ of an element within its array (Notation), and N the number of elements of one array, N_1N_2 or $N_1N_2N_3$; the half spectrum of a 2D real array has $N_1(N_2/2+1)$ bins. In place, a real batch pads its blocks or its rows: Section 8.3 and Section 8.4 give where its samples lie.

For a lane-major batch, `vfft_info()` reports K as `stride`: sample n of transform t is element `n * stride + t`.

8.2 Run a batch of complex transforms

Set `howmany` and `batch`, then size your buffers from `vfft_info()`: a batch of K complex transforms of N points reads and writes $2NK$ doubles. The program `ch08_c2c_batch.c` runs $K = 4$ transforms of $N = 64$ points, once in each geometry. Transform t holds a complex exponential of frequency $t+1$, so the whole of its spectrum, N, lies in bin $t+1$; the program checks every bin of every transform against that, through the function `position()`, which is Eqs. (8.1) in code:

ch08_c2c_batch.c

```
/* ch08_c2c_batch.c: one plan runs K complex transforms of N points, first with
 * the transforms one after another, then side by side. Transform t holds a
 * complex exponential of frequency t + 1, so its spectrum is N at bin t + 1
 * and zero at every other bin.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

enum { N = 64, K = 4 };
static const double TOL = 1e-12;
static const double PI = 3.14159265358979323846;

/* where element n of transform t sits: the pair z[2p], z[2p + 1] */
static size_t position(vfft_batch_t batch, int t, int n)
{
    if (batch == VFFT_BATCH_LANE_MAJOR)
        return (size_t)n * K + (size_t)t;
    return (size_t)t * N + (size_t)n;           /* VFFT_BATCH_CONTIGUOUS */
}

/* runs the batch in one geometry; returns its largest error, or -1 */
static double run(vfft_batch_t batch, const char *name)
{
    vfft_config_t cfg = {0};        /* VFFT_C2C, one dimension */
    cfg.n[0] = N;
    cfg.howmany = K;                /* K transforms of N points */
    cfg.batch = batch;
    vfft_plan plan = vfft_create(&cfg);
    if (!plan)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return -1.0;
    }

    vfft_info_t info;
    vfft_info(plan, &info);         /* in_doubles = out_doubles = 2NK */
    double *z = vfft_malloc(info.in_doubles * sizeof(double));
    double *Z = vfft_malloc(info.out_doubles * sizeof(double));
    if (!z || !Z)
    {
        fprintf(stderr, "out of memory\n");
        vfft_free(Z);
        vfft_free(z);
        vfft_destroy(plan);
```

```
        return -1.0;
    }
    for (int t = 0; t < K; t++)
        for (int n = 0; n < N; n++)
        {
            size_t p = position(batch, t, n);
            z[2 * p] = cos(2.0 * PI * (t + 1) * n / N);
            z[2 * p + 1] = sin(2.0 * PI * (t + 1) * n / N);
        }

    vfft_execute(plan, VFFT_FORWARD, z, Z);     /* all K transforms */

    double err = 0.0;
    printf("%s: the peaks are at bins", name);
    for (int t = 0; t < K; t++)
    {
        int peak = 0;
        double best = -1.0;
        for (int k = 0; k < N; k++)
        {
            size_t p = position(batch, t, k);
            double re = Z[2 * p], im = Z[2 * p + 1];
            double d = hypot(re - (k == t + 1 ? N : 0.0), im);
            if (d > err || isnan(d))
                err = d;
            if (hypot(re, im) > best)
            {
                best = hypot(re, im);
                peak = k;
            }
        }
        printf(" %d", peak);
    }
    printf("\n");

    vfft_free(Z);
    vfft_free(z);
    vfft_destroy(plan);
    return err;
}

int main(void)
{
    double e1 = run(VFFT_BATCH_CONTIGUOUS, "VFFT_BATCH_CONTIGUOUS");
    double e2 = run(VFFT_BATCH_LANE_MAJOR, "VFFT_BATCH_LANE_MAJOR");
    if (e1 < 0.0 || e2 < 0.0)
        return 1;
    double err = e1;
    if (e2 > err || isnan(e2))
        err = e2;
    int pass = err <= TOL;          /* false for a NaN */
    printf("max error %.1e, tolerance %.0e: %s\n", err, TOL,
           pass ? "PASS" : "FAIL");
    return pass ? 0 : 1;
}
```

Both plans find each transform's peak in its own bin:

```
VFFT_BATCH_CONTIGUOUS: the peaks are at bins 1 2 3 4
VFFT_BATCH_LANE_MAJOR: the peaks are at bins 1 2 3 4
max error 3.4e-14, tolerance 1e-12: PASS
```

A batch of complex transforms also runs in place, in either geometry: set `placement` to `VFFT_PLACEMENT_IN` and pass one buffer of $2NK$ doubles as both `in` and `out`.

8.3 Run a batch of real transforms

A 1D batch of `VFFT_R2C` or `VFFT_C2R` transforms is lane-major by default. The lane-major geometry needs an even N of at least 4. For any other N, name `VFFT_BATCH_CONTIGUOUS`. The program `ch08_real_batch.c` creates both plans of a lane-major batch of $K = 4$ real transforms of $N = 64$ points:

From ch08_real_batch.c

```
vfft_config_t cfg = {0};
cfg.transform = transform;      /* VFFT_R2C or VFFT_C2R */
cfg.n[0] = N;
cfg.howmany = K;
cfg.batch = VFFT_BATCH_LANE_MAJOR;  /* also the default for real data */
```

It puts sample n of transform t at `x[n * K + t]`. Transform t holds $\cos(2\pi(t+1)n/N)$, whose half spectrum is $N/2$ at bin $t+1$ and zero elsewhere. The program checks every bin at `z[2(k * K + t)]`, then runs the `VFFT_C2R` plan on the spectra and checks that it returns N times every sample:

```
R2C: the peaks are at bins 1 2 3 4
C2R and R2C: max error 1.8e-14, tolerance 1e-12: PASS
```

Out of place, the `VFFT_R2C` plan reads NK doubles and writes $2(N/2+1)K$. In place, both geometries use one buffer, which `vfft_info()` sizes:

- lane-major: one buffer of $(N+2)K$ doubles, the samples at `x[n * K + t]` first and the bins written over them at `z[2(k * K + t)]`;
- contiguous: one block of $2(N/2+1)$ doubles per transform, block t starting at double $2t(N/2+1)$, each laid out as one in-place real transform (Chapter 7).

8.4 Run a batch of 2D or 3D arrays

A batch of 2D or 3D arrays is contiguous by default: array t is a block of its own, after array $t-1$. Lane-major, element ℓ of array t lies at $z[2(\ell K + t)]$, as in Eq. (8.1b) with ℓ for n. A lane-major batch of real arrays needs $N_1 \geq 2$ and an even last extent of at least 4, and in 3D $N_2 \geq 2$ as well.

In place, a batch of real arrays pads every row to $2(N_{\text{last}}/2+1)$ doubles, as one in-place real array does (Chapter 7), and `vfft_info()` sizes the one buffer:

- contiguous: K padded arrays one after another, each laid out as one in-place real array, its rows `stride` doubles apart. In 2D, sample $[n_1, n_2]$ of array t lies at `x[(t * N1 + n1) * stride + n2]`, and the forward transform writes bin $[k_1, k_2]$ over the samples at `z[2((t * N1 + k1) * (N2/2 + 1) + k2)]`; in 3D, sample $[n_1, n_2, n_3]$ lies at `x[((t * N1 + n1) * N2 + n2) * stride + n3]` and bin $[k_1, k_2, k_3]$ at `z[2(((t * N1 + k1) * N2 + k2) * (N3/2 + 1) + k3)]`;
- lane-major: the padded rows side by side. In 2D, one buffer of $N_1(N_2+2)K$ doubles: sample $[n_1, n_2]$ of array t lies at `x[(n1 * (N2 + 2) + n2) * K + t]`, and bin $[k_1, k_2]$ at `z[2((k1 * (N2/2 + 1) + k2) * K + t)]`; in 3D, one buffer of $N_1N_2(N_3+2)K$ doubles: sample $[n_1, n_2, n_3]$ lies at `x[((n1 * N2 + n2) * (N3 + 2) + n3) * K + t]` and bin $[k_1, k_2, k_3]$ at `z[2(((k1 * N2 + k2) * (N3/2 + 1) + k3) * K + t)]`.

A batch of 3D complex arrays runs in either geometry. The program `ch08_volume_batch.c` transforms $K = 3$ arrays of $8 \times 8 \times 16$ points with one plan, lane-major:

From ch08_volume_batch.c

```
    vfft_config_t cfg = {0};
    cfg.rank = 3;
    cfg.n[0] = N1;
    cfg.n[1] = N2;
    cfg.n[2] = N3;                  /* the contiguous axis */
    cfg.howmany = K;
    cfg.batch = VFFT_BATCH_LANE_MAJOR;  /* side by side */
    vfft_plan plan = vfft_create(&cfg);
    if (!plan)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return 1;
    }
```

Array t holds a complex exponential of frequency $(t + 1, t + 2, t + 3)$, and the program finds each array's spectrum in that one bin:

```
array 0: the peak is at bin (1, 2, 3)
array 1: the peak is at bin (2, 3, 4)
array 2: the peak is at bin (3, 4, 5)
spectra / (N1 N2 N3): max error 1.4e-15, tolerance 1e-12: PASS
```

A batch of 3D real arrays runs in either geometry too: contiguous by default, lane-major when named, at $N_1, N_2 \geq 2$ and an even $N_3 \geq 4$. The program `ch08_batch3d_real.c` creates a `VFFT_R2C` plan of $K = 3$ arrays of $8 \times 8 \times 16$ points in each geometry, and a plan of one array as the reference. Its function `run()` fills the batch through the geometry's position functions, as `position()` does in `ch08_c2c_batch.c`, transforms it, and compares each array's bins with those the plan of one array computes from the same samples:

ch08_batch3d_real.c

```
/* ch08_batch3d_real.c: one plan transforms K real 3D arrays of N1 x N2 x N3
 * points, in each geometry. Lane-major, sample l = (n1 N2 + n2) N3 + n3 of
 * array t is x[lK + t], and bin m = (k1 N2 + k2) H + k3, H = N3/2 + 1, the
 * pair z[2(mK + t)]. Contiguous, array t is the block x[tP + l], P = N1 N2 N3,
 * and its bins the block z[2(tM + m)], M = N1 N2 H. Each array's bins are
 * checked against a plan of one array, executed on the same samples.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

enum { N1 = 8, N2 = 8, N3 = 16, K = 3, H = N3 / 2 + 1 };
enum { P = N1 * N2 * N3, M = N1 * N2 * H };     /* samples, bins of one array */
static const double TOL = 1e-12;

typedef size_t (*where_t)(size_t, int);     /* sample l or bin m of array t */
static size_t lane(size_t i, int t) { return i * K + t; }      /* Eq. (8.1b) */
static size_t block_in(size_t l, int t) { return (size_t)t * P + l; }
static size_t block_out(size_t m, int t) { return (size_t)t * M + m; }

static double sample(int t, size_t l)       /* what array t holds at l */
{
    return cos(0.37 * (double)l + 1.3 * t);
}
```

```
static vfft_plan make(size_t howmany, vfft_batch_t batch)
{
    vfft_config_t cfg = {0};
    cfg.transform = VFFT_R2C;
    cfg.rank = 3;
    cfg.n[0] = N1;
    cfg.n[1] = N2;
    cfg.n[2] = N3;                  /* lane-major: even, at least 4 */
    cfg.howmany = howmany;
    cfg.batch = batch;
    vfft_plan plan = vfft_create(&cfg);
    if (!plan)
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
    return plan;
}

/* Fill the K arrays through the geometry's positions, run the batch, and
 * return the largest difference, per P, between each array's bins and those
 * the plan of one array computes from the same samples. */
static double run(vfft_plan batch, where_t at_in, where_t at_out,
                  vfft_plan one, double *x, double *z, double *x1, double *z1)
{
    double err = 0.0;
    for (int t = 0; t < K; t++)
        for (size_t l = 0; l < P; l++)
            x[at_in(l, t)] = sample(t, l);
    vfft_execute(batch, VFFT_FORWARD, x, z);        /* all K arrays */
    for (int t = 0; t < K; t++)
    {
        for (size_t l = 0; l < P; l++)
            x1[l] = sample(t, l);
        vfft_execute(one, VFFT_FORWARD, x1, z1);    /* array t alone */
        for (size_t m = 0; m < M; m++)
        {
            size_t p = at_out(m, t);
            double d = hypot(z[2 * p] - z1[2 * m],
                             z[2 * p + 1] - z1[2 * m + 1]) / P;
            if (d > err || isnan(d))
                err = d;
        }
    }
    return err;
}

int main(void)
{
    vfft_plan one = make(1, VFFT_BATCH_DEFAULT);        /* the reference */
    vfft_plan lm = make(K, VFFT_BATCH_LANE_MAJOR);
    vfft_plan ct = make(K, VFFT_BATCH_CONTIGUOUS);      /* also the default */
    if (!one || !lm || !ct)
        return 1;

    vfft_info_t i1, ilm, ict;
    vfft_info(one, &i1);            /* in: P reals; out: 2M doubles */
    vfft_info(lm, &ilm);            /* in: PK; out: 2MK; stride K */
    vfft_info(ct, &ict);            /* in: PK; out: 2MK */
    double *x1 = vfft_malloc(i1.in_doubles * sizeof(double));
    double *z1 = vfft_malloc(i1.out_doubles * sizeof(double));
    double *x = vfft_malloc(ilm.in_doubles * sizeof(double));
    double *z = vfft_malloc(ilm.out_doubles * sizeof(double));
    if (!x1 || !z1 || !x || !z)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
```

```
    }

    double e_lm = run(lm, lane, lane, one, x, z, x1, z1);
    printf("lane-major: %zu reals in, %zu doubles out, stride %zu: "
           "max error %.1e\n", ilm.in_doubles, ilm.out_doubles, ilm.stride,
           e_lm);
    double e_ct = run(ct, block_in, block_out, one, x, z, x1, z1);
    printf("contiguous: %zu reals in, %zu doubles out: max error %.1e\n",
           ict.in_doubles, ict.out_doubles, e_ct);

    double err = e_lm > e_ct ? e_lm : e_ct;
    int pass = err <= TOL;          /* false for a NaN */
    printf("K = %d arrays against a plan of one array, tolerance %.0e: %s\n",
           K, TOL, pass ? "PASS" : "FAIL");

    vfft_free(z);
    vfft_free(x);
    vfft_free(z1);
    vfft_free(x1);
    vfft_destroy(ct);
    vfft_destroy(lm);
    vfft_destroy(one);
    return pass ? 0 : 1;
}
```

Both batches agree with the plan of one array:

```
lane-major: 3072 reals in, 3456 doubles out, stride 3: max error 3.9e-17
contiguous: 3072 reals in, 3456 doubles out: max error 0.0e+00
K = 3 arrays against a plan of one array, tolerance 1e-12: PASS
```

Table 8.2 lists the batches the library serves. Every one runs in either placement.

Table 8.2 The batches the library serves.

Arrays	Transform	`VFFT_BATCH_DEFAULT`	`VFFT_BATCH_CONTIGUOUS`	`VFFT_BATCH_LANE_MAJOR`
1D	`VFFT_C2C`	contiguous	served	served
1D	`VFFT_R2C`, `VFFT_C2R`	lane-major	served	served at an even $N \geq 4$
2D	`VFFT_C2C`	contiguous	served	served
2D	`VFFT_R2C`, `VFFT_C2R`	contiguous	served	served at $N_1 \geq 2$ and an even $N_2 \geq 4$
3D	`VFFT_C2C`	contiguous	served	served
3D	`VFFT_R2C`, `VFFT_C2R`	contiguous	served	served at $N_1, N_2 \geq 2$ and an even $N_3 \geq 4$

8.5 Fix a refused batch

A batch the library does not serve makes `vfft_create()` return `NULL`, and `vfft_error()` says why (Chapter 11). Two requests are refused most often:

- **A 1D real batch at an odd N, or at $N < 4$, in the default geometry.** The default is lane-major, which needs an even $N \geq 4$. Name `VFFT_BATCH_CONTIGUOUS`.
- **A lane-major batch of 2D or 3D real arrays at an odd last extent, or at a last extent below 4.** Name `VFFT_BATCH_CONTIGUOUS` and lay the arrays out one after another.

The library cannot see how you filled a buffer. If you lay out the data in one geometry and create the plan with the other, every call succeeds, and each transform the plan computes mixes samples of several of your transforms. Fill and read the buffers through one function of the plan's geometry, as `position()` does in `ch08_c2c_batch.c`.

9 Threads

You run a transform on several cores by giving its plan a thread count, and you run plans from threads of your own by giving each thread its own plan. This chapter shows both, and what to expect of the results.

9.1 Run a plan on several threads

Set `nthreads` in the configuration to the number of threads the plan runs on, your calling thread included: the library adds its own worker threads for the rest. 0 and 1 both mean one thread. To use every performance core of the machine, ask the library how many there are: `vfft_info()` called with `NULL` in place of a plan fills `cores` without creating anything. It counts the performance cores of a hybrid CPU, and every physical core of any other.

The program `ch09_nthreads.c` creates two plans of one configuration, a complex transform of 65536 points, one on a single thread and one on `cores` threads, runs both on the same input, and compares the two results:

ch09_nthreads.c

```
/* ch09_nthreads.c: one transform of N points run on one thread and on every
 * performance core of the machine. The two results agree to within rounding,
 * and the threaded plan's backward transform returns N times the input.
 */
#include <math.h>
#include <stdio.h>
#include "vfft.h"

enum { N = 65536 };
static const double TOL = 1e-12;

static vfft_plan make(int nthreads)
{
    vfft_config_t cfg = {0};
    cfg.n[0] = N;
    cfg.nthreads = nthreads;        /* the calling thread included */
    vfft_plan plan = vfft_create(&cfg);
    if (!plan)
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
    return plan;
}

int main(void)
{
    vfft_info_t lib;
    vfft_info(NULL, &lib);          /* the library's facts: no plan needed */
    vfft_plan one = make(1);
    vfft_plan all = make(lib.cores);
    if (!one || !all)
        return 1;

    vfft_info_t info;
    vfft_info(all, &info);
    printf("performance cores: %d; the second plan runs on %d threads\n",
           lib.cores, info.nthreads);
    double *x = vfft_malloc(info.in_doubles * sizeof(double));
    double *X1 = vfft_malloc(info.out_doubles * sizeof(double));
    double *Xn = vfft_malloc(info.out_doubles * sizeof(double));
    double *y = vfft_malloc(info.in_doubles * sizeof(double));
    if (!x || !X1 || !Xn || !y)
    {
```

```
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (int n = 0; n < N; n++)
    {
        x[2 * n] = cos(0.001 * n) + 0.5 * sin(0.37 * n);
        x[2 * n + 1] = sin(0.002 * n) - 0.25 * cos(1.3 * n);
    }

    vfft_execute(one, VFFT_FORWARD, x, X1);
    vfft_execute(all, VFFT_FORWARD, x, Xn);
    vfft_execute(all, VFFT_BACKWARD, Xn, y);    /* y = N x */

    double big = 0.0, diff = 0.0, err = 0.0;
    for (size_t i = 0; i < info.out_doubles; i++)
    {
        double d = fabs(Xn[i] - X1[i]);
        if (d > diff || isnan(d))
            diff = d;
        if (fabs(X1[i]) > big)
            big = fabs(X1[i]);
    }
    for (size_t i = 0; i < info.in_doubles; i++)
    {
        double d = fabs(y[i] / N - x[i]);
        if (d > err || isnan(d))
            err = d;
    }
    double rel = diff / big;
    printf("1 thread against %d threads: largest difference %.1e of the "
           "largest |X[k]|\n", info.nthreads, rel);
    printf("roundtrip on %d threads: max |y/N - x| = %.1e\n", info.nthreads,
           err);
    if (rel > err || isnan(rel))
        err = rel;
    int pass = err <= TOL;          /* false for a NaN */
    printf("tolerance %.0e: %s\n", TOL, pass ? "PASS" : "FAIL");

    vfft_free(y);
    vfft_free(Xn);
    vfft_free(X1);
    vfft_free(x);
    vfft_destroy(all);
    vfft_destroy(one);
    return pass ? 0 : 1;
}
```

On the machine that built this guide, `cores` is 8, and the second plan reports 8 threads:

```
performance cores: 8; the second plan runs on 8 threads
1 thread against 8 threads: largest difference 0.0e+00 of the largest |X[k]|
roundtrip on 8 threads: max |y/N - x| = 1.3e-15
tolerance 1e-12: PASS
```

A count past the machine's logical CPUs, or past 64, is refused, not reduced, and so is a negative count (Chapter 11). Use `cores` threads or fewer, and no more than 64.

The thread count is part of the configuration: the store keeps the winner of each count separately (Chapter 10). A create at a count the store has not seen measures before it returns, even when the store holds the plan of another count. Both creates of

`ch09_nthreads.c` were served from the wisdom the library ships for the CPU of the machine that built this guide; on a CPU without shipped wisdom, the first run of the program measures both plans, and later runs are served.

9.2 Compare results across thread counts

A plan on several threads can compute its sums in another order than the plan of the same configuration on one thread, so the two results may differ in the last bits. Compare them with a tolerance, relative to the largest value, never bit for bit. In the run shown, the two results are equal to the last bit; other configurations differ by a few units in the last place.

9.3 Call plans from threads of your own

These rules hold when threads of your program call the library:

- Different plans can execute at the same time on different threads.
- Never execute one plan from two threads at once. Give each thread that transforms a plan of its own, or make the threads take turns on one plan, with a mutex around its executes.
- Creates can run while other threads execute; creates from several threads take turns.
- Destroy a plan only when no thread executes it.
- `vfft_error()` reports the last refusal of the thread that calls it (Chapter 11).

Figure 9.1 draws the second rule.

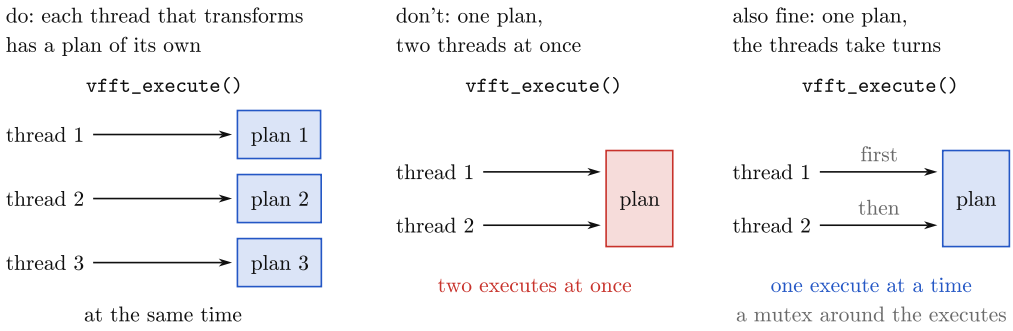


Figure 9.1 One plan per thread at a time. Threads with plans of their own (blue) execute at the same time; two threads never execute one plan at once (red); threads may take turns on one plan, one execute at a time, with a mutex around its executes.

The program `ch09_own_threads.c` starts four threads. Each creates a plan of its own, of one configuration, and runs it 200 times on its own data:

From ch09_own_threads.c

```
static void *work(void *arg)
{
    job_t *job = arg;
    vfft_config_t cfg = {0};
    cfg.n[0] = N;
    vfft_plan plan = vfft_create(&cfg);     /* this thread's own plan */
    if (!plan)
    {
        fprintf(stderr, "vfft_create: %s\n", vfft_error());
        return NULL;
    }
    for (int r = 0; r < RUNS; r++)
        vfft_execute(plan, VFFT_FORWARD, job->x, job->X);
```

```
    vfft_destroy(plan);
    job->ok = 1;
    return NULL;
}
```

The main thread creates a plan of the same configuration first, then starts the four threads and waits for them:

From ch09_own_threads.c

```
    pthread_t th[THREADS];
    for (int i = 0; i < THREADS; i++)
        if (pthread_create(&th[i], NULL, work, &jobs[i]) != 0)
        {
            fprintf(stderr, "pthread_create failed\n");
            return 1;
        }
    for (int i = 0; i < THREADS; i++)
        pthread_join(th[i], NULL);
```

It then transforms the four inputs with its own plan and finds every thread's result equal to its own:

```
4 threads, a plan each, 200 transforms each
largest difference from the main thread's results 0.0e+00, tolerance 1e-12: PASS
```

The program uses POSIX threads, which MinGW-w64 GCC provides; on Linux, compile and link it with `-pthread`.

Your own threads and the threads of plans with `nthreads` above 1 share the cores of the machine. Two plans with `nthreads` above 1 that execute at the same time, from two of your threads, take turns on the library's worker threads. So when your program spreads its transforms over threads of its own, give each thread a plan of one thread; when it runs one large transform at a time, give that plan the threads.

A create that measures keeps a core busy for seconds, and threads that execute meanwhile disturb its timings (Chapter 10). Create a plan of every configuration once before your threads start, as the main thread of `ch09_own_threads.c` does: the creates the threads make later are then served at once.

10 Planning and wisdom

Every `vfft_create()` takes its plan from wisdom: the record of the plan that ran its configuration fastest on your CPU. The library comes with wisdom for the CPUs it has seen and picks the folder of your CPU by itself; a create whose configuration that folder lacks measures the candidates and saves the winner there. In this chapter you see what a create costs with and without wisdom, find the wisdom of your CPU, name a store of your own for the cases that need one, ship wisdom with an application, measure a configuration again, and plan a complex transform without measuring at all.

10.1 Know what a create costs

Every create plans PATIENT unless it sets `VFFT_FLAG_ESTIMATE` (Section 10.6): `VFFT_FLAG_PATIENT` names this default, so setting it changes nothing. The create looks its configuration up in the store, the directory it reads wisdom from and saves winners into (Section 10.2). On a hit, the store holds wisdom for the configuration, and the create builds the plan from the recorded winner at once. On a miss, it measures the candidate plans of the configuration on this machine, saves the winner in the store and then builds the plan, which takes seconds, up to about a minute for some configurations; every later create of the configuration, in this process or in a later one, is a hit. The first create that uses a store also reads it, once per process. Figure 4.1 draws the two paths; Table 10.1 and Figure 10.1 give what creates took on the machine that built this guide.

Table 10.1 What creates took on the Intel Core i9-14900KF that built this guide.

Create	**Store**	**Time**
the first create of a process: `VFFT_C2C`, $N = 64$, a hit	the folder shipped for this CPU, 22425 records	0.16 s
a second create of that configuration in the process	the same	under 1 ms
`VFFT_C2C`, $N = 4096$, on two threads: a miss	the same	0.34 s
`VFFT_C2C`, $N = 64$, with `VFFT_FLAG_RECALIBRATE`	the same	3.6 s
`VFFT_C2C`, $N = 64$: a miss	an empty folder, as on a CPU without shipped wisdom	3.4 s
`VFFT_C2C`, $N = 4096$: a miss	an empty folder	21 s

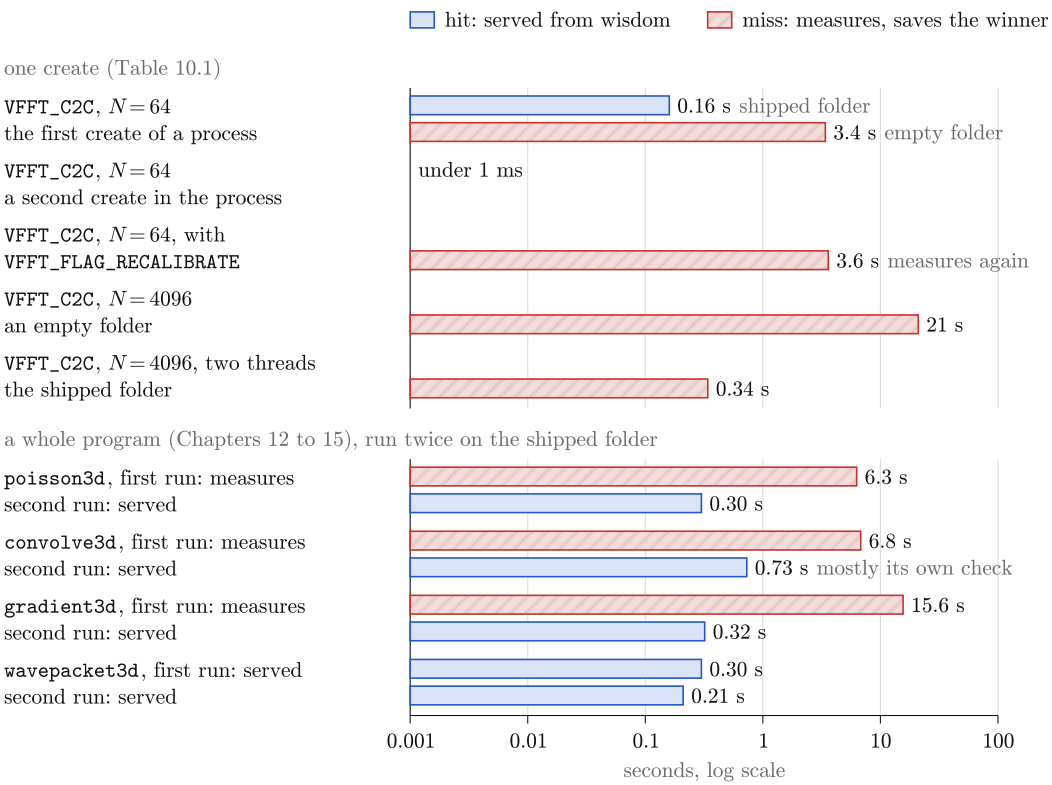


Figure 10.1 What a create costs on the Intel Core i9-14900KF that built this guide. One create, the rows of Table 10.1: a hit (blue) is served from wisdom, in 0.16 s at the first create of a process and under 1 ms after it; a miss (red) measures for seconds. A whole program: the wall time of each example of Part III, from start to exit, run twice in a row on one fresh copy of the folder shipped for this CPU, against the installed library. Three of the four measure on their first run and are served on the second; `wavepacket3d` is served on both. The second run of `convolve3d` is mostly its own direct check of the result, not the transforms.

Measurements take turns across the machine: a create that misses while another VectorFFT program measures waits for it, up to 60 s. Any other work on the machine meanwhile, your own threads included, disturbs the timings, so create plans while the machine is quiet; `vfft_execute()` never measures. Two things that happen while a create measures keep it from saving the winner, even in a store it can write: another VectorFFT program held the machine's measurements for more than 60 s, or the process could run on no performance core. The plan then serves this process only, the library prints why (Section 11.4), and the next process measures the configuration again.

Create every plan your program needs once, at startup (Section 4.2). The program `ch10_startup.c` creates its three plans, a `VFFT_C2C`, a `VFFT_R2C` and a `VFFT_C2R` plan of 4096 points, before it transforms anything:

From ch10_startup.c

```
/* creates the plans the program uses; 0 when one is refused */
static int create_plans(vfft_plan plan[PLANS])
{
    static const vfft_transform_t transform[PLANS] = {
        VFFT_C2C, VFFT_R2C, VFFT_C2R
    };
    for (int i = 0; i < PLANS; i++)
    {
        vfft_config_t cfg = {0};
        cfg.transform = transform[i];
        cfg.n[0] = N;
```

```
            plan[i] = vfft_create(&cfg);
            if (!plan[i])
            {
                fprintf(stderr, "vfft_create: %s\n", vfft_error());
                return 0;
            }
        }
        return 1;
    }
```

Here the folder of this CPU holds all three configurations, so the three creates are served at once; on a CPU without shipped wisdom, the first run measures the three before the program goes on, and later runs are served. The program then runs a complex and a real transform forward and back with the plans and checks the results:

```
roundtrips with the 3 plans: max error 9.4e-16, tolerance 1e-12: PASS
```

10.2 Find the wisdom of your CPU

VectorFFT comes with its own wisdom folder: `src/wisdom/` in the sources, and `<prefix>/share/vectorfft/wisdom/` once installed. It holds one folder for each CPU, as Figure 10.2 shows: `14900KF/` for the Intel Core i9-14900KF and every CPU of the same identity, such as the Core i9-14900K or the Core i9-13900K; `Zen4/` for an AMD Zen 4 laptop CPU; and `new/`, which is empty and is taken by the next CPU the wisdom folder has not seen. A folder belongs to the CPU its files are stamped with, not to its name. The stamp is the `@meta` line near the top of each file: the vendor and model of the CPU, the instruction set the library was built for, the cache sizes, and the numbers of performance and efficient cores.

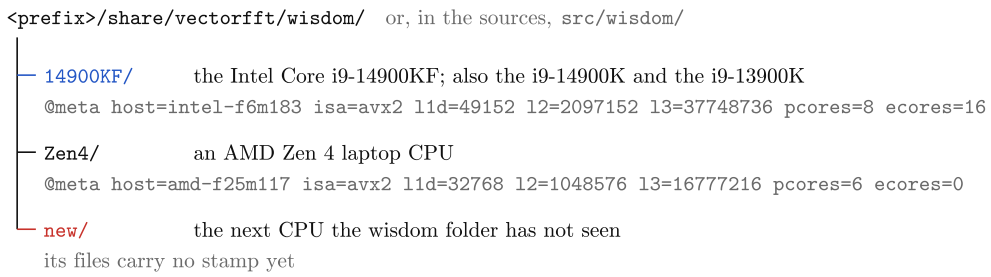


Figure 10.2 The wisdom folder of the library, with one folder for each CPU. The `@meta` line of a folder's files is its stamp; a program uses the folder whose stamp equals the identity of its CPU (blue: `14900KF/` on the machine that built this guide), and a CPU that matches none takes `new/` (red).

With no store named, `wisdom_dir` `NULL` in the configuration and `VFFT_WISDOM_DIR` unset, the library picks the folder of this CPU by itself, once per process:

1. the folder whose stamp equals the identity of this CPU;
2. otherwise `new/`, while no CPU has taken it: the first winner saved there stamps it as this CPU's;
3. otherwise a new folder, which the library makes and names after the identity of this CPU.

It serves the process from that folder and saves every new winner into it. On a CPU the folder already covers, most configurations are served at once. On any other CPU, the first create of each configuration measures for seconds and saves the winner, and every later run is served. A library built inside your project with `add_subdirectory()` or `FetchContent` uses the wisdom folder of its copy of the sources, `src/wisdom/`, in the same way.

Saving needs a folder the program can write. Install the library to a prefix you can write (Section 1.2): under a prefix that only an administrator can write, such as a system folder, nothing is saved, and every process measures its misses again. Where the library must stay in such a prefix, name a store as Section 10.3 shows.

The first create that uses a store prints one line on standard error, unasked, the store line. Where this guide shows what a program printed on standard error, the store line reads

```
[wisdom2] <the folder of this CPU>: 22425 record(s) loaded, writable
```

`<the folder of this CPU>` stands for the path of the folder the library picked, `<prefix>/share/vectorfft/wisdom/14900KF` on the machine that built this guide, and 22425 is the number of records that folder held. A store you name appears as you named it. When the library takes `new/` or makes a folder, a line before the store line says so (Section 11.4).

10.3 Name a store of your own

You can name the store a program uses, in place of the folder the library picks: in `wisdom_dir` in the configuration, or, without changing the program, in the environment variable `VFFT_WISDOM_DIR` (Section 19.2). Name one in these cases:

- to keep the wisdom of one application apart from the library's wisdom folder;
- when the program cannot write the library's wisdom folder, as under a system prefix;
- for an application you install on other machines, with the wisdom you ship with it (Section 10.4).

The library opens a named directory as it is: it picks no folder of a CPU in it, serves from the wisdom files that lie directly in it, and saves every new winner into them. So name a directory of wisdom files, such as a copy of the folder of this CPU, never the wisdom folder that holds the folders of the CPUs. The directory must exist, and the program must be able to write it and its files; the library never creates it. A process opens each directory once, at the first create that names it, and keeps it until the process ends.

Before its first use, seed the store with a copy of the folder of this CPU from the library's wisdom folder. `<store>` is the directory you name, and `<CPU>` the folder of this CPU, which the store line of a program run with no store named shows:

```
mkdir -p <store>
cp <prefix>/share/vectorfft/wisdom/<CPU>/* <store>/
```

A store that starts empty throws the shipped wisdom away: the first create of every configuration then measures. Copy the folder of the CPU that runs the program, and only that one: a directory measured on another CPU is served as it is, and the library says so once, on standard error (Section 11.4). On a CPU that has no folder of its own, the store starts empty and fills as the program measures.

In a program, give `wisdom_dir` the absolute path of the store, in every configuration that uses it, so that the store does not depend on the folder the program runs in.

If the directory is missing, or the process cannot write it, the program still runs: each miss measures, and its winner serves this process only. The library says so once, at the first winner it cannot save, with the line `[wisdom2] the store directory '<dir>' is missing or cannot be written: winners are kept for this process only`.

10.4 Ship wisdom with an application

Wisdom is measured on one CPU model, so you ship it as the folder of the CPU your users run. Measure on a machine with that CPU: create every plan the application uses once, while the machine is quiet, and take the folder the library saved the winners into. Then:

- where VectorFFT is installed at the prefix your program was built against, copy that folder into `<prefix>/share/vectorfft/wisdom/`; if that wisdom folder already holds a folder of the same CPU, such as `14900KF/`, copy the files into that folder instead, because the library reads one folder for each CPU;
- for an application you install anywhere else, install the folder with the application, in a directory the program can write, and name that directory in `wisdom_dir` (Section 10.3).

A folder of wisdom is a directory of plain files, which your installer copies like any other data. You can also fill the folder on the target machine instead: at installation, or at the first start, create every plan the application uses once, while the machine is quiet. Each miss saves its winner.

10.5 Measure a configuration again

`VFFT_FLAG_RECALIBRATE` makes a create measure its configuration even when the store holds wisdom for it, or this process has measured it, and save the winner in place of the old one. Use it on a store you named that was measured on another CPU, or when a measurement ran while the machine was busy. The program `ch10_recalibrate.c` measures again a configuration the folder of this CPU holds:

From ch10_recalibrate.c

```
vfft_config_t cfg = {0};
cfg.n[0] = N;
cfg.flags = VFFT_FLAG_RECALIBRATE;      /* measure again, save the winner */
vfft_plan plan = vfft_create(&cfg);
```

The plan is used like any other:

```
plan created with VFFT_FLAG_RECALIBRATE
roundtrip of 64 points: max |y/N - x| = 4.4e-16, tolerance 1e-12: PASS
```

Its create took 3.6 s here (Table 10.1). Every create with the flag measures, so set it in a run made for planning, and leave it out of the configurations of your program's ordinary runs.

10.6 Plan without measuring

`VFFT_FLAG_ESTIMATE` makes a create choose the plan by formulas from the lengths of the transform and the cache sizes of this machine, instead of from wisdom. It measures nothing, reads no wisdom and saves none, so the create is fast on any CPU, with shipped wisdom or without: on the machine that built this guide, under a millisecond for most configurations, and up to a tenth of a second where the tables of the transform take longer to build, as for a prime length near a million. Every run of a program on the same machine gets the same plan. Use the flag when a program cannot wait for a create to measure, or runs on CPUs you have not measured.

The flag plans `VFFT_C2C` transforms, one transform per plan:

- in one dimension, in either order, in either placement and on any number of threads;
- in two and three dimensions, in natural order on one thread, in either placement.

A create refuses the flag on any other configuration and says why (Section 11.1); drop the flag to plan such a configuration by measuring. A create also refuses it together with `VFFT_FLAG_PATIENT` or `VFFT_FLAG_RECALIBRATE`, and while one of the library's variables that change plans is set (Chapter 19). The program `ch10_estimate.c` plans a 1D, a 2D and a 3D transform with the flag:

From ch10_estimate.c

```
        vfft_config_t cfg = {0};
        cfg.rank = rank[p];
        cfg.n[0] = ext[p][0];
        cfg.n[1] = ext[p][1];
        cfg.n[2] = ext[p][2];
        cfg.flags = VFFT_FLAG_ESTIMATE; /* plan by formulas, measure nothing */
        vfft_plan plan = vfft_create(&cfg);
```

It runs each plan forward and back, then asks for a real transform with the flag, which the create refuses:

From ch10_estimate.c

```
    vfft_config_t real = {0};
    real.transform = VFFT_R2C;
    real.n[0] = 4096;
    real.flags = VFFT_FLAG_ESTIMATE;
    vfft_plan none = vfft_create(&real);
    if (!none)
        printf("R2C with VFFT_FLAG_ESTIMATE refused: %s\n", vfft_error());
```

```
rank 1, 4096 points: max |y/scale - x| = 9.4e-16: ok
rank 2, 65536 points: max |y/scale - x| = 1.6e-15: ok
rank 3, 32768 points: max |y/scale - x| = 1.1e-15: ok
R2C with VFFT_FLAG_ESTIMATE refused: vfft_create: VFFT_FLAG_ESTIMATE plans
    VFFT_C2C only, not VFFT_R2C (drop the flag to plan it by measuring)
tolerance 1e-12: PASS
```

A formula does not know your machine as a measurement does. On the machine that built this guide, two thirds of the power-of-two configurations tried ran within 5% of the time of the plan a create measures, and the rest between 19% faster and 22% slower. When the fastest plan matters, let the create measure once and keep the wisdom (Section 10.1).

11 Errors and messages

When `vfft_create()` returns `NULL` or `vfft_execute()` does nothing, `vfft_error()` says why. In this chapter you read the reason of a refused create or execute, have the reasons printed as they happen, and read the lines the library prints on standard error without being asked.

11.1 Read why a create was refused

A refused create returns `NULL`, and `vfft_error()` returns the reason, one line of text. The reason belongs to the thread that made the call: that thread's next create clears it, and its next refusal replaces it, so read it at once. A create that succeeds leaves `vfft_error()` empty. Nothing is converted and nothing falls back: you get a plan of exactly the configuration you described, or no plan.

The program `ch11_refusals.c` asks for five configurations that are refused and prints the reason of each:

ch11_refusals.c

```
/* ch11_refusals.c: configurations that vfft_create() refuses, and the reason
 * vfft_error() gives for each. One of them also asks for VFFT_FLAG_VERBOSE,
 * so its reason is printed on standard error as well.
 */
#include <stdio.h>
#include "vfft.h"

static const struct
{
    const char *what;
    vfft_config_t cfg;
} cases[] = {
    {"no length", {.n = {0}}},
    {"a bit that is not a flag", {.n = {64}, .flags = 1u << 10}},
    {"nthreads -1, with VFFT_FLAG_VERBOSE",
     {.n = {64}, .nthreads = -1, .flags = VFFT_FLAG_VERBOSE}},
    {"nthreads 1000", {.n = {64}, .nthreads = 1000}},
    {"a lane-major batch of two 3D real arrays at N3 = 7",
     {.transform = VFFT_R2C, .rank = 3, .n = {8, 8, 7}, .howmany = 2,
      .batch = VFFT_BATCH_LANE_MAJOR}},
};
enum { CASES = sizeof cases / sizeof cases[0] };

int main(void)
{
    int refused = 0;
    for (int i = 0; i < CASES; i++)
    {
        vfft_plan plan = vfft_create(&cases[i].cfg);
        if (!plan)
        {
            printf("%s:\n  %s\n", cases[i].what, vfft_error());
            refused += vfft_error()[0] != '\0';     /* a reason is given */
        }
        vfft_destroy(plan);         /* NULL is accepted */
    }
    int pass = refused == CASES;
    printf("%d configurations, %d refused with a reason: %s\n", CASES, refused,
           pass ? "PASS" : "FAIL");
    return pass ? 0 : 1;
}
```

```
no length:
  vfft_create: n[0]=0 invalid (every transform length must be >= 1)
a bit that is not a flag:
  vfft_create: unknown flag bits 0x400
nthreads -1, with VFFT_FLAG_VERBOSE:
  vfft_create: nthreads -1 is negative (0 or 1 is one thread)
nthreads 1000:
  vfft_create: nthreads=1000 is outside the pool's range: 0 or 1 (one thread) up
      to 32 (this machine's 32 logical CPUs, at most 64 dispatched workers)
a lane-major batch of two 3D real arrays at N3 = 7:
  vfft_create: the lane-major interleaved 3D real batch serves R2C and C2R at
      howmany >= 2, N1 >= 2, N2 >= 2 and an even N3 >= 4 (got R2C 8x8x7
      howmany=2) -- an odd N3 has no real column engine; lay the 2 volumes out
      one after another (VFFT_BATCH_DEFAULT or VFFT_BATCH_CONTIGUOUS) for it
5 configurations, 5 refused with a reason: PASS
```

Table 11.1 lists the reasons you meet most often. A word in angle brackets stands for a value from your configuration or your machine.

Table 11.1 The reasons of common refusals of `vfft_create()`.

You asked for	**`vfft_error()` returns**	**Change**
a `NULL` configuration	`vfft_create: NULL config`	pass the address of a `vfft_config_t`
an extent of 0 or less	`vfft_create: n[<axis>]=<extent> invalid (every transform length must be >= 1)`	set every extent of the rank to 2 or more (Section 3.3)
a bit that is not a flag	`vfft_create: unknown flag bits 0x<bits>`	the `VFFT_FLAG_*` names only
a negative `nthreads`	`vfft_create: nthreads <count> is negative (0 or 1 is one thread)`	0 or 1 for one thread
more threads than the machine has logical CPUs, or more than 64	`vfft_create: nthreads=<count> is outside the pool's range: 0 or 1 (one thread) up to <limit> (this machine's <cpus> logical CPUs, at most 64 dispatched workers)`	at most `vfft_info().cores` (Chapter 9)
a value outside its enumeration in `placement`, `order` or `batch`	`vfft_create: invalid batch <value> (valid: VFFT_BATCH_DEFAULT, VFFT_BATCH_CONTIGUOUS, VFFT_BATCH_LANE_MAJOR)`, and the same for `placement` and `order`	one of the names listed
a configuration `VFFT_FLAG_ESTIMATE` does not plan	`vfft_create: VFFT_FLAG_ESTIMATE plans VFFT_C2C only, not VFFT_<transform> (drop the flag to plan it by measuring)`, and the same form for a batch and for a 2D or 3D transform in scrambled order or on several threads	drop the flag (Section 10.6)
`VFFT_FLAG_ESTIMATE` while a variable of the library that changes plans is set	`vfft_create: VFFT_FLAG_ESTIMATE: the developer switch <name> is set; estimate refuses while a plan-steering switch is set (unset it: estimate plans from formulas only)`	unset the variable (Chapter 19)

You asked for	vfft_error() returns	Change
a lane-major batch of 2D real arrays at an odd N_2, out of place	vfft_create: the lane-major interleaved 2D real batch serves R2C and C2R at howmany >= 2, N1 >= 2 and an even N2 >= 4 (got <transform> <N1>x<N2> howmany=<K>) -- an odd N2 has no real column engine	VFFT_BATCH_CONTIGUOUS (Chapter 8)
a lane-major batch of 3D real arrays at an odd N_3, out of place	vfft_create: the lane-major interleaved 3D real batch serves R2C and C2R at howmany >= 2, N1 >= 2, N2 >= 2 and an even N3 >= 4 (got <transform> <N1>x<N2>x<N3> howmany=<K>) -- an odd N3 has no real column engine; lay the <K> volumes out one after another (VFFT_BATCH_DEFAULT or VFFT_BATCH_CONTIGUOUS) for it	VFFT_BATCH_CONTIGUOUS (Chapter 8)

The same two requests in place are refused with another reason.

11.2 Read why an execute was refused

`vfft_execute()` returns nothing. A call the library refuses computes nothing, writes nothing and returns at once, and leaves its reason in `vfft_error()`. The program `ch11_execute.c` runs a `VFFT_R2C` plan forward, then calls it with `VFFT_BACKWARD`, a direction a `VFFT_R2C` plan does not run, after it marks the spectrum the forward transform wrote:

From ch11_execute.c

```
X[0] = -1.0;                    /* a mark the refused call must keep */
vfft_execute(r2c, VFFT_BACKWARD, X, x);     /* R2C runs forward only */
printf("vfft_error(): %s\n", vfft_error());
```

The refused call leaves both buffers as they were, and the next create clears the reason:

```
vfft_error(): vfft_execute: R2C plans are forward-only (real -> spectrum); the
    unnormalized inverse is a separate VFFT_C2R plan (executed with
    VFFT_BACKWARD) - nothing executed
the buffers are unchanged: yes
after the next vfft_create(): ""
forward error 0.0e+00, tolerance 1e-12: PASS
```

Table 11.2 lists the reasons of refused executes. Make every call fit its plan, as Section 4.3 describes, instead of testing for them.

Table 11.2 The reasons of refused calls of `vfft_execute()`.

The call	vfft_error() returns	Change
a NULL plan	vfft_execute: NULL plan (vfft_create failed, or the plan was destroyed) - nothing executed	check every create
a direction that is neither VFFT_FORWARD nor VFFT_BACKWARD	vfft_execute: invalid dir value <value> (valid: VFFT_FORWARD, VFFT_BACKWARD) - nothing executed	one of the two
VFFT_BACKWARD on a VFFT_R2C plan	vfft_execute: R2C plans are forward-only (real -> spectrum); the unnormalized inverse is a separate VFFT_C2R plan (executed with VFFT_BACKWARD) - nothing executed	a VFFT_C2R plan for the backward transform

The call	`vfft_error()` returns	Change
`VFFT_FORWARD` on a `VFFT_C2R` plan	`vfft_execute: C2R plans are backward-only (spectrum -> real, the unnormalized inverse); the forward transform is a separate VFFT_R2C plan (executed with VFFT_FORWARD) - nothing executed`	a `VFFT_R2C` plan for the forward transform
a `NULL` buffer to a `VFFT_C2C` plan	`vfft_execute: INTERLEAVED C2C needs in=z_in and out=z_out non-NULL (out may equal in) - nothing executed`	both buffers; in place, the same buffer twice

11.3 Print the reasons as they happen

To see a reason on standard error at the moment it is given, ask the library to print it:

- `VFFT_FLAG_VERBOSE` in the `flags` of a configuration prints every reason that one create gives, a refusal or a warning;
- the environment variable `VFFT_VERBOSE`, set to any value other than `0`, prints every reason the process gives, those of executes included (Chapter 19).

Each reason is printed on a line of its own that begins with `vfft:`. The third configuration of `ch11_refusals.c` sets `VFFT_FLAG_VERBOSE`, so its reason also appears on standard error; the `[wisdom2]` line after it comes from the program's last create, the first that opened the store:

```
vfft: vfft_create: nthreads -1 is negative (0 or 1 is one thread)
[wisdom2] <the folder of this CPU>: 22425 record(s) loaded, writable
```

The store line names the folder of wisdom the program uses; this guide writes its path as `<the folder of this CPU>` (Section 10.2). `ch11_execute.c` runs with `VFFT_VERBOSE=1`, which prints the reason of its refused execute as the call is made:

```
[wisdom2] <the folder of this CPU>: 22425 record(s) loaded, writable
vfft: vfft_execute: R2C plans are forward-only (real -> spectrum); the
    unnormalized inverse is a separate VFFT_C2R plan (executed with
    VFFT_BACKWARD) - nothing executed
```

Set `VFFT_VERBOSE=1` while you develop a program, and leave it unset in the program's ordinary runs.

11.4 Read the lines the library prints unasked

The library prints a few lines on standard error without being asked: lines about the store begin with `[wisdom2]`, lines about measurements with `[measure]`. Table 11.3 lists them. The store line appears in every run (Section 10.2); the others appear on a CPU the wisdom folder has no folder for, with a store measured on another CPU, or when something keeps the library from saving or measuring as it should.

Table 11.3 The lines the library prints on standard error unasked.

Line	When it appears	What to do
`[wisdom2] <dir>: <n> record(s) loaded, writable`	the first create that uses a store, once per store and process; `<dir>` is the folder of this CPU in the library's wisdom folder, or the store you named	nothing
`[wisdom2] the store directory '<dir>' is missing or cannot be written: winners are kept for this process only`	the first winner a process cannot save: the library's wisdom folder lies under a prefix the program cannot write, or a store you named is missing or cannot be written	install the library to a prefix you can write (Section 10.2), or name a writable store seeded from the folder of this CPU; make a store you named a copy of that folder, or make it and its files writable (Section 10.3)
`[wisdom2] the store '<dir>' was raced on another CPU (<that CPU>); this one is (<this CPU>). Its rows are served as that machine's measurements: recalibrate, or name a store of this CPU's own.`	a store you named was measured on another CPU	name no store, or seed it from the folder of this CPU (Section 10.3); or measure again with `VFFT_FLAG_RECALIBRATE`
`[wisdom2] no folder under <root> carries this CPU's identity (<this CPU>): its rows go to <root>/new, which the first save stamps as this CPU's`	the library's wisdom folder holds no folder for this CPU, and the library takes `new` for it; when `new` already belongs to another CPU, the library makes a folder named after the identity of this CPU, and the line ends `its rows go to the new folder <root>/<name>` instead	nothing: the first create of each configuration measures and saves into that folder, and later runs are served (Section 10.2)
`[measure] waiting for another VectorFFT measurement on this machine (up to 60 s)`	a create misses while another program on the machine measures	nothing: the create waits for it
`[measure] another VectorFFT measurement still holds this machine after 60000 ms: this race runs beside it, and its winner is kept for this process only`	the wait has ended: the create measures beside the other program and does not save the winner	run the program again when the machine is quiet: the next process measures the configuration again and saves the winner
`[measure] this thread is not pinned to a P-core (<why>): its races run there, and their winners are kept for this process only`	the process may run on no performance core, so its measurements are not saved	let the process run on the performance cores

Part III **Worked examples**

12 Solving a periodic Poisson equation

You solve Poisson's equation $\nabla^2\phi = f$ for a periodic source f on a 3D grid: one `VFFT_R2C` transform of f, a division of every bin of its half spectrum, and one `VFFT_C2R` transform back to ϕ. The program is `poisson3d.c`, the first of the four example programs of Part III, and this chapter also builds and runs all four.

A simulation meets this problem whenever it needs a potential from a density: gravity from a mass density, an electric potential from a charge density, a pressure from a divergence. `poisson3d.c` samples f on a grid of $128 \times 128 \times 64$ points over the box $[0, 2\pi) \times [0, 2\pi) \times [0, \pi)$, at the same spacing on every axis, and compares the ϕ it computes with a known solution.

12.1 Build and run the examples

The four programs are in the folder `how-to-use/examples` of the VectorFFT repository, beside a `CMakeLists.txt` that builds them against an installed VectorFFT (Chapter 1 installs it). From the root of your clone, with `<prefix>` the folder you installed VectorFFT to:

```
cmake -S how-to-use/examples -B build-examples -G Ninja \
      -DCMAKE_C_COMPILER=gcc -DCMAKE_PREFIX_PATH=<prefix>
cmake --build build-examples
```

The build makes `poisson3d`, `convolve3d`, `gradient3d` and `wavepacket3d` in the folder `build-examples`. Each of them also builds alone with the one compiler command of Chapter 1, and the build of VectorFFT in that chapter compiles all four too, into the folder `build/how-to-use/examples`; they run the same way once the library is installed.

Run each program from the build folder:

```
cd build-examples
./poisson3d
```

In PowerShell, the same steps are:

```
cd build-examples
.\poisson3d.exe
```

On Windows, a program built with MinGW-w64 needs `libwinpthread-1.dll` when it starts: run it from a shell whose `PATH` holds the MinGW-w64 `bin` folder, or use one of the two fixes of Section 1.7.

The first run can spend seconds in `vfft_create()`, up to about a minute for some configurations: when the wisdom of your CPU lacks a configuration, the create measures it and saves the winner in the library's wisdom folder before it returns (Chapter 10). Run the program again, and its creates are served at once. It prints:

```
poisson3d: mean of f = F[0,0,0]/(N1 N2 N3) = -1.6e-18 (a periodic phi needs 0)
poisson3d 128x128x64: max |phi[n] - phi_e(nh)| = 3.64e-15, tolerance 1e-12: PASS
```

The first line is the mean of f, which must be zero for a periodic ϕ to exist (Section 12.4). The second compares ϕ with the known solution at every grid point: the error is rounding, far smaller than the tolerance of 10^{-12}. The last digits of your run can differ from these, because they depend on the plans the library serves on your machine. Each of the four programs ends its output with `PASS` or `FAIL`, and exits with status 0 only on `PASS`.

12.2 Create the two plans

A zeroed `vfft_config_t` describes one interleaved transform, out of place, in natural order, on one thread. `poisson3d.c` makes both of its plans with one function that sets the transform, the rank and the three extents, `n[0]` the slowest axis and `n[2]` the contiguous one:

From examples/poisson3d.c, lines 103-117

```
static vfft_plan plan3d(vfft_transform_t transform)
{
    vfft_config_t c;
    vfft_plan p;
    memset(&c, 0, sizeof c);
    c.transform = transform;
    c.rank = 3;
    c.n[0] = N1;
    c.n[1] = N2;
    c.n[2] = N3;
    p = vfft_create(&c);
    if (!p) /* read it now: the next create clears it */
        fprintf(stderr, "poisson3d: vfft_create: %s\n", vfft_error());
    return p;
}
```

A create that fails returns `NULL`, and `vfft_error()` says why; read it at once, because the next create on the same thread clears it (Chapter 11). The extents, and the bins the real transforms keep on the halved last axis, are an `enum`:

From examples/poisson3d.c, lines 64-70

```
enum
{
    N1 = 128,        /* n[0], the slowest axis */
    N2 = 128,        /* n[1] */
    N3 = 64,         /* n[2], contiguous */
    H3 = N3 / 2 + 1 /* the bins kept on the halved axis */
};
```

`main` creates the R2C plan and the C2R plan, and allocates three buffers with `vfft_malloc()`. The source `f` and the solution `phi` hold `nr` $= N_1N_2N_3$ doubles each; the half spectrum `F` holds `nc` $= N_1N_2(N_3/2+1)$ bins of two doubles each. These are the sizes `vfft_info()` reports as `in_doubles` and `out_doubles` for the two plans.

From examples/poisson3d.c, lines 140-143

```
    vfft_plan r2c = plan3d(VFFT_R2C), c2r = plan3d(VFFT_C2R);
    double *f = vfft_malloc(nr * sizeof(double));
    double *F = vfft_malloc(2 * nc * sizeof(double));
    double *phi = vfft_malloc(nr * sizeof(double));
```

12.3 Divide in the half spectrum

The solve is three steps: the R2C plan transforms f into its half spectrum F, a loop divides every bin, and the C2R plan transforms the result back into ϕ. Between the first two, the program prints the mean of f, from `F[0]`: the real part of bin $(0,0,0)$ is the sum of all the samples.

From examples/poisson3d.c, lines 154, 157-158

```
        vfft_execute(r2c, VFFT_FORWARD, f, F);
        ...
        divide_by_laplacian(F);
        vfft_execute(c2r, VFFT_BACKWARD, F, phi);
```

In the spectrum the Laplacian is a product: bin (k_1, k_2, k_3) of ϕ is that bin of f divided by $-|\kappa|^2 = -(\kappa_1^2 + \kappa_2^2 + \kappa_3^2)$, where κ_i is the angular wavenumber of bin k_i on an axis of N_i points over the length L_i. The C2R returns $N_1 N_2 N_3$ times the inverse transform (Chapter 7), so the division takes that factor out as well:

$$\Phi[k_1, k_2, k_3] = -\frac{F[k_1, k_2, k_3]}{N_1 N_2 N_3 \, |\kappa|^2}, \qquad \kappa_i = \frac{2\pi \tilde{k}_i}{L_i}, \tag{12.1}$$

where $\tilde{k}_i = k_i$ for $k_i \le N_i/2$, and $\tilde{k}_i = k_i - N_i$ for $k_i > N_i/2$: those bins hold the negative frequencies. The half spectrum stores only $0 \le k_3 \le N_3/2$, so the last axis never wraps. Bin $(0, 0, 0)$ has $\kappa = 0$, and the program sets it to zero.

From examples/poisson3d.c, lines 97-100

```
static double kappa(int k, int n, double len)
{
    return 2 * PI * (k <= n / 2 ? k : k - n) / len;
}
```

The division walks the half spectrum in storage order. Bin (k_1, k_2, k_3) is the pair that starts at `F[2 * ((k1 * N2 + k2) * H3 + k3)]`, real part first: the spectrum has a row of $N_3/2 + 1$ bins for each row of f, in the same order.

From examples/poisson3d.c, lines 120-135

```
static void divide_by_laplacian(double *F)
{
    const double scale = 1.0 / ((double)N1 * N2 * N3);
    int k1, k2, k3;
    for (k1 = 0; k1 < N1; k1++)
        for (k2 = 0; k2 < N2; k2++)
            for (k3 = 0; k3 < H3; k3++) /* k3 <= N3/2: this axis never wraps */
            {
                const double a = kappa(k1, N1, L1), b = kappa(k2, N2, L2);
                const double c = kappa(k3, N3, L3), kk = a * a + b * b + c * c;
                const double g = kk > 0.0 ? -scale / kk : 0.0; /* Phi[0,0,0] = 0
                    */
                double *z = F + 2 * (((size_t)k1 * N2 + k2) * H3 + k3);
                z[0] *= g;
                z[1] *= g;
            }
}
```

The multiplier of Eq. (12.1) is real and has the same value at $\tilde{k}$ and $-\tilde{k}$, so the bins at $k_i = N_i/2$ need no rule of their own; an odd multiplier, such as the $j\kappa$ of a derivative, does (Chapter 14). A wrong bin index, a bin $k_i > N_i/2$ read as a positive frequency, or a missing $1/(N_1 N_2 N_3)$ makes ϕ wrong by far more than the tolerance, and the program prints `FAIL`.

12.4 Adapt the program

Your own source. Fill `f` from your simulation, row-major with the last axis contiguous: sample (n_1, n_2, n_3) at `f[(n1 * N2 + n2) * N3 + n3]`. Bin $(0, 0, 0)$ holds $N_1 N_2 N_3$ times the mean of f. A periodic ϕ exists only when that mean is zero; with $\Phi[0, 0, 0] = 0$, the program returns the ϕ of zero mean whose Laplacian is f minus its mean.

Other grids. Change `N1`, `N2`, `N3` and the box lengths `L1`, `L2`, `L3`: the plans, the buffers and the division follow from them. The program's check assumes one spacing, L_i/N_i, on all three axes.

Many solves. Create the two plans once and keep them; each solve is then the three steps of Section 12.3. An execute allocates, measures and decides nothing (Chapter 4).

Keep the spectrum. Out of place, the C2R leaves `F` as it was, so after the solve `F` still holds Φ. A plan created with `VFFT_FLAG_DESTROY_INPUT` may overwrite its input instead.

In place. Both plans can run in place in one buffer (`VFFT_PLACEMENT_IN`) instead of three. The rows of that buffer are padded to a pitch you read from `vfft_info()`, and bin (k_1, k_2, k_3) starts at `(k1 * N2 + k2) * stride + 2 * k3`; Chapter 14 shows the form.

Threads. Set `nthreads` in the configuration of both plans (Chapter 9). A thread count is part of the configuration, so a create at a new count can measure again.

13 Linear convolution of 3D data

You filter a 3D volume with a kernel, such as the point-spread function of an instrument, by transforming both, multiplying their spectra and transforming the product back. The program is `convolve3d.c`: it convolves a volume of 56^3 samples with a kernel of 9^3 samples through transforms of 64^3 points, and checks every output against the direct sum.

13.1 Run the program

Build the examples as in Section 12.1, and run `convolve3d` from the build folder:

```
./convolve3d
```

The first run measures what the wisdom of your CPU lacks, for seconds, and saves the winners; later runs are served at once (Chapter 10). The program prints:

```
convolve3d 56^3 * 9^3 on 64^3: max |y - direct| = 2.13e-14, max |y| = 37.7, ratio
    5.66e-16, tolerance 1e-12: PASS
```

The error is measured against the largest output, `max |y|`: a ratio near 10^{-15} is rounding.

13.2 Pad both arrays to the length of the result

A volume of L samples per axis, convolved with a kernel of P samples per axis, gives a result of $L + P - 1$ samples per axis. A product of N-point transforms gives the circular convolution instead, whose indices wrap around modulo N. The two agree when both arrays are padded with zeros to

$$N \geq L + P - 1 \tag{13.1}$$

samples per axis, so that no term wraps. `convolve3d.c` names the three sizes in an `enum`; $N = 64 = 56 + 9 - 1$ is the smallest that works:

From examples/convolve3d.c, lines 56-62

```
enum
{
    L = 56,        /* the volume's samples per axis */
    P = 9,         /* the point-spread function's */
    N = 64,        /* the DFT's: N >= L + P - 1 */
    NB = N / 2 + 1 /* the bins R2C keeps on the last axis */
};
```

Both arrays fill the whole N^3 grid, with zeros beyond their own extent:

From examples/convolve3d.c, lines 83-90

```
static void fill_padded(double *a, int len, uint32_t *s)
{
    int n1, n2, n3;
    for (n1 = 0; n1 < N; n1++)
        for (n2 = 0; n2 < N; n2++)
            for (n3 = 0; n3 < N; n3++)
                a[idx(n1, n2, n3)] = (n1 < len && n2 < len && n3 < len) ? urand(s)
                    : 0.0;
}
```

With an N smaller than $L + P - 1$, the last $L + P - 1 - N$ samples of the result on each axis wrap around onto its first ones, and those outputs are wrong.

13.3 Transform both arrays, multiply, transform back

The program creates an R2C plan and a C2R plan of $N \times N \times N$ points, each from a zeroed configuration as in Section 12.2. It creates the C2R plan only when the R2C plan exists, so that `vfft_error()` still holds the reason when the first create fails. `nr` is the number of samples, N^3, and `nc` the number of bins of a half spectrum, $N \cdot N \cdot (N/2+1)$:

From examples/convolve3d.c, lines 111-113

```
const size_t nr = (size_t)N * N * N, nc = (size_t)N * N * NB;
vfft_plan r2c = create3d(VFFT_R2C);
vfft_plan c2r = r2c ? create3d(VFFT_C2R) : NULL;
```

One R2C plan serves both arrays: a plan holds none of your buffers, so you run it on each buffer of its size in turn. The product of the two spectra, divided by the N^3 that `VFFT_BACKWARD` leaves out (Chapter 7), is the spectrum of the result:

$$Y[k_1,k_2,k_3] = \frac{1}{N^3}\,X[k_1,k_2,k_3]\,H[k_1,k_2,k_3], \qquad 0 \le k_3 \le N/2. \tag{13.2}$$

From examples/convolve3d.c, lines 126-136

```
vfft_execute(r2c, VFFT_FORWARD, xp, X);
vfft_execute(r2c, VFFT_FORWARD, hp, H);
for (i = 0; i < nc; i++)
{
    /* Y[k] = X[k] H[k] / N^3, into X: the inverse DFT's 1/N^3, which
        VFFT_BACKWARD
     * leaves out */
    const double a = X[2 * i], b = X[2 * i + 1], c = H[2 * i], d = H[2 * i
        + 1];
    X[2 * i] = (a * c - b * d) / (double)nr;
    X[2 * i + 1] = (a * d + b * c) / (double)nr;
}
vfft_execute(c2r, VFFT_BACKWARD, X, y);
```

The two half spectra have the same layout, so the product runs over their `nc` bins as one flat array of (re, im) pairs, with no 3D index. The product of the spectra of two real arrays keeps the symmetry of a real array's spectrum, so the C2R returns the real result `y`. Out of place, an R2C leaves its input as it was: `xp` and `hp` still hold the two arrays when the program compares `y` with the direct sum.

Multiplying by the conjugate of one spectrum gives the correlation of the two arrays instead of their convolution. The kernel of `convolve3d.c` is not symmetric, so its check tells the two apart, and it also catches a wrap-around or a missing $1/N^3$.

13.4 Adapt the program

Other sizes. Set `L` and `P`, and an `N` of at least `L + P - 1`; every extra sample is padding. Each axis can have a length of its own: give the plans N_1, N_2, N_3 in `n[0]`, `n[1]`, `n[2]`, each at least the sum of the two arrays' lengths on that axis minus one. The half spectrum then holds $N_1N_2(N_3/2+1)$ bins, and the scale is $1/(N_1N_2N_3)$.

One kernel, many volumes. Transform the kernel once and keep `H`. For each volume, run the R2C plan, the product and the C2R plan; the plans and the buffers stay as they are.

In place. Both plans can also run in place, each array in a buffer of its own whose rows are padded to the pitch `vfft_info()` reports (Chapter 14). The result then overwrites the spectrum of the volume, which saves the separate output buffer, and the product finds bin (k_1,k_2,k_3) at `(k1 * N + k2) * stride + 2 * k3` in both buffers.

Threads. Set `nthreads` in the configuration of both plans (Chapter 9).

14 Spectral derivatives in place

You differentiate a periodic real field along each of its three axes and keep the field and its spectrum in one buffer with padded rows: one in-place `VFFT_R2C` transform, then for each axis a multiply of the spectrum and one in-place `VFFT_C2R` transform. The program is `gradient3d.c`: a field on a 64^3 grid, differentiated along each axis and checked against the exact derivatives.

14.1 Run the program

Build the examples as in Section 12.1, and run `gradient3d` from the build folder:

```
./gradient3d
```

The first run measures what the wisdom of your CPU lacks, for seconds, and saves the winners; later runs are served at once (Chapter 10). The program prints:

```
gradient3d 64x64x64 in place, row pitch 66 doubles: max |error| d/dx1 1.22e-14,
    d/dx2 2.13e-14, d/dx3 3.73e-14, tolerance 1e-12: PASS
```

The row pitch, 66 doubles, is $2(N/2+1)$ for $N = 64$. The three errors, one per derivative, are rounding.

14.2 Create the in-place plans and read the pitch

A plan made with `placement` set to `VFFT_PLACEMENT_IN` transforms one buffer in place. `gradient3d.c` makes its R2C plan and its C2R plan this way:

From examples/gradient3d.c, lines 92-106

```
static vfft_plan plan_in_place(vfft_transform_t t)
{
    vfft_config_t c;
    vfft_plan p;
    memset(&c, 0, sizeof c);
    c.transform = t;
    c.rank = 3;
    c.n[0] = c.n[1] = c.n[2] = N;
    c.placement = VFFT_PLACEMENT_IN;
    p = vfft_create(&c);
    if (!p)
        fprintf(stderr, "gradient3d: vfft_create(%s): %s\n",
                t == VFFT_R2C ? "VFFT_R2C" : "VFFT_C2R", vfft_error());
    return p;
}
```

In place, the half spectrum overwrites the field, so each row of N reals is padded to hold the row's $N/2+1$ bins, two doubles each. Row (n_1, n_2) of the field and row (k_1, k_2) of the spectrum begin at the same double, and sample n_3 of the row follows:

$$\ell = (n_1 N + n_2)\, s + n_3, \tag{14.1}$$

where s is the row pitch in doubles. Bin k_3 of a row is the pair at doubles $2k_3$ and $2k_3+1$ of that row. Figure 14.1 shows one row before and after the transform.

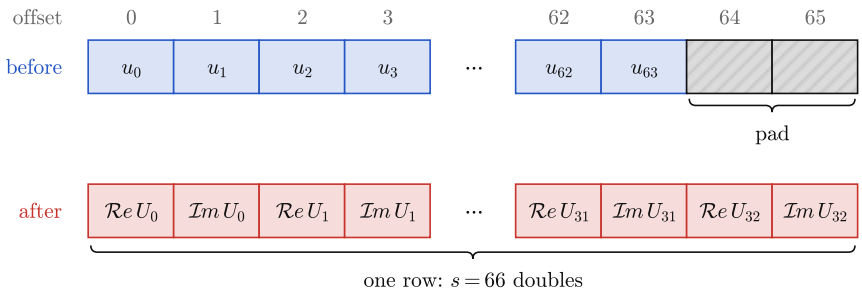


Figure 14.1 One row of the buffer of `gradient3d.c`, $N = 64$, before and after the in-place R2C. Before, the row holds the 64 samples u_{n_3} (blue) and two pad doubles (grey), which need no value. After, the same 66 doubles hold the row's $N/2 + 1 = 33$ bins U_{k_3} (red), each a (re, im) pair.

The pitch s is `vfft_info().stride`, and the size of the buffer is `vfft_info().in_doubles`: read both from each plan, never compute them. Without `VFFT_FLAG_OWN_PITCH` the pitch is the tight $2(N/2+1)$, but a plan that may choose its pitch can choose a wider one. Two plans share one buffer only when they report the same pitch and the same size, which the program checks before it allocates the field `U` and the second buffer `G`:

From examples/gradient3d.c, lines 135-137, 142-144

```
vfft_info(r2c, &fwd);
vfft_info(c2r, &bwd);
if (fwd.stride != bwd.stride || fwd.in_doubles != bwd.in_doubles)
    ...
    s = fwd.stride; /* no VFFT_FLAG_OWN_PITCH: the tight pitch, 2 (N/2 +
        1) */
    U = (double *)vfft_malloc(fwd.in_doubles * sizeof(double));
    G = (double *)vfft_malloc(fwd.in_doubles * sizeof(double));
```

The field goes into the rows at the index of Eq. (14.1), and the pad is left as it is:

From examples/gradient3d.c, lines 153-156

```
for (n1 = 0; n1 < N; n1++)
    for (n2 = 0; n2 < N; n2++)
        for (n3 = 0; n3 < N; n3++)
            U[((size_t)n1 * N + n2) * s + n3] = field(n1, n2, n3, du);
```

Rows written N doubles apart, as for an out-of-place plan, put every row after the first in the wrong place, and the plan transforms a different field.

14.3 Differentiate along each axis

The R2C runs once, in place. Then, for each axis a, the program copies the spectrum into `G`, because an in-place C2R overwrites its input, multiplies the copy, and runs the C2R on it in place:

From examples/gradient3d.c, lines 157-162

```
vfft_execute(r2c, VFFT_FORWARD, U, U);
for (a = 0; a < 3; a++)
{
    memcpy(G, U, fwd.in_doubles * sizeof(double)); /* the C2R consumes its
        input */
    times_j_kappa(G, s, a);
    vfft_execute(c2r, VFFT_BACKWARD, G, G);
```

The derivative along axis a multiplies each bin by $j\kappa_a$, where κ_a is the angular wavenumber of the bin along that axis. On this box of side 2π, $\kappa_a = \tilde{k}_a$, with $\tilde{k}_a = k_a$ for $k_a \le N/2$ and $k_a - N$ for $k_a > N/2$. The factor $1/N^3$ is the scale the C2R leaves out (Chapter 7):

$$G_a[k_1, k_2, k_3] = \begin{cases} j\kappa_a \, U[k_1, k_2, k_3]/N^3, & k_a \neq N/2, \\ 0, & k_a = N/2. \end{cases} \tag{14.2}$$

The bins with $k_a = N/2$ hold the frequency $N/2$ and $-N/2$ at once, so an odd multiplier such as $j\kappa_a$ has no single value there: set them to zero. An even multiplier, such as the $-|\kappa|^2$ of Chapter 12, needs no such rule.

From examples/gradient3d.c, lines 109-123

```
static void times_j_kappa(double *G, size_t stride, int a)
{
    const double scale = 1.0 / ((double)N * N * N);
    int k[3];
    for (k[0] = 0; k[0] < N; k[0]++)
        for (k[1] = 0; k[1] < N; k[1]++)
            for (k[2] = 0; k[2] <= N / 2; k[2]++)
            {
                double *z = G + ((size_t)k[0] * N + k[1]) * stride + 2 *
                    (size_t)k[2];
                const double g = k[a] == N / 2 ? 0.0 : (k[a] <= N / 2 ? k[a] :
                    k[a] - N) * scale;
                const double re = z[0];
                z[0] = -g * z[1]; /* (re + j im) j g */
                z[1] = g * re;
            }
}
```

After the C2R, `G` holds the derivative along axis a in the same padded rows: sample (n_1, n_2, n_3) at the ℓ of Eq. (14.1). The program's field has a term at the frequency $N/2$ of its second axis, so a multiply that keeps the plane $k_a = N/2$ fails the check; so does a wrong pitch, or a multiplier of the wrong sign, which negates every derivative.

14.4 Adapt the program

Other extents. Give the plans N_1, N_2, N_3 in `n[0]`, `n[1]`, `n[2]`. Row (n_1, n_2) then begins at $(n_1 N_2 + n_2)\, s$, a row holds $N_3/2 + 1$ bins, and on a box of sides L_a the multiplier is $j2\pi\tilde{k}_a/L_a$ with the scale $1/(N_1 N_2 N_3)$, zero where $k_a = N_a/2$. An odd extent has no bin $N_a/2$, and nothing to set to zero.

Let the plan choose the pitch. Add `VFFT_FLAG_OWN_PITCH` to `flags` in both configurations. The plan may then choose a wider pitch; the program reads it from `vfft_info()` as before, and still checks that both plans report the same one.

Out of place. Leave `placement` at zero, `VFFT_PLACEMENT_OUT`. The field then has rows of N_3 doubles and its spectrum a buffer of its own, as in Chapter 12, and each derivative takes a spectrum buffer for the product and a real buffer for the result: more memory, and no pitch to read.

Threads. Set `nthreads` in the configuration of both plans (Chapter 9). A thread count is part of the configuration, so a create at a new count can measure again.

15 Propagating a wave packet

You advance a complex field one step in time by filtering its spectrum: a forward transform, a phase factor on every bin, and the backward transform of the same plan, all in place in one buffer. The program is `wavepacket3d.c`: a quantum wave packet that moves freely on a periodic 64^3 grid, advanced by one step in time and checked against the packet's exact form.

15.1 Run the program

Build the examples as in Section 12.1, and run `wavepacket3d` from the build folder:

```
./wavepacket3d
```

The first run measures what the wisdom of your CPU lacks, for seconds, and saves the winner; later runs are served at once (Chapter 10). The program prints:

```
wavepacket3d 64^3 C2C in place, t = 1: max |y[n] - psi_t(nL/N)| = 2.95e-14,
    energy change 2.2e-14, tolerance 1e-12: PASS
```

The first number compares the field after the step with the exact packet at every grid point. The second is the change of the field's energy, $\sum_n |\psi[n]|^2$, which the step keeps. Both are rounding.

15.2 Create one plan for both directions

A `VFFT_C2C` plan runs both directions: `VFFT_FORWARD` computes the spectrum, and `VFFT_BACKWARD` the inverse without its $1/N^3$ (Chapter 6). `wavepacket3d.c` makes one rank-3 C2C plan that runs in place, and allocates the field with `vfft_malloc()`:

From examples/wavepacket3d.c, lines 120, 122, 130-141

```
const size_t vol = (size_t)N * N * N;
...
double complex *psi;
...
memset(&c, 0, sizeof c);
c.transform = VFFT_C2C;
c.rank = 3;
c.n[0] = c.n[1] = c.n[2] = N;
c.placement = VFFT_PLACEMENT_IN;
plan = vfft_create(&c);
if (!plan)
{
    fprintf(stderr, "wavepacket3d: vfft_create: %s\n", vfft_error());
    return 1;
}
psi = vfft_malloc(vol * sizeof *psi);
```

The field is a C99 `double complex` array of `vol` $= N^3$ elements. Each element is a (re, im) pair of doubles, the interleaved layout the plan reads, so the program passes `psi` to `vfft_execute()` as it is. Sample (n_1, n_2, n_3) is `psi[(n1 * N + n2) * N + n3]`, the last axis contiguous.

15.3 Filter the spectrum between the two transforms

The step is three passes over the one buffer: the forward transform, a multiply of every bin, and the backward transform. The equation numbers in the program's comments refer to its own header comment.

From examples/wavepacket3d.c, lines 157-165

```
    vfft_execute(plan, VFFT_FORWARD, psi, psi); /* X[k], eq. (2) */
    for (k[0] = 0, i = 0; k[0] < N; k[0]++)     /* Y[k] / N^3, eq. (3) */
        for (k[1] = 0; k[1] < N; k[1]++)
            for (k[2] = 0; k[2] < N; k[2]++, i++)
            {
                const double kk = kappa2(k[0]) + kappa2(k[1]) + kappa2(k[2]); /*
                    |kappa|^2 */
                psi[i] *= cexp(-I * T * kk / 2) / (double)vol;
            }
    vfft_execute(plan, VFFT_BACKWARD, psi, psi); /* y[n] */
```

The input is complex, so its spectrum has no symmetry to halve: all N^3 bins are stored, bin (k_1, k_2, k_3) at the index of sample (n_1, n_2, n_3). A step of length t multiplies every bin by a phase that depends on $|\kappa|^2$, and the $1/N^3$ that the backward transform leaves out is folded in:

$$Y[k_1, k_2, k_3] = \frac{1}{N^3}\, e^{-jt|\kappa|^2/2}\, X[k_1, k_2, k_3], \qquad \kappa_i = \frac{2\pi \tilde{k}_i}{L}, \tag{15.1}$$

where L is the side of the box, and $\tilde{k}_i = k_i$ for $k_i \le N/2$ and $k_i - N$ for $k_i > N/2$, the negative frequencies:

From examples/wavepacket3d.c, lines 97-101

```
static double kappa2(int k)
{
    const double kt = 2 * PI * (k <= N / 2 ? k : k - N) / L;
    return kt * kt;
}
```

The phase depends on κ_i^2 alone, so the bins $k_i = N/2$ need no rule of their own. In the code, `I` is the imaginary unit of `<complex.h>`, the j of Eq. (15.1). Reading the bins $k_i > N/2$ as positive frequencies, or leaving out the $1/N^3$, makes the result wrong by order one, and the program prints `FAIL`.

15.4 Adapt the program

Many steps. Keep the plan and the buffer, and repeat the three passes once per step. The phase factors depend on the bin alone, so you can compute them once, into a second array of N^3 elements, and multiply by it at each step.

Out of place. Leave `placement` at zero, `VFFT_PLACEMENT_OUT`, and pass two buffers of N^3 elements: `vfft_execute(plan, VFFT_FORWARD, psi, spec)`, the multiply on `spec`, then `vfft_execute(plan, VFFT_BACKWARD, spec, psi)`. Out of place, a plan leaves its input as it was, unless it was created with `VFFT_FLAG_DESTROY_INPUT`.

Other sizes. Change `N` and the box side `L`. For a box with an extent of its own on each axis, give the plan N_1, N_2, N_3 in `n[0]`, `n[1]`, `n[2]`: sample (n_1, n_2, n_3) then sits at `(n1 * N2 + n2) * N3 + n3`, bin (k_1, k_2, k_3) at the same index, and each axis has its own N_i and L_i in κ_i.

Threads. Set `nthreads` in the configuration (Chapter 9). A thread count is part of the configuration, so a create at a new count can measure again.

Part IV **Moving a DFTI program**

16 Switching a DFTI program to VectorFFT

You have a C program written for Intel MKL's DFTI interface. In this chapter you build it against VectorFFT without changing its source, run it, and keep MKL linked beside it when the program also calls MKL for BLAS or LAPACK. Chapter 17 lists what the interface serves.

16.1 Change the build

VectorFFT's DFTI interface is two files in the folder `src/shims/dfti/` of the VectorFFT sources: `mkl_dfti.h`, a header with DFTI's names and constants, and `vfft_dfti.c`, which runs DFTI's calls on VectorFFT plans. The install of Chapter 1 does not copy them, so your build takes them from the sources, the folder `<VectorFFT>` in the commands of this chapter, and links the library installed in `<prefix>`. Three things change, as Figure 16.1 shows:

Step 1. Compile `vfft_dfti.c` together with your program.

Step 2. Put its folder, `<VectorFFT>/src/shims/dfti`, first on the include path, so that `#include "mkl_dfti.h"` finds VectorFFT's header.

Step 3. Link VectorFFT instead of MKL.

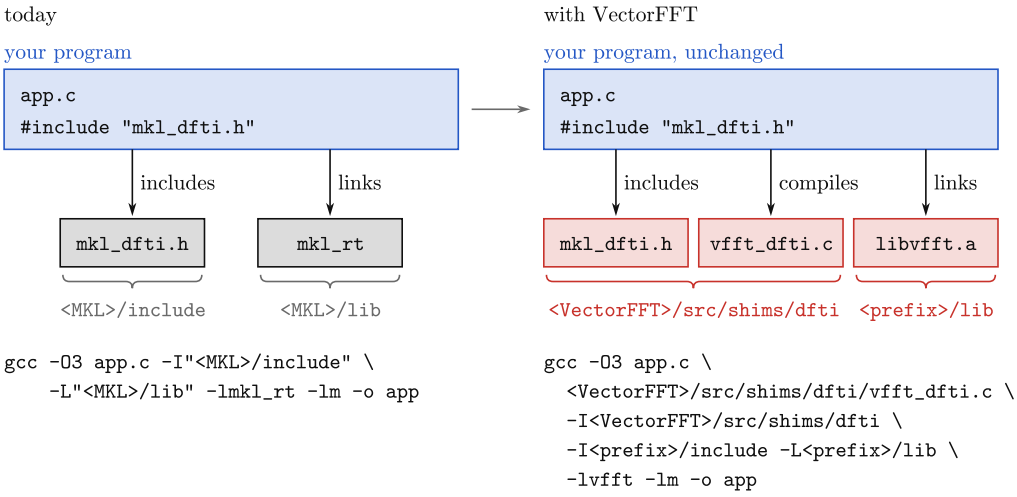


Figure 16.1 The build of a DFTI program today and with VectorFFT. Your program (blue) does not change; the three pieces that change (red) are the `mkl_dfti.h` of `src/shims/dfti/`, found ahead of MKL's include folder, `vfft_dfti.c` compiled with the program, and `libvfft.a` linked in place of MKL.

Your source files stay as they are. VectorFFT's `mkl_dfti.h` gives DFTI's names to functions of its own, named `vfft_dfti_...`, so a program compiled against it calls no DFTI function of MKL. A program that builds against MKL with one command, `<MKL>` being the folder MKL is installed in,

```
gcc -O3 app.c -I"<MKL>/include" -L"<MKL>/lib" -lmkl_rt -lm -o app
```

builds against VectorFFT with the command of Chapter 1, the C file and its folder added:

```
gcc -O3 app.c <VectorFFT>/src/shims/dfti/vfft_dfti.c \
    -I<VectorFFT>/src/shims/dfti -I<prefix>/include \
    -L<prefix>/lib -lvfft -lm -o app
```

In a CMake project, the build against MKL

```
cmake_minimum_required(VERSION 3.16)
project(app LANGUAGES C)
find_package(MKL CONFIG REQUIRED)
add_executable(app app.c)
target_link_libraries(app PRIVATE MKL::MKL m)
```

becomes

```
cmake_minimum_required(VERSION 3.16)
project(app LANGUAGES C)
find_package(VectorFFT CONFIG REQUIRED)
set(DFTI_DIR "<VectorFFT>/src/shims/dfti")
add_executable(app app.c "${DFTI_DIR}/vfft_dfti.c")
target_include_directories(app PRIVATE "${DFTI_DIR}")
target_link_libraries(app PRIVATE VectorFFT::vfft m)
```

Configure it with a build type and the prefix, and build it:

```
cmake -S . -B build -G Ninja -DCMAKE_C_COMPILER=gcc \
      -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=<prefix>
cmake --build build
```

Without a build type, CMake compiles `vfft_dfti.c` without optimization. What Section 1.7 says about running programs on Windows holds for these programs too: link with `-static`, or put `libwinpthread-1.dll` beside the program.

16.2 Build and run a DFTI program

The program `ch16_dfti_c2c.c` is a DFTI program as you write it for MKL. It creates a descriptor for $N = 1024$ complex points out of place, sets `DFTI_BACKWARD_SCALE` to $1/N$, commits the descriptor, and computes forward and backward. With the scales σ_f, the value of `DFTI_FORWARD_SCALE`, and σ_b, the value of `DFTI_BACKWARD_SCALE`, both 1 unless set, the two directions compute

$$\text{Forward:} \qquad X[k] = \sigma_f \sum_{n=0}^{N-1} x[n]\, W_N^{kn}, \qquad 0 \le k \le N-1, \tag{16.1a}$$

$$\text{Backward:} \qquad y[n] = \sigma_b \sum_{k=0}^{N-1} X[k]\, W_N^{-kn}, \qquad 0 \le n \le N-1, \tag{16.1b}$$

so with $\sigma_b = 1/N$ the backward returns the input, $y[n] = x[n]$. With both scales 1 the backward returns $N\,x[n]$, as `VFFT_BACKWARD` does (Chapter 2). At rank 2 or 3 the factor is the number of points, N_1N_2 or $N_1N_2N_3$.

ch16_dfti_c2c.c

```
/* ch16_dfti_c2c.c: a DFTI program. One 1D complex DFT of N = 1024 points,
 * out of place, forward and then backward with DFTI_BACKWARD_SCALE = 1/N,
 * checked against a direct evaluation of the DFT and against the input.
 */
#include <complex.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "mkl_dfti.h"

enum { N = 1024 };
static const double TOL = 1e-12;

int main(void)
```

```
{
    const double pi = 3.14159265358979323846;
    double complex *x = malloc(N * sizeof *x);
    double complex *X = malloc(N * sizeof *X);
    double complex *y = malloc(N * sizeof *y);
    if (!x || !X || !y)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (int n = 0; n < N; n++)
        x[n] = cos(0.1 * n) + I * sin(0.3 * n);

    DFTI_DESCRIPTOR_HANDLE h = NULL;
    MKL_LONG status = DftiCreateDescriptor(&h, DFTI_DOUBLE, DFTI_COMPLEX, 1,
                                           (MKL_LONG)N);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(h, DFTI_PLACEMENT, DFTI_NOT_INPLACE);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(h, DFTI_BACKWARD_SCALE, 1.0 / N);
    if (status == DFTI_NO_ERROR)
        status = DftiCommitDescriptor(h);
    if (status == DFTI_NO_ERROR)
        status = DftiComputeForward(h, x, X);       /* X[k] */
    if (status == DFTI_NO_ERROR)
        status = DftiComputeBackward(h, X, y);      /* y[n] = x[n] */
    if (status != DFTI_NO_ERROR)
    {
        fprintf(stderr, "DFTI: %s\n", DftiErrorMessage(status));
        return 1;
    }
    DftiFreeDescriptor(&h);

    /* X[k] against the sum of x[n] W_N^{kn}; y[n] against x[n] */
    double err_X = 0.0, err_y = 0.0, top = 0.0;
    for (int k = 0; k < N; k++)
    {
        double complex ref = 0.0;
        for (int n = 0; n < N; n++)
            ref += x[n] * cexp(-2.0 * pi * I * ((k * n) % N) / N);
        double d = cabs(X[k] - ref);
        if (d > err_X || isnan(d))
            err_X = d;
        if (cabs(ref) > top)
            top = cabs(ref);
        d = cabs(y[k] - x[k]);
        if (d > err_y || isnan(d))
            err_y = d;
    }
    err_X /= top;
    int pass = err_X <= TOL && err_y <= TOL;    /* false for a NaN */
    printf("1D complex DFT of %d points, DFTI_BACKWARD_SCALE = 1/N\n", N);
    printf("forward:  max |X[k] - X_ref[k]| / max |X_ref[k]| = %.1e\n", err_X);
    printf("backward: max |y[n] - x[n]| = %.1e\n", err_y);
    printf("tolerance %.0e: %s\n", TOL, pass ? "PASS" : "FAIL");

    free(y);
    free(X);
    free(x);
    return pass ? 0 : 1;
}
```

Build it as `app.c` with the CMake project of Section 16.1, and run it:

```
./build/app
```

Built with the one command instead, the program is `./app`. It prints:

```
1D complex DFT of 1024 points, DFTI_BACKWARD_SCALE = 1/N
forward:  max |X[k] - X_ref[k]| / max |X_ref[k]| = 2.0e-15
backward: max |y[n] - x[n]| = 8.0e-16
tolerance 1e-12: PASS
```

`DftiCommitDescriptor()` is where the planning happens: it creates the descriptor's plans with `vfft_create()` (Chapter 4). A commit whose configuration has no wisdom in the store measures for seconds, up to about a minute for some configurations, and saves the winner before it returns; a later commit of the same configuration, in the same run or a later one, is served from the store at once. A DFTI program uses the library's wisdom folder as any program does (Section 10.2). The DFTI interface has no setting that names a store, so a DFTI program that needs one names it in `VFFT_WISDOM_DIR` (Section 10.3). Commit a descriptor once and compute with it as often as you need. A successful `DftiSetValue()` uncommits the descriptor; the next commit keeps its plans when the change leaves them as they were, as a new scale does, and creates them again otherwise.

When the switch goes wrong:

- **A commit returns 3 or 6.** The descriptor asks for something VectorFFT does not serve. A line on standard error that starts with `vfft dfti:` says what, and Chapter 17 lists what is served.
- **The program still calls MKL.** A file found MKL's `mkl_dfti.h` instead of VectorFFT's; Section 16.5 shows how to check.
- **A first run takes seconds.** Its commits measured what the wisdom of your CPU lacks; a second run is served at once (Chapter 10).

16.3 Run a 3D real transform

The program `ch16_dfti_real3d.c` transforms a real array of $16 \times 32 \times 64$ points out of place. The forward reads the reals and writes the half spectrum, $H = N_3/2 + 1$ complex bins in each row (Chapter 7); the backward reads the half spectrum and writes reals. In DFTI the strides belong to the call: a transform reads through `DFTI_INPUT_STRIDES` and writes through `DFTI_OUTPUT_STRIDES`, each counted in the elements it reads or writes. The forward reads the reals through `rs`, $\{0, N_2N_3, N_3, 1\}$ in real elements, and writes the bins through `cs`, $\{0, N_2H, H, 1\}$ in complex elements; the backward reads through `cs` and writes through `rs`. The program keeps one descriptor for each direction, both made by its function `describe()`:

ch16_dfti_real3d.c

```
/* ch16_dfti_real3d.c: a DFTI program. One 3D real DFT of 16 x 32 x 64
 * points, out of place: one descriptor for the forward (the reals in, the
 * half spectrum out) and one for the backward, whose strides are the
 * forward's swapped and whose DFTI_BACKWARD_SCALE is 1/(N1 N2 N3). The input
 * is a constant plus one plane wave, so its spectrum is known; the backward
 * must return the input.
 */
#include <complex.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "mkl_dfti.h"

enum { N1 = 16, N2 = 32, N3 = 64, H = N3 / 2 + 1 };
```

```
enum { K1 = 3, K2 = 5, K3 = 7 };              /* the plane wave's bin */
static const double TOL = 1e-12;

/* a descriptor of the 3D real transform, out of place, that reads through
 * the strides in and writes through the strides out */
static MKL_LONG describe(DFTI_DESCRIPTOR_HANDLE *h, MKL_LONG *in,
                         MKL_LONG *out)
{
    MKL_LONG n[3] = {N1, N2, N3};
    MKL_LONG status = DftiCreateDescriptor(h, DFTI_DOUBLE, DFTI_REAL, 3, n);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(*h, DFTI_PLACEMENT, DFTI_NOT_INPLACE);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(*h, DFTI_CONJUGATE_EVEN_STORAGE,
                              DFTI_COMPLEX_COMPLEX);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(*h, DFTI_INPUT_STRIDES, in);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(*h, DFTI_OUTPUT_STRIDES, out);
    return status;
}

int main(void)
{
    const double pi = 3.14159265358979323846;
    const double P = (double)N1 * N2 * N3;
    double *x = malloc(sizeof(double) * N1 * N2 * N3);
    double *y = malloc(sizeof(double) * N1 * N2 * N3);
    double complex *X = malloc(sizeof(double complex) * N1 * N2 * H);
    if (!x || !y || !X)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    /* x = 1 + cos(2 pi (K1 n1 / N1 + K2 n2 / N2 + K3 n3 / N3)), the phase
     * counted in steps of 2 pi / (N1 N2 N3) and reduced exactly */
    for (int n1 = 0; n1 < N1; n1++)
        for (int n2 = 0; n2 < N2; n2++)
            for (int n3 = 0; n3 < N3; n3++)
            {
                int m = (K1 * n1 * N2 * N3 + K2 * n2 * N1 * N3 +
                         K3 * n3 * N1 * N2) % (N1 * N2 * N3);
                x[(n1 * N2 + n2) * N3 + n3] = 1.0 + cos(2.0 * pi * m / P);
            }

    MKL_LONG rs[4] = {0, N2 * N3, N3, 1};      /* the reals: rows of N3 */
    MKL_LONG cs[4] = {0, N2 * H, H, 1};        /* the bins: rows of H */
    DFTI_DESCRIPTOR_HANDLE fwd = NULL, bwd = NULL;
    MKL_LONG status = describe(&fwd, rs, cs);
    if (status == DFTI_NO_ERROR)
        status = DftiCommitDescriptor(fwd);
    if (status == DFTI_NO_ERROR)
        status = describe(&bwd, cs, rs);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(bwd, DFTI_BACKWARD_SCALE, 1.0 / P);
    if (status == DFTI_NO_ERROR)
        status = DftiCommitDescriptor(bwd);
    if (status == DFTI_NO_ERROR)
        status = DftiComputeForward(fwd, x, X);
    if (status == DFTI_NO_ERROR)
        status = DftiComputeBackward(bwd, X, y);
    if (status != DFTI_NO_ERROR)
    {
        fprintf(stderr, "DFTI: %s\n", DftiErrorMessage(status));
```

```
        return 1;
    }
    DftiFreeDescriptor(&bwd);
    DftiFreeDescriptor(&fwd);

    /* the spectrum: X[0,0,0] = N1 N2 N3, X[K1,K2,K3] = N1 N2 N3 / 2, every
     * other stored bin 0; the backward: y = x */
    double err_X = 0.0, err_y = 0.0;
    for (int k1 = 0; k1 < N1; k1++)
        for (int k2 = 0; k2 < N2; k2++)
            for (int k3 = 0; k3 < H; k3++)
            {
                double ref = 0.0;
                if (k1 == 0 && k2 == 0 && k3 == 0)
                    ref = P;
                if (k1 == K1 && k2 == K2 && k3 == K3)
                    ref = P / 2;
                double d = cabs(X[(k1 * N2 + k2) * H + k3] - ref) / P;
                if (d > err_X || isnan(d))
                    err_X = d;
            }
    for (int i = 0; i < N1 * N2 * N3; i++)
    {
        double d = fabs(y[i] - x[i]);
        if (d > err_y || isnan(d))
            err_y = d;
    }
    int pass = err_X <= TOL && err_y <= TOL;    /* false for a NaN */
    printf("3D real DFT of %d x %d x %d points, one descriptor per "
           "direction\n", N1, N2, N3);
    printf("forward:  max |X - X_ref| / (N1 N2 N3) = %.1e\n", err_X);
    printf("backward: max |y - x| = %.1e\n", err_y);
    printf("tolerance %.0e: %s\n", TOL, pass ? "PASS" : "FAIL");

    free(X);
    free(y);
    free(x);
    return pass ? 0 : 1;
}
```

Built and run the same way, it prints:

```
3D real DFT of 16 x 32 x 64 points, one descriptor per direction
forward:  max |X - X_ref| / (N1 N2 N3) = 7.0e-17
backward: max |y - x| = 8.9e-16
tolerance 1e-12: PASS
```

A real descriptor of rank 2 or 3 serves the one direction its strides describe: `DftiComputeBackward()` on `fwd` returns 3, `DFTI_INCONSISTENT_CONFIGURATION`. Keep one descriptor for each direction, as here, and commit each once. A program that runs both directions on one descriptor swaps the strides and commits again before each change of direction, and each such commit creates a plan again. Section 17.4 gives every layout of real data.

16.4 Pass the values at DFTI's types

In VectorFFT's `mkl_dfti.h`, `MKL_LONG` is `long`, or `long long` when `MKL_ILP64` is defined. Compile your files and `vfft_dfti.c` with the same setting.

`DftiCreateDescriptor()` and `DftiSetValue()` take their values as variable arguments and read each one at the type of its parameter: an integer as `MKL_LONG`, a scale as `double`, lengths and strides as an array of `MKL_LONG`, a named value such as

`DFTI_NOT_INPLACE` as `int`. Pass them at those types, as the listings do: `(MKL_LONG)N` for a length, arrays of `MKL_LONG` for lengths and strides, `(MKL_LONG)K` for a number of transforms, and `1.0 / N`, not `1 / N`, for a scale.

16.5 Keep MKL for BLAS and LAPACK

A program that also calls MKL, for BLAS or LAPACK, keeps MKL in its build and takes only DFTI from VectorFFT:

- Put `<VectorFFT>/src/shims/dfti` before MKL's include folder. A file then finds VectorFFT's `mkl_dfti.h`, and MKL's other headers in MKL's folder.
- In a file that calls DFTI, include the MKL headers it needs by name, such as `mkl_cblas.h` or `mkl_lapacke.h`, not `mkl.h`: `mkl.h` includes MKL's own `mkl_dfti.h` from its folder, and the file then calls MKL's DFTI.
- Build every file that calls DFTI this way, so that each descriptor VectorFFT creates reaches VectorFFT's functions only.

With one command, the folders come in that order, and the link line takes your files and `vfft_dfti.c` first, then VectorFFT, then MKL:

```
gcc -O3 app.c <VectorFFT>/src/shims/dfti/vfft_dfti.c \
    -I<VectorFFT>/src/shims/dfti -I"<MKL>/include" -I<prefix>/include \
    -L<prefix>/lib -lvfft -L"<MKL>/lib" -lmkl_rt -lm -o app
```

With CMake, the project links both packages:

```
cmake_minimum_required(VERSION 3.16)
project(app LANGUAGES C)
find_package(VectorFFT CONFIG REQUIRED)
find_package(MKL CONFIG REQUIRED)
set(DFTI_DIR "<VectorFFT>/src/shims/dfti")
add_executable(app app.c "${DFTI_DIR}/vfft_dfti.c")
target_include_directories(app PRIVATE "${DFTI_DIR}")
target_link_libraries(app PRIVATE VectorFFT::vfft MKL::MKL m)
```

CMake passes the folder of `target_include_directories()` to the compiler with `-I`, and MKL's include folder, which comes with `MKL::MKL`, with `-isystem`. GCC searches every `-I` folder first, so each file finds VectorFFT's `mkl_dfti.h`; do not add MKL's include folder to `target_include_directories()`. MKL's package compiles every source of the target, `vfft_dfti.c` included, with `MKL_ILP64` defined unless you configure it with `-DMKL_INTERFACE=lp64`, as the command below does; either way your files and `vfft_dfti.c` get the same setting (Section 16.4). With MinGW-w64, also pass `-DMKL_LINK=sdl`, which links MKL through its single library, `mkl_rt`:

```
cmake -S . -B build -G Ninja -DCMAKE_C_COMPILER=gcc \
      -DCMAKE_BUILD_TYPE=Release "-DCMAKE_PREFIX_PATH=<prefix>;<MKL>" \
      -DMKL_INTERFACE=lp64 -DMKL_LINK=sdl
cmake --build build
```

Then check the program. On Windows, `objdump -p` lists the functions a program takes from each DLL, and no DFTI function may come from MKL's:

```
objdump -p build/app.exe | grep Dfti
```

For a program built as here, it prints nothing. For the same program built against MKL alone, it prints the DFTI functions the program calls, such as `DftiCommitDescriptor` and `DftiComputeForward`, which then come from an MKL

library such as `mkl_rt.2.dll`. At run time, `DftiGetValue(h, DFTI_VERSION, text)` on a descriptor `h` fills `text`, an array of `DFTI_VERSION_LENGTH` characters, with a text that starts with `VectorFFT`.

17 What the DFTI interface supports

Before you switch a program (Chapter 16), check what it asks of DFTI against the tables of this chapter: the functions it calls, each value it sets, the layout its strides describe and the threads it uses. The interface computes in double precision, on complex data stored as pairs of doubles, at ranks 1 to 3. What it does not serve, it refuses: the call returns a status other than `DFTI_NO_ERROR`, nothing is converted, and most refusals print a line on standard error that says why (Section 17.6).

17.1 Look up a function

Table 17.1 lists every DFTI function and what the interface does with it.

Table 17.1 The DFTI functions in VectorFFT's interface.

Function	What it does
`DftiCreateDescriptor`	a descriptor of precision `DFTI_DOUBLE`, domain `DFTI_COMPLEX` or `DFTI_REAL`, rank 1 to 7: one length at rank 1, an array of lengths above; its refusals are in Table 17.2
`DftiCreateDescriptor_d_1d`, `DftiCreateDescriptor_d_md`	the same in double precision
`DftiCreateDescriptor_s_1d`, `DftiCreateDescriptor_s_md`	refused with 6: single precision
`DftiSetValue`, `DftiGetValue`	set and read a parameter (Table 17.2); a set that succeeds uncommits the descriptor
`DftiCommitDescriptor`	creates `DFTI_NUMBER_OF_USER_THREADS` plans with `vfft_create()`: for complex data one set for both directions, for real data one set for each direction the strides serve. A configuration with no wisdom in the store is measured first. Like `vfft_create()`, it leaves flush-to-zero and denormals-are-zero on in the calling thread (Chapter 4). On a committed descriptor it returns at once
`DftiComputeForward`, `DftiComputeBackward`	run a plan in place on one buffer, or out of place from an input buffer to an output buffer, then apply the direction's scale; pass buffers that are not `NULL`. They return 2 on an uncommitted descriptor and 3 in a direction the committed strides do not serve
`DftiCopyDescriptor`	copies the settings; the copy of a committed descriptor is committed, with plans of its own
`DftiFreeDescriptor`	frees the plans and the descriptor, and sets the handle to `NULL`
`DftiErrorMessage`	the text of a status (Table 17.6)
`DftiErrorClass`	1 when the status equals the class, else 0

17.2 Look up a configuration parameter

The type of a value follows its parameter: a named value such as `DFTI_INPLACE` is an `int`, an integer an `MKL_LONG`, a scale a `double`, lengths and strides an array of `MKL_LONG` (Section 16.4). Table 17.2 gives each parameter, value and call one of three statuses: ● served as DFTI defines it, the note giving the values, the default first; ◐ served with a difference, the note saying what differs; ○ refused, the note giving the status returned and the call that refuses. Where the interface prints a line on standard error, the note quotes it, filled in for one example where it names a rank, a count or a placement; a refusal without a quoted line prints one that names the setting, unless the note says no line (Section 17.6).

Table 17.2 The parameters, values and calls of the interface, each with its status.

Parameter or value	Status	Note
`DFTI_FORWARD_DOMAIN`, `DFTI_DIMENSION`, `DFTI_LENGTHS`, `DFTI_PRECISION`	● served	set by `DftiCreateDescriptor`; read only
`DFTI_NUMBER_OF_TRANSFORMS`	● served	1, or more for a batch (Section 17.3)
`DFTI_PLACEMENT`	● served	`DFTI_INPLACE`, `DFTI_NOT_INPLACE`
`DFTI_INPUT_STRIDES`, `DFTI_OUTPUT_STRIDES`	● served	rank + 1 values; by default the row-major strides of the lengths, $\{0, N_2N_3, N_3, 1\}$ at rank 3, on both sides; Section 17.3 and Section 17.4 list the layouts served
`DFTI_INPUT_DISTANCE`, `DFTI_OUTPUT_DISTANCE`	● served	0; a batch sets them
`DFTI_COMPLEX_STORAGE`	● served	`DFTI_COMPLEX_COMPLEX`
`DFTI_REAL_STORAGE`	● served	`DFTI_REAL_REAL`
`DFTI_CONJUGATE_EVEN_STORAGE`	● served	`DFTI_COMPLEX_COMPLEX`: the half spectrum as complex values
`DFTI_PACKED_FORMAT`	● served	`DFTI_CCE_FORMAT`
`DFTI_DESTROY_INPUT`	● served	`DFTI_AVOID`, `DFTI_ALLOW`: an out-of-place transform may overwrite its input
`DFTI_DESCRIPTOR_NAME`	● served	a text, of which the first `DFTI_MAX_NAME_LENGTH` (10) characters are kept
`DFTI_TRANSPOSE`	● served	read only: `DFTI_NONE`
`DFTI_COMMIT_STATUS`	● served	read only: `DFTI_COMMITTED`, `DFTI_UNCOMMITTED`
`DFTI_VERSION`	● served	read only, into an array of `DFTI_VERSION_LENGTH` characters: a text that starts with `VectorFFT`
`DFTI_FORWARD_SCALE`, `DFTI_BACKWARD_SCALE`	◐ served with a difference	a `double`, 1.0; a scale other than 1 is one more pass over the output after the transform, on the calling thread
`DFTI_ORDERING`	◐ served with a difference	`DFTI_ORDERED`, `DFTI_BACKWARD_SCRAMBLED`: the output is in natural order with either value, and the value reads back as set. A commit with `DFTI_BACKWARD_SCRAMBLED` prints `vfft dfti: DFTI_ORDERING = DFTI_BACKWARD_SCRAMBLED: the output is in natural order, which DFTI permits`
`DFTI_WORKSPACE`	◐ served with a difference	`DFTI_ALLOW`, `DFTI_AVOID`: kept, with no effect
`DFTI_THREAD_LIMIT`	◐ served with a difference	0 or more, 0: the number of threads each plan is created for, not an upper limit; read after a commit, the number the plans were created for (Section 17.5)

Parameter or value	Status	Note
`DFTI_NUMBER_OF_USER_THREADS`	◐ served with a difference	1 or more, 1: the number of plans, one for each thread that computes with the descriptor at once (Section 17.5). Further threads take turns, and the first compute that waits prints `vfft dfti: one descriptor is computed from more threads at once than DFTI_NUMBER_OF_USER_THREADS = 1: the computes take turns; set it to the number of threads`, with the number of plans, the value set up to 64, in place of 1
`DFTI_SINGLE`; `DftiCreateDescriptor_s_1d`, `DftiCreateDescriptor_s_md`	○ refused	6, `DftiCreateDescriptor`: `vfft dfti: DftiCreateDescriptor: single precision is not served (VectorFFT computes in double precision)`
`DFTI_COMPLEX_STORAGE` = `DFTI_REAL_REAL`	○ refused	6, `DftiSetValue`
`DFTI_CONJUGATE_EVEN_STORAGE` = `DFTI_REAL_REAL`	○ refused	6, `DftiSetValue`
`DFTI_CONJUGATE_EVEN_STORAGE` = `DFTI_COMPLEX_REAL`	○ refused	6 at rank 1, else 3, `DftiSetValue`: `vfft dfti: DftiSetValue: DFTI_CONJUGATE_EVEN_STORAGE = DFTI_COMPLEX_REAL (the packed real formats) is not served; use DFTI_COMPLEX_COMPLEX`
`DFTI_PACKED_FORMAT` = `DFTI_CCS_FORMAT`, `DFTI_PACK_FORMAT` or `DFTI_PERM_FORMAT`	○ refused	6, `DftiSetValue`: `vfft dfti: DftiSetValue: DFTI_PACKED_FORMAT CCS, PACK and PERM are not served; use DFTI_CCE_FORMAT`
`DFTI_REAL_STORAGE` = `DFTI_REAL_COMPLEX`	○ refused	6, `DftiSetValue`; no line
`DFTI_FWD_DISTANCE`, `DFTI_BWD_DISTANCE`	○ refused	3, `DftiSetValue`; no line
a read-only parameter; a value DFTI does not define for the parameter; a count below 1; a negative `DFTI_THREAD_LIMIT`; a `NULL` array or text	○ refused	3, `DftiSetValue`; no line
`DFTI_DIMENSION` above 3	○ refused	6, `DftiCommitDescriptor`
a batch without its distances	○ refused	3, `DftiCommitDescriptor`: `vfft dfti: DftiCommitDescriptor: a batch (DFTI_NUMBER_OF_TRANSFORMS > 1) needs DFTI_INPUT_DISTANCE and DFTI_OUTPUT_DISTANCE`; for complex data in place, `... needs DFTI_INPUT_DISTANCE`

Parameter or value	Status	Note
strides and distances that are not a layout of Section 17.3 or Section 17.4	○ refused	6, `DftiCommitDescriptor`: `vfft dfti: DftiCommitDescriptor: complex rank 1, 1 transform(s), out of place: the strides and distances are not a layout VectorFFT serves (tight strides for one transform or contiguous transforms; the lane-major batch, strides times the batch count and distance 1; real data in place also with padded rows; docs/roadmap/dfti_shim_design.md section 3)`
real data in place on DFTI's default strides	○ refused	3, `DftiCommitDescriptor`: `vfft dfti: DftiCommitDescriptor: an in-place real transform of rank 2 needs the padded strides (forward: DFTI_INPUT_STRIDES {0, ..., 2(n/2+1), 1}, DFTI_OUTPUT_STRIDES {0, ..., n/2+1, 1}), as DFTI requires`
real data out of place on DFTI's default strides, at rank 2 or 3	○ refused	6, `DftiCommitDescriptor`: `vfft dfti: DftiCommitDescriptor: an out-of-place real transform of rank 2 writes packed rows: set DFTI_OUTPUT_STRIDES to {0, ..., n/2+1, 1} (DFTI's default output rows are padded to the real pitch, which VectorFFT does not write)`
`DFTI_THREAD_LIMIT` above the logical CPUs of the machine	○ refused	6, `DftiCommitDescriptor`: `DftiCommitDescriptor:` and the reason `vfft_create()` gives (Chapter 11), here `vfft dfti: DftiCommitDescriptor: vfft_create: nthreads=33 is outside the pool's range: 0 or 1 (one thread) up to 32 (this machine's 32 logical CPUs, at most 64 dispatched workers)`
a compute in a direction the committed strides do not serve	○ refused	3, `DftiComputeForward` or `DftiComputeBackward`: `vfft dfti: DftiComputeBackward: the committed strides and distances serve the forward transform only; swap DFTI_INPUT_STRIDES and DFTI_OUTPUT_STRIDES (and the distances) and recommit, as DFTI requires`; for `DftiComputeForward`, `... serve the backward transform only ...`

17.3 Describe complex data with strides and distances

A descriptor finds its data through strides and distances: element (n_1, n_2, n_3) of transform t of a batch is element

$$\ell = s_0 + n_1 s_1 + n_2 s_2 + n_3 s_3 + t\,d \tag{17.1}$$

of the buffer, with s_0, s_1, s_2, s_3 the values of `DFTI_INPUT_STRIDES` for what a transform reads and of `DFTI_OUTPUT_STRIDES` for what it writes, and d `DFTI_INPUT_DISTANCE` or `DFTI_OUTPUT_DISTANCE`; at rank 2 the strides end with s_2, at rank 1 with s_1. A complex element is two doubles, and complex data may start at any $s_0 \geq 0$.

The commit serves the three layouts of Table 17.3, the batches of Chapter 8 in DFTI's terms, and refuses any other with 6: a stride with gaps, such as rows padded to a longer pitch, a stride of 2 or -1, or another distance. A batch of K transforms sets `DFTI_NUMBER_OF_TRANSFORMS` to K and the distances, out of place both and in place `DFTI_INPUT_DISTANCE` alone; in place, the output strides and distance are not read. At a lower rank the pattern keeps fewer lengths: one transform or a contiguous batch of rank 2 takes $\{s_0, N_2, 1\}$ and of rank 1 $\{s_0, 1\}$, the batch with the distance $N_1 N_2$ or N; a lane-major batch of rank 2 takes $\{s_0, KN_2, K\}$ and one of rank 1 $\{s_0, K\}$.

Table 17.3 The layouts of complex data, with their strides at rank 3; the input and the output take the same strides.

Layout	Ranks	Strides, in complex elements	Distance
one transform	1 to 3	$\{s_0, N_2N_3, N_3, 1\}$	not read
contiguous batch of K	1 to 3	$\{s_0, N_2N_3, N_3, 1\}$	$N_1N_2N_3$
lane-major batch of K	1 to 3	$\{s_0, KN_2N_3, KN_3, K\}$	1

17.4 Store real data

A real descriptor, domain `DFTI_REAL`, transforms reals into the half spectrum: each row of N_{last} reals, N_{last} the last length, gives $H = \lfloor N_{\text{last}}/2 \rfloor + 1$ complex bins (Chapter 7), kept as complex values, `DFTI_CONJUGATE_EVEN_STORAGE` = `DFTI_COMPLEX_COMPLEX`. The strides of the bins count complex elements, those of the reals real elements. Out of place, the reals and the bins sit in rows of their own lengths; in place, the reals sit in rows padded to $2H$ doubles, and the forward writes each row's H bins over the same doubles (Figure 17.1).

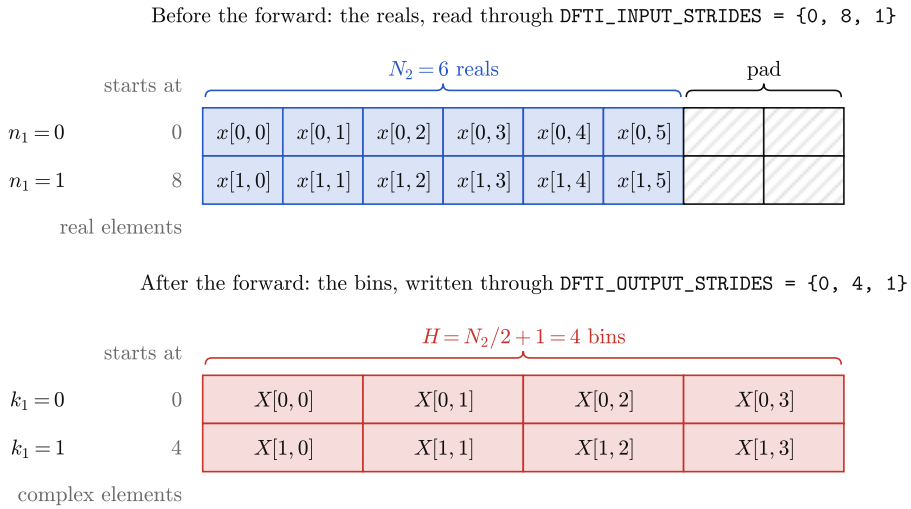


Figure 17.1 A real array of 2×6 points in place. Each row takes $2H = 8$ doubles, $H = 6/2 + 1 = 4$: its six reals (blue), then two doubles of pad (hatched grey). The forward reads row n_1 from real element $8n_1$ and writes row k_1 of the bins (red) from complex element $4k_1$, the same double.

Table 17.4 gives the layouts. A descriptor that reads reals and writes bins serves the forward, one that reads bins and writes reals the backward; one descriptor serves both only where the reals and the bins are read alike, for one transform of rank 1 and the lane-major batch of rank 1. Everywhere else a commit serves one direction, and a compute in the other returns 3: keep one descriptor for each direction, as the listing of this section does, or swap the strides and the distances and commit again before each change of direction.

Table 17.4 The layouts of real data, with their strides at rank 3.

Layout	Ranks	Reals, in real elements	Bins, in complex elements	Distances, reals / bins
one transform, out of place	1 to 3	$\{s_0, N_2N_3, N_3, 1\}$	$\{s_0, N_2H, H, 1\}$	not read
one transform, in place	1 to 3	$\{0, 2N_2H, 2H, 1\}$	$\{0, N_2H, H, 1\}$	not read
contiguous batch, out of place	1 to 3	$\{s_0, N_2N_3, N_3, 1\}$	$\{s_0, N_2H, H, 1\}$	$N_1N_2N_3$ / N_1N_2H
contiguous batch, in place	1 to 3	$\{0, 2N_2H, 2H, 1\}$	$\{0, N_2H, H, 1\}$	$2N_1N_2H$ / N_1N_2H
lane-major batch, out of place	1 to 3	$\{s_0, KN_2N_3, KN_3, K\}$	$\{s_0, KN_2H, KH, K\}$	1 / 1
lane-major batch, in place	1 to 3	$\{0, 2KN_2H, 2KH, K\}$	$\{0, KN_2H, KH, K\}$	1 / 1

At rank 2 the pattern drops N_2N_3 and N_2H: one transform or a contiguous batch takes $\{s_0, N_2, 1\}$ and $\{s_0, H, 1\}$ out of place, and $\{0, 2H, 1\}$ and $\{0, H, 1\}$ in place, as the listing does, the batch with the distances N_1N_2 / N_1H out of place and $2N_1H$ / N_1H in place; a lane-major batch takes $\{s_0, KN_2, K\}$ and $\{s_0, KH, K\}$ out of place, $\{0, 2KH, K\}$ and $\{0, KH, K\}$ in place. At rank 1, every layout takes $\{s_0, 1\}$ on both sides, or $\{s_0, K\}$ for the lane-major batch; the distances of a contiguous batch are N / H out of place and $2H$ / H in place, and one transform in place takes a buffer of $2H$ doubles. Three more rules: in place the data start at element 0, $s_0 = 0$; at rank 2 and 3 DFTI's default strides are refused, so set both strides in place, and out of place

set the strides of the bins, `DFTI_OUTPUT_STRIDES` on a descriptor for the forward and `DFTI_INPUT_STRIDES` on one for the backward; a lane-major batch of real data needs an even last length of at least 4, and at rank 2 and 3 its other lengths at least 2.

`ch17_real2d_in_place.c` transforms a real array of 24×30 points in place, with one descriptor for each direction. Its buffer `z` holds $N_1 = 24$ rows of $2H = 32$ doubles: row n_1 of the reals starts at `z[2 * H * n1]`, and after the forward, bin (k_1, k_2) is the complex value at `z[2 * (k1 * H + k2)]`.

ch17_real2d_in_place.c

```
/* ch17_real2d_in_place.c: a DFTI program. One 2D real DFT of 24 x 30 points
 * in place: the reals sit in rows padded to 2H = 32 doubles (H = 30/2 + 1),
 * and the forward overwrites each row with its H complex bins. One
 * descriptor per direction; the backward's DFTI_BACKWARD_SCALE is
 * 1/(N1 N2). Checked against a direct evaluation of the DFT and against the
 * input.
 */
#include <complex.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "mkl_dfti.h"

enum { N1 = 24, N2 = 30, H = N2 / 2 + 1 };
static const double TOL = 1e-12;

/* a descriptor of the 2D real transform, in place, that reads through the
 * strides in and writes through the strides out */
static MKL_LONG describe(DFTI_DESCRIPTOR_HANDLE *h, MKL_LONG *in,
                         MKL_LONG *out)
{
    MKL_LONG n[2] = {N1, N2};
    MKL_LONG status = DftiCreateDescriptor(h, DFTI_DOUBLE, DFTI_REAL, 2, n);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(*h, DFTI_CONJUGATE_EVEN_STORAGE,
                              DFTI_COMPLEX_COMPLEX);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(*h, DFTI_INPUT_STRIDES, in);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(*h, DFTI_OUTPUT_STRIDES, out);
    return status;
}

int main(void)
{
    const double pi = 3.14159265358979323846;
    double *z = malloc(sizeof(double) * N1 * 2 * H);     /* the one buffer */
    double *x = malloc(sizeof(double) * N1 * N2);        /* the input, kept */
    if (!z || !x)
    {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (int n1 = 0; n1 < N1; n1++)
        for (int n2 = 0; n2 < N2; n2++)
        {
            x[n1 * N2 + n2] = sin(0.3 * n1 + 0.2 * n2) + cos(0.11 * n1 * n2);
            z[n1 * 2 * H + n2] = x[n1 * N2 + n2];     /* row n1 at 2H n1 */
        }

    MKL_LONG rs[3] = {0, 2 * H, 1};        /* the reals: rows of 2H doubles */
    MKL_LONG cs[3] = {0, H, 1};            /* the bins: rows of H complex */
    DFTI_DESCRIPTOR_HANDLE fwd = NULL, bwd = NULL;
```

```
    MKL_LONG status = describe(&fwd, rs, cs);
    if (status == DFTI_NO_ERROR)
        status = DftiCommitDescriptor(fwd);
    if (status == DFTI_NO_ERROR)
        status = describe(&bwd, cs, rs);
    if (status == DFTI_NO_ERROR)
        status = DftiSetValue(bwd, DFTI_BACKWARD_SCALE, 1.0 / (N1 * N2));
    if (status == DFTI_NO_ERROR)
        status = DftiCommitDescriptor(bwd);
    if (status == DFTI_NO_ERROR)
        status = DftiComputeForward(fwd, z);        /* rows of H bins */
    if (status != DFTI_NO_ERROR)
    {
        fprintf(stderr, "DFTI: %s\n", DftiErrorMessage(status));
        return 1;
    }

    /* bin (k1, k2) at z[2 (k1 H + k2)], against the sum over x[n1, n2] */
    double err_X = 0.0, err_y = 0.0, top = 0.0;
    for (int k1 = 0; k1 < N1; k1++)
        for (int k2 = 0; k2 < H; k2++)
        {
            double complex ref = 0.0;
            for (int n1 = 0; n1 < N1; n1++)
                for (int n2 = 0; n2 < N2; n2++)
                    ref += x[n1 * N2 + n2] * cexp(-2.0 * pi * I *
                           ((double)(k1 * n1 % N1) / N1 +
                            (double)(k2 * n2 % N2) / N2));
            int b = 2 * (k1 * H + k2);
            double d = cabs(z[b] + I * z[b + 1] - ref);
            if (d > err_X || isnan(d))
                err_X = d;
            if (cabs(ref) > top)
                top = cabs(ref);
        }
    err_X /= top;

    status = DftiComputeBackward(bwd, z);           /* the reals again */
    if (status != DFTI_NO_ERROR)
    {
        fprintf(stderr, "DFTI: %s\n", DftiErrorMessage(status));
        return 1;
    }
    DftiFreeDescriptor(&bwd);
    DftiFreeDescriptor(&fwd);
    for (int n1 = 0; n1 < N1; n1++)
        for (int n2 = 0; n2 < N2; n2++)
        {
            double d = fabs(z[n1 * 2 * H + n2] - x[n1 * N2 + n2]);
            if (d > err_y || isnan(d))
                err_y = d;
        }
    int pass = err_X <= TOL && err_y <= TOL;    /* false for a NaN */
    printf("2D real DFT of %d x %d points in place, rows of %d doubles\n",
           N1, N2, 2 * H);
    printf("forward:  max |X - X_ref| / max |X_ref| = %.1e\n", err_X);
    printf("backward: max |y - x| = %.1e\n", err_y);
    printf("tolerance %.0e: %s\n", TOL, pass ? "PASS" : "FAIL");

    free(x);
    free(z);
    return pass ? 0 : 1;
}
```

Built and run as in Section 16.2, it prints:

```
2D real DFT of 24 x 30 points in place, rows of 32 doubles
forward:  max |X - X_ref| / max |X_ref| = 3.4e-16
backward: max |y - x| = 1.4e-15
tolerance 1e-12: PASS
```

17.5 Compute on several threads

A VectorFFT plan runs one compute at a time. A committed descriptor holds `DFTI_NUMBER_OF_USER_THREADS` plans, so that as many threads of your program can compute with it at once: for complex data the plans serve both directions, and real data has that many for each direction its strides serve. Each plan is created for the number of threads Table 17.5 gives (`nthreads`, Chapter 9).

Table 17.5 The plans a commit creates, and the threads each one is created for.

`DFTI_THREAD_LIMIT`	`DFTI_NUMBER_OF_USER_THREADS`	**Plans**	**Threads of each plan**
0, the default	1, the default	1	the machine's performance cores, or all its physical cores on a CPU with one kind of core
0	$n > 1$	n, at most 64	1
$t > 0$	n	n, at most 64	t

Set `DFTI_NUMBER_OF_USER_THREADS` to the number of threads that compute with one descriptor at once: a compute that finds every plan busy waits for one (Table 17.2). Different descriptors may compute at once; computes whose plans run on several threads take turns on the library's worker threads, from one descriptor or from several (Chapter 9). Do not call `DftiSetValue`, `DftiCommitDescriptor` or `DftiFreeDescriptor` on a descriptor while another thread computes with it. The thread count is part of the configuration a commit creates: a transform measured at one count is measured again at another (Chapter 10). A `DFTI_THREAD_LIMIT` above the logical CPUs of the machine is refused at the commit with 6: keep it at or below the logical CPUs of every machine your program runs on, or leave it 0.

17.6 Read a status and its message

Every DFTI function returns a status. Table 17.6 gives the statuses the interface returns, the only ones, and the message of `DftiErrorMessage()` for each: the words `VectorFFT DFTI:` followed by those in the table.

Table 17.6 The statuses of the interface.

Status	Name	Message	Returned for
0	`DFTI_NO_ERROR`	no error	success
1	`DFTI_MEMORY_ERROR`	not enough memory	memory exhausted at a create, a commit or a copy
2	`DFTI_INVALID_CONFIGURATION`	invalid configuration, or the descriptor is not committed	a create with a rank outside 1 to 7, a length below 1, or another precision or domain; a compute on an uncommitted descriptor
3	`DFTI_INCONSISTENT_CONFIGURATION`	inconsistent configuration	the refusals of Table 17.2 with status 3
5	`DFTI_BAD_DESCRIPTOR`	bad descriptor	a `NULL` descriptor, such as the handle `DftiFreeDescriptor` set to `NULL`, or a `NULL` pointer to a handle
6	`DFTI_UNIMPLEMENTED`	not implemented	the refusals of Table 17.2 with status 6

Most refusals also print a line on standard error that says why, as do two settings served with a difference; Table 17.2 quotes each line beside its setting. Each line starts with `vfft dfti:` and prints once per process. A commit that `vfft_create()` refuses prints `DftiCommitDescriptor:` and the reason `vfft_create()` gives, the text `vfft_error()` returns (Chapter 11). Nothing is printed for the statuses 2 and 5, for the refusals of `DftiSetValue` with status 3, and for `DFTI_REAL_STORAGE` = `DFTI_REAL_COMPLEX`. The library prints lines of its own besides these (Chapter 10, Chapter 11).

17.7 Know what DFTI does not reach

Every commit plans the way `vfft_create()` does: served from the store when it holds wisdom for the configuration, otherwise measured, and the winner saved (Chapter 10). Table 17.7 lists what a DFTI program cannot reach and what `vfft.h` offers instead; a program that needs one of these moves its transforms to `vfft.h` (Chapter 3 describes a transform there, Section 10.5 measures one again).

Table 17.7 What a DFTI program cannot reach.

What	Through DFTI	Through `vfft.h`
measuring a configuration again	not reachable	`VFFT_FLAG_RECALIBRATE`
naming a store of your own (Section 10.3)	`VFFT_WISDOM_DIR` only	`wisdom_dir`, or `VFFT_WISDOM_DIR`
printing the refusals of one plan	not reachable	`VFFT_FLAG_VERBOSE`
a row pitch the plan chooses	not reachable	`VFFT_FLAG_OWN_PITCH`
the bins in the plan's own order	`DFTI_BACKWARD_SCRAMBLED` is served in natural order	`VFFT_ORDER_SCRAMBLED`
single precision; the packed real formats; ranks above 3	refused	not served

Part V **Reference**

18 vfft.h reference

This chapter lists what vfft.h declares, in the header's words: the functions, the configuration, the enumerations, the flags and the facts vfft_info() reports. Look a name up here: the chapters of Part II show each one at work, and the symbols are those of Notation, at the front of the guide.

18.1 What the header declares

vfft.h is the library's interface. It includes only <stddef.h> and declares its names extern "C", so a C or a C++ program includes it as it is and links VectorFFT::vfft (Chapter 1). Table 18.1 lists every name of this release and the section that describes it.

Table 18.1 The names vfft.h declares, and where this chapter describes them.

Name	What it is	Section
vfft_create()	commits a configuration to one plan	Section 18.2.1
vfft_execute()	runs a plan on your buffers	Section 18.2.2
vfft_destroy()	frees a plan	Section 18.2.3
vfft_malloc(), vfft_free()	memory at the alignment the plans were measured on, and its release	Section 18.2.4
vfft_info()	a plan's facts, or the library's	Section 18.2.5
vfft_error()	why the last create or execute on this thread was refused	Section 18.2.6
vfft_plan	a plan: an opaque pointer, ready to execute	Section 18.2.1
vfft_config_t	the configuration a create commits to	Section 18.3
vfft_transform_t, vfft_dir_t	the transform and the direction	Section 18.4
vfft_layout_t, vfft_placement_t, vfft_order_t, vfft_batch_t	how the data is stored	Section 18.5
VFFT_FLAG_*	the flags of a configuration	Section 18.6
vfft_info_t	the facts vfft_info() reports	Section 18.7

The header also declares names that are not part of this release. Section 18.8 lists them; no other page of this guide uses them.

18.2 Functions

Each entry gives the declaration as vfft.h writes it, then the parameters, what the call returns, and what you need to know when you call it.

18.2.1 vfft_create()

```
typedef struct vfft_plan_s *vfft_plan;
vfft_plan vfft_create(const vfft_config_t *config);
```

config The configuration, Table 18.2.

Returns the plan, or NULL when the configuration is refused; vfft_error() then says why. A vfft_plan is an opaque pointer, ready to execute.

The create checks the configuration and takes its plan from wisdom. On a miss it measures the candidates and saves the winner in the store before it returns: this takes seconds, up to about a minute for some configurations, and later creates of the same configuration are served from the store at once. Every create plans PATIENT, except

one with VFFT_FLAG_ESTIMATE, which plans by formulas and measures nothing (Section 10.6). The create then builds the plan: its tables, its scratch memory and its threads are set up here, and the plan owns what it allocated until vfft_destroy().

Creates on several threads take turns on one lock, and a create may run while other threads execute plans. A create leaves flush-to-zero and denormals-are-zero enabled on the calling thread.

18.2.2 vfft_execute()

```
void vfft_execute(vfft_plan p, vfft_dir_t dir, const void *in, void *out);
```

p A plan from vfft_create().

dir VFFT_FORWARD or VFFT_BACKWARD, Table 18.3. A VFFT_R2C plan runs VFFT_FORWARD only, and a VFFT_C2R plan VFFT_BACKWARD only.

in The input, in_doubles doubles (Table 18.8): complex pairs for VFFT_C2C, reals for VFFT_R2C, the half spectrum for VFFT_C2R.

out The output, out_doubles doubles. In place, out is in; out of place, a buffer of its own.

The execute runs the plan on your buffers; it allocates, measures and decides nothing. Pass buffers that are not NULL, of the sizes vfft_info() reports. Out of place, in is left as it was unless the plan was created with VFFT_FLAG_DESTROY_INPUT. An in-place real plan holds the reals at the start of the buffer, or of each row, and writes the bins over them.

Different plans may execute at the same time on different threads. Never execute one plan from two threads at once: give each thread its own plan, or make the threads take turns. A call the library refuses writes nothing, and vfft_error() says why (Table 11.2).

18.2.3 vfft_destroy()

```
void vfft_destroy(vfft_plan p);
```

p A plan from vfft_create(), or NULL, which is accepted.

Frees the plan and everything it allocated. The buffers you passed to vfft_execute() stay yours, and a destroyed plan is not executed again.

18.2.4 vfft_malloc() and vfft_free()

```
void *vfft_malloc(size_t bytes);
void vfft_free(void *ptr);
```

bytes The size in bytes, for example info.in_doubles * sizeof(double).

ptr A block from vfft_malloc(), or NULL, which is accepted.

vfft_malloc() returns a block aligned to vfft_info().alignment bytes, the alignment every plan was measured on, or NULL when memory runs out. The block is not initialized. Release it with vfft_free() only, never with free(). In C++, cast the result: static_cast<double *>(vfft_malloc(bytes)).

18.2.5 vfft_info()

```
int vfft_info(vfft_plan p, vfft_info_t *out);
```

p A plan, or NULL for the library's facts only.

out Where the facts go, Table 18.7.

Returns 0, or -1 when `out` is `NULL`. With `p` set to `NULL`, it fills `version`, `alignment` and `cores`, sets `route` to `"-"` and every other field to 0.

18.2.6 `vfft_error()`

```
const char *vfft_error(void);
```

Returns why the last `vfft_create()` or `vfft_execute()` on the calling thread was refused, in words. The text is in static storage, one for each thread, and is `""` when nothing was refused since the thread's last create: a create empties it when it starts, and leaves it empty when it succeeds. `VFFT_FLAG_VERBOSE` and the environment variable `VFFT_VERBOSE` print the same text on standard error (Chapter 19).

18.3 The configuration

A `vfft_config_t` describes one transform: what it computes, on which array, on how many threads. Start from a zeroed one, `vfft_config_t cfg = {0};` in C or `vfft_config_t cfg = {};` in C++. Zeroed, with `n[0]` set, it is one 1D `VFFT_C2C` transform of `n[0]` points, interleaved, out of place, in natural order, on one thread, served from the folder of this CPU in the library's wisdom folder. Table 18.2 gives each field.

Table 18.2 The fields of `vfft_config_t`, and what a zero in each means.

Field	Type	What it holds	Zero
`transform`	`vfft_transform_t`	the transform, Table 18.3	`VFFT_C2C`
`rank`	`int`	the dimensions: 0 or 1 for 1D, 2 or 3	1D
`n[4]`	`int`	the extents: `n[0]` the slowest axis, `n[rank - 1]` the contiguous one, Eq. (3.1)	set `n[0]` at least
`howmany`	`size_t`	the batch count K; 0 or 1 is one transform	one transform
`layout`	`vfft_layout_t`	`VFFT_LAYOUT_INTERLEAVED`, Table 18.4	interleaved
`placement`	`vfft_placement_t`	out of place or in place	out of place
`order`	`vfft_order_t`	the order of the output bins	natural
`batch`	`vfft_batch_t`	where the K transforms of a batch sit, Table 18.5	the default geometry
`nthreads`	`int`	the threads the plan runs on, the caller's included: 0 or 1 is one; at most the machine's logical CPUs, and at most 64	one thread
`flags`	`unsigned`	`VFFT_FLAG_*` bits, Table 18.6	none
`wisdom_dir`	`const char *`	a store you name: a directory that holds wisdom files, opened as it is; `NULL`: the directory `VFFT_WISDOM_DIR` names, or else the folder of this CPU in the library's wisdom folder	the folder of this CPU in the library's wisdom folder, or the directory `VFFT_WISDOM_DIR` names

A plan is measured at its own `nthreads`, so each thread count is a configuration of its own, with its own wisdom. `wisdom_dir` names an existing directory: the create serves the configuration from the wisdom in it and saves every new winner into it, and it never creates the directory. A process keeps each directory it opens, up to 64 of them, each named in at most 511 characters. Leave `wisdom_dir` `NULL` unless your program needs a store of its own (Section 10.3); `VFFT_WISDOM_DIR` names one without changing the program (Section 19.2).

18.4 Transforms and directions

A `VFFT_C2C` plan of one transform of N points computes Eq. (2.1a) with `VFFT_FORWARD` (0) and Eq. (2.1b) with `VFFT_BACKWARD` (1), the two values of `vfft_dir_t`. The backward transform is not normalized: a forward transform followed by a backward one returns the input times N. At rank 2 and 3 the same sums run along every axis, and the roundtrip returns the input times N_1N_2 or $N_1N_2N_3$; `VFFT_C2R` after `VFFT_R2C` does the same. Bin k of an axis of N points holds frequency k for $k \le N/2$, and frequency $k - N$ for $k > N/2$.

Table 18.3 The transforms of this release, the values of `vfft_transform_t`.

Value	`in`	`out`	Directions
`VFFT_C2C` (0)	complex	complex	`VFFT_FORWARD`, Eq. (2.1a); `VFFT_BACKWARD`, Eq. (2.1b)
`VFFT_R2C` (1)	real	the half spectrum	`VFFT_FORWARD` only
`VFFT_C2R` (2)	the half spectrum	real	`VFFT_BACKWARD` only: the inverse of `VFFT_R2C`, times the number of points

The half spectrum is bins 0 to $\lfloor N/2 \rfloor$ of the last axis, `N/2 + 1` complex values per row in C. The transform of a real array has $X[k] = X^*[((-k))_N]$, so these bins determine the others. `VFFT_C2R` reads the half spectrum of a real array, as `VFFT_R2C` writes it.

The data is row-major, the last axis contiguous. Element $x[n_1, n_2, n_3]$ of a 3D array of extents $N_1 \times N_2 \times N_3$ (`n[0]`, `n[1]`, `n[2]`) is complex number ℓ of Eq. (3.1): its real part is double 2ℓ, its imaginary part double $2\ell + 1$. A 2D array drops n_3: $\ell = n_1N_2 + n_2$. A real array holds one double per element, at ℓ; a half spectrum has $\lfloor N_3/2 \rfloor + 1$ in place of N_3.

18.5 Layout, placement, order and batch

Table 18.4 gives the values of the four enumerations that say how the data is stored. The value 0 of each is what a zeroed configuration asks for.

Table 18.4 The values of `vfft_layout_t`, `vfft_placement_t`, `vfft_order_t` and `vfft_batch_t`.

Value	Number	Meaning
`VFFT_LAYOUT_INTERLEAVED`	0	one array of adjacent (re, im) pairs
`VFFT_PLACEMENT_OUT`	0	separate input and output buffers
`VFFT_PLACEMENT_IN`	1	one buffer: `out == in`
`VFFT_ORDER_NATURAL`	0	bin k at position k
`VFFT_ORDER_SCRAMBLED`	1	`VFFT_C2C` only: an order of the plan's own, which can be faster; only the backward transform of the same plan reads it
`VFFT_BATCH_DEFAULT`	0	1D: contiguous for `VFFT_C2C`, lane-major for `VFFT_R2C` and `VFFT_C2R`; 2D and 3D: contiguous arrays, one after another
`VFFT_BATCH_CONTIGUOUS`	1	each transform a block of its own, one after another
`VFFT_BATCH_LANE_MAJOR`	2	the transforms side by side, sample by sample

18.5.1 Batch addressing

A batch is K transforms (`howmany`) of one size. Table 18.5 gives where element n, or bin k, of transform t sits, counted in doubles, for $0 \le t \le K - 1$. A 2D or 3D batch is contiguous arrays by default, or lane-major when named. A lane-major real batch

needs an even N of at least 4 in 1D, $N_1 \geq 2$ and an even N_2 of at least 4 in 2D, and $N_1, N_2 \geq 2$ and an even N_3 of at least 4 in 3D. Chapter 8 shows each geometry at work. Figure 3.2 shows at a glance which placement, order and batch geometry a create serves for each transform and rank.

Table 18.5 Where element n (bin k) of transform t of a batch sits, in doubles; a layout that names no placement holds in both.

Geometry	Complex data	Real data
1D, contiguous	`z[2(t N + n)]`	out of place: reals at `x[t N + n]`; bins at `z[2(t (N/2+1) + k)]`
1D, lane-major	`z[2(n K + t)]`	reals at `x[n K + t]`; bins at `z[2(k K + t)]`
2D or 3D, contiguous	each array a block of its own, one after another	out of place: the same
2D, lane-major	element (n_1, n_2) at `z[2((n1 N2 + n2) K + t)]`	out of place: reals at `x[(n1 N2 + n2) K + t]`; bin (k_1, k_2) at `z[2((k1 (N2/2+1) + k2) K + t)]`
3D, lane-major	element (n_1, n_2, n_3) at `z[2(((n1 N2 + n2) N3 + n3) K + t)]`	out of place: reals at `x[((n1 N2 + n2) N3 + n3) K + t]`; bin (k_1, k_2, k_3) at `z[2(((k1 N2 + k2) (N3/2+1) + k3) K + t)]`
1D, contiguous, in place		reals at `x[2t (N/2+1) + n]`; bins at `z[2(t (N/2+1) + k)]`
2D, contiguous, in place		rows padded to $2(N_2/2+1)$ doubles, array t after array $t-1$: reals at `x[(t N1 + n1) 2(N2/2+1) + n2]`; bin (k_1, k_2) at `z[2((t N1 + k1) (N2/2+1) + k2)]`
3D, contiguous, in place		rows padded to $2(N_3/2+1)$ doubles, array t after array $t-1$: reals at `x[((t N1 + n1) N2 + n2) 2(N3/2+1) + n3]`; bin (k_1, k_2, k_3) at `z[2(((t N1 + k1) N2 + k2) (N3/2+1) + k3)]`
2D, lane-major, in place		rows padded to $N_2 + 2$ doubles: reals at `x[(n1 (N2+2) + n2) K + t]`; bin (k_1, k_2) at `z[2((k1 (N2/2+1) + k2) K + t)]`
3D, lane-major, in place		rows padded to $N_3 + 2$ doubles: reals at `x[((n1 N2 + n2) (N3+2) + n3) K + t]`; bin (k_1, k_2, k_3) at `z[2(((k1 N2 + k2) (N3/2+1) + k3) K + t)]`

18.6 Flags

`flags` is the bitwise OR of the values in Table 18.6; 0 sets none. A create refuses any other bit.

Table 18.6 The flags of this release.

Flag	Value	Effect
`VFFT_FLAG_PATIENT`	`1u << 0`	names the default: every create without `VFFT_FLAG_ESTIMATE` plans PATIENT, the wide search on a miss, whether this flag is set or not
`VFFT_FLAG_RECALIBRATE`	`1u << 2`	measure this configuration again, even when the store holds wisdom for it or this process measured it, and save the winner
`VFFT_FLAG_DESTROY_INPUT`	`1u << 3`	an out-of-place plan may overwrite its input
`VFFT_FLAG_OWN_PITCH`	`1u << 4`	an in-place 2D or 3D real plan of one transform may choose its row pitch; read it from `vfft_info().stride`. Other plans refuse the flag

Flag	Value	Effect
VFFT_FLAG_VERBOSE	1u << 5	print this create's refusals and warnings on standard error as well
VFFT_FLAG_ESTIMATE	1u << 6	plan a VFFT_C2C transform by formulas from its lengths and this machine's cache sizes: nothing is measured, no wisdom is read or saved, and wisdom_dir is not opened. One transform; in 2D and 3D, natural order on one thread. A create refuses it on any other configuration, together with VFFT_FLAG_PATIENT or VFFT_FLAG_RECALIBRATE, and while a variable of the library that changes plans is set

18.7 The facts of a plan

vfft_info() fills a vfft_info_t, Table 18.7. Read the sizes of your buffers from it, never from your own arithmetic.

Table 18.7 The fields of vfft_info_t.

Field	Type	What it reports
version	const char *	the library's version, "MAJOR.MINOR.PATCH"
alignment	size_t	in bytes: the alignment of vfft_malloc() blocks, and of the buffers the plans were measured on
cores	int	this machine's performance cores: those of a hybrid CPU, every physical core otherwise; the nthreads that runs a plan on all of them
route	const char *	a short name of the method the plan runs, for information only; "-" when it has no single name
nthreads	int	the threads the plan runs on
in_doubles	size_t	the doubles vfft_execute() reads
out_doubles	size_t	the doubles vfft_execute() writes
stride	size_t	a lane-major batch: element n of transform t is complex element n * stride + t (on the real side of a real batch, double n * stride + t, Table 18.5); an in-place 2D or 3D real plan of one transform, or of a contiguous batch: the doubles from one row to the next. Read it, never compute it

Table 18.8 gives the sizes in_doubles and out_doubles report. P is the product of the extents, L the last extent, H the same product with L replaced by $\lfloor L/2 \rfloor + 1$ (the bins of the half spectrum), and K the batch count.

Table 18.8 What in_doubles and out_doubles hold.

Plan	in_doubles	out_doubles
VFFT_C2C	$2PK$	$2PK$
VFFT_R2C, out of place	PK	$2HK$
VFFT_C2R, out of place	$2HK$	PK
VFFT_R2C or VFFT_C2R, in place, 1D	$2HK$	$2HK$
VFFT_R2C or VFFT_C2R, in place, 2D or 3D, one transform	P/L rows of stride doubles	the same
VFFT_R2C or VFFT_C2R, in place, a contiguous 2D or 3D batch	K blocks of P/L rows of stride doubles	the same
VFFT_R2C or VFFT_C2R, in place, a 2D lane-major batch	$N_1(N_2+2)K$	the same
VFFT_R2C or VFFT_C2R, in place, a 3D lane-major batch	$N_1N_2(N_3+2)K$	the same

Without `VFFT_FLAG_OWN_PITCH`, the stride of an in-place 2D or 3D real plan is the tight $2(\lfloor L/2 \rfloor + 1)$. For twelve plans, `vfft_info()` reports these sizes, each as Table 18.8 gives it:

```
vfft_malloc() blocks are aligned to 64 bytes

plan                                          in_doubles out_doubles stride
1D C2C, N = 64                                       128         128      -
1D R2C, N = 64                                        64          66      -
1D C2R, N = 63                                        64          63      -
1D R2C, N = 64, in place                              66          66      -
1D R2C, N = 63, in place                              64          64      -
1D R2C, N = 64, K = 4 lane-major                     256         264      4
2D C2C, 16 x 32                                     1024        1024      -
2D R2C, 256 x 64, in place                         16896       16896     66
3D R2C, 8 x 8 x 8                                    512         640      -
2D R2C, 16 x 8, K = 4, in place                      640         640     10
3D R2C, 8 x 8 x 8, K = 2, in place                  1280        1280     10
3D C2R, 8 x 8 x 8, K = 2 lane-major, in place       1280        1280      2

every size as the reference gives it: PASS
```

18.8 Names not part of this release

`vfft.h` also declares the names of Table 18.9. They are not part of this release: this guide does not describe them, and a program of this release does not use them.

Table 18.9 Names in `vfft.h` that are not part of this release.

Name	Where it appears
`VFFT_LAYOUT_SPLIT`	a value of `vfft_layout_t`
`vfft_execute_split()`	a function
`VFFT_DCT1` to `VFFT_DCT4`, `VFFT_DST1` to `VFFT_DST3`, `VFFT_DHT`	values of `vfft_transform_t`
`rank` 4, `n[3]`	fields of `vfft_config_t`
`VFFT_FLAG_EXHAUSTIVE`	a flag, `1u << 1`; `vfft_create()` refuses it
`vfft_diagnostics.h`	a second header, installed beside `vfft.h`

19 Environment variables

You set these variables in a program's environment to name a store of its own, to see why a call was refused, to keep a store unchanged, or to change how long idle worker threads wait, all without changing the program. Table 19.1 lists them; each section of this chapter gives one in full.

Table 19.1 The environment variables a program meets.

Variable	Set it to	Effect	Unset
`VFFT_WISDOM_DIR`	an existing directory	the store of every create whose configuration names none, opened as it is	the folder of this CPU in the library's wisdom folder
`VFFT_VERBOSE`	`1`	every refusal and warning is printed on standard error	only creates with `VFFT_FLAG_VERBOSE` print; `vfft_error()` holds the reason
`VFFT_WISDOM_WRITE`	`0`	nothing is saved: the winner of a miss serves this process only	every miss saves its winner
`VFFT_THREAD_BLOCK_MS`	milliseconds	how long an idle worker thread spins before it parks	200

The library reads each variable once, the first time it needs it, so set them before the program starts. The library also reads other variables whose names start with `VFFT_`; they exist for testing the library. Leave them unset: a create with `VFFT_FLAG_ESTIMATE` refuses while one of them that changes plans is set (Section 10.6).

19.1 Set a variable for a program

In Git Bash, or in a Linux shell, put the assignment in front of the command. It holds for that run only:

```
VFFT_VERBOSE=1 ./app
```

`export VFFT_VERBOSE=1` sets it for every later command of the shell. In PowerShell, set it for the session and remove it when you are done:

```
$env:VFFT_VERBOSE = "1"
.\app.exe
Remove-Item Env:VFFT_VERBOSE
```

19.2 `VFFT_WISDOM_DIR`: a store of your own

`VFFT_WISDOM_DIR` names a store for every create whose configuration's `wisdom_dir` is `NULL`, in place of the folder of this CPU that the library picks in its wisdom folder (Section 10.2). Leave it unset for ordinary runs; Section 10.3 gives the cases that call for a store of your own and shows how to seed one. The library reads the variable at the first such create of the process; an empty value counts as unset.

- The directory is opened as Section 10.3 describes: it must exist, it and its files must be writable, and it is the store itself, so name a copy of the folder of this CPU, never the wisdom folder that holds the folders of the CPUs.
- A relative name is resolved against the program's working directory. Use an absolute name for a program that can be started from anywhere.

- A configuration's `wisdom_dir` comes first. A create that names a directory there uses that directory, whatever `VFFT_WISDOM_DIR` says. Inside a program, name the store there instead (Section 18.3).
- Unset, the program uses the library's wisdom folder: the folder of this CPU under `<prefix>/share/vectorfft/wisdom/` for an installed library, or under `src/wisdom/` of the copy of the sources for a library built with `add_subdirectory()` or `FetchContent`.
- A DFTI program can name a store only this way: the DFTI interface has no setting for it (Chapter 16).

19.3 `VFFT_VERBOSE`: print every refusal

With `VFFT_VERBOSE` set to any value other than an empty one or `0`, the library prints every reason it gives `vfft_error()` on standard error as well, one line each, in the form `vfft: <reason>`. That covers refused creates, refused executes and warnings, on every thread, for the whole process.

`VFFT_FLAG_VERBOSE` in a configuration does the same for one create only: it prints nothing for the executes of the plan. Set `VFFT_VERBOSE=1` while you develop a program, to see at once why a call did nothing; Appendix B lists the reasons you meet most often.

19.4 `VFFT_WISDOM_WRITE`: keep the store unchanged

With `VFFT_WISDOM_WRITE` set to exactly `0`, the process saves nothing into its store. A miss still measures, and its winner serves the process until it exits; the next process measures the same configuration again. The library says once, on standard error, that saving is off. Any other value, and no value, leaves saving on.

Set it for a run that must leave its store as it is, such as a trial run against the wisdom you ship with an application (Section 10.4).

19.5 `VFFT_THREAD_BLOCK_MS`: how long idle workers spin

A plan with `nthreads` of 2 or more runs on the library's pool of worker threads. Between two executes, an idle worker spins, checking for work, for `VFFT_THREAD_BLOCK_MS` milliseconds, and then parks until an execute wakes it. A spinning worker keeps its core busy; a parked worker uses no processor time, and is woken before it runs again. A worker that no plan needs parks at once, whatever the value.

The value is a number of milliseconds, and fractions are allowed. Unset, it is 200. A negative value, or one that is not a number, counts as 0: each worker parks as soon as it is idle. Set it lower when other threads of your program need those cores between executes.

A Notes for Windows and MinGW

This appendix collects what you need to know to build and run VectorFFT programs on Windows with MinGW-w64 GCC, the toolchain this release is built and tested with: which compiler to use, what to tell CMake, which DLL a program needs, where to build, and which CPU the program runs on.

A.1 Use one compiler for the library and your programs

The library is one static archive, `libvfft.a`, compiled by one compiler. Build every program that links it with that same compiler: the same MinGW-w64 GCC, of the same version and the same kind, which `gcc --version` names, or the same ICX. This release is built and tested with GCC 15.2.0 of the kind `x86_64-ucrt-posix-seh`: 64-bit, the UCRT runtime, POSIX threads; Intel's ICX builds it as well. Microsoft's compilers, `cl` and `clang-cl`, are not supported, for the library or for programs that link it.

Put the compiler's `bin` folder first on `PATH` in the shell you build in, so that `gcc`, `cmake --build` and the programs you run find the same tools. In Git Bash, for a compiler installed in `C:\mingw64`:

```
export PATH=/c/mingw64/bin:$PATH
```

A.2 Name the generator and the compiler to CMake

On a machine with Visual Studio, CMake picks Visual Studio and `cl` unless it is told otherwise, and the library does not build with `cl`. Name both at every configure, for the library, for your program and for the examples:

```
cmake -S . -B build -G Ninja -DCMAKE_C_COMPILER=gcc
```

`-G "MinGW Makefiles"` works as well where Ninja is not installed. A build folder keeps the compiler it was first configured with, so configure a fresh folder when you change compilers. Write paths in CMake arguments with forward slashes, as in `-DCMAKE_INSTALL_PREFIX=C:/vectorfft`, and pick an install prefix you can write to: the default, under `C:/Program Files (x86)`, needs administrator rights, and your programs save their wisdom under the prefix (Section 10.2). Keep the prefix, and the path of every store you name, in ASCII characters; CMake warns at configure time when the path of the installed wisdom folder holds other characters.

A.3 Give the program `libwinpthread-1.dll`, or link it statically

A program built with MinGW-w64 GCC loads `libwinpthread-1.dll`, from the compiler's `bin` folder, when it starts. Where Windows cannot find the DLL, the program stops before `main()` and prints nothing of its own; Section 1.7 gives what Git Bash and PowerShell then report, and the commands of the two fixes. Use them this way:

- While you develop, keep the compiler's `bin` folder on `PATH`.
- When you ship the program, copy `libwinpthread-1.dll` from that folder into the folder of the `.exe`, or link the program with `-static`; a static program needs no MinGW DLL.

The library never adds `-static` to your link by itself. A static program still uses the C runtime DLLs, `api-ms-win-crt-*.dll`, which are part of Windows 10 and later. Git Bash puts Git's own copy of `libwinpthread-1.dll` on its `PATH`, so a program that starts in Git Bash can still stop elsewhere: try a program you ship from a PowerShell window whose `PATH` holds no MinGW folder.

A.4 Keep build folders short

Windows limits most programs to paths of 260 characters, and the library's build writes object files with long names: CMake warns when the full path of an object file passes 250 characters, and such a build can fail. Build in a folder with a short path, such as `C:/build/vectorfft`, rather than deep inside your user profile. With CMake 4.2 or newer, `-DCMAKE_INTERMEDIATE_DIR_STRATEGY=SHORT` gives the folders and object files inside the build folder short, hashed names:

```
cmake -S . -B C:/build/vectorfft -G Ninja -DCMAKE_C_COMPILER=gcc \
      -DCMAKE_INTERMEDIATE_DIR_STRATEGY=SHORT
```

The same holds for a project that builds the library with `add_subdirectory()` or `FetchContent`: the library's object files then lie in your project's build folder, one or two levels deeper.

A.5 Build for the CPU that runs the program

By default the library is compiled for the CPU of the machine that builds it: `VFFT_NATIVE` is `ON`, which compiles with `-march=native`. The library does not check the CPU when a program starts, so on a CPU that lacks one of the build machine's instructions, a program that links the library stops with an illegal-instruction error. To build a library for programs that run on other machines, configure it with `-DVFFT_NATIVE=OFF`, as Section 1.3 shows; it then needs a CPU with AVX2 and FMA.

Wisdom belongs to one CPU as well: the library keeps one folder of wisdom for each CPU, and a program reads and saves the folder of the CPU it runs on (Section 10.2).

A.6 Use `pkgconf` where `pkg-config` is missing

The WinLibs build of MinGW-w64 ships `pkgconf.exe` and no `pkg-config`. Use `pkgconf` where a build line asks for `pkg-config`, with `PKG_CONFIG_PATH` naming the installed package, as Section 1.6 shows.

Section 19.1 shows how to set the library's environment variables in Git Bash and in PowerShell.

B Troubleshooting

Find what you see in the tables of this appendix, then its cause and the fix. When the library refuses a call, it says why: a create that returns `NULL`, or an execute that writes nothing, leaves the reason in `vfft_error()`, and with `VFFT_VERBOSE=1` in the environment every reason is also printed on standard error as it happens (Section 19.3).

B.1 The build fails

Table B.1 Symptoms while you configure, compile, link or install.

Symptom	Cause	Fix
CMake picks Visual Studio or `cl`, and the library does not compile	no generator and no compiler named	configure a new build folder with `-G Ninja -DCMAKE_C_COMPILER=gcc` (Section A.2)
CMake warns that the path of an object file passes 250 characters	a build folder with a long path	build in a folder with a short path (Section A.4)
`cmake --install` stops: VectorFFT "was configured for the prefix" ...	the prefix lies inside the VectorFFT clone, or differs from the one you configured	configure with the prefix you install to, outside the clone
`find_package(VectorFFT)` finds no `VectorFFTConfig.cmake`	CMake does not know where the library is installed	pass `-DCMAKE_PREFIX_PATH=<prefix>`
CMake warns that VectorFFT "was installed for the prefix" ... "and found under" another	the installed files were moved or copied	install again with the prefix you use, or name a store seeded from the folder of this CPU (Section 10.3)
`add_subdirectory` was "not given a binary directory"	the VectorFFT clone lies outside your project	name one: `add_subdirectory(<clone> vectorfft EXCLUDE_FROM_ALL)`
`vfft.h: No such file or directory`	the compiler does not see the include folder	link `VectorFFT::vfft`, or pass `-I<prefix>/include`
the linker finds no `vfft_create` and the other names	the library is not linked	link `VectorFFT::vfft`, or put `-L<prefix>/lib -lvfft -lm` after your source files
`pkg-config: command not found`	WinLibs ships `pkgconf` only	run `pkgconf` in its place (Section A.6)

B.2 The program does not start

Table B.2 Symptoms when a program starts.

Symptom	Cause	Fix
the program prints nothing and ends at once: exit code -1073741515 in PowerShell; in Git Bash, only when no copy of the DLL is on its `PATH`, `error while loading shared libraries` and 127	Windows cannot find `libwinpthread-1.dll`	put the compiler's `bin` folder on `PATH`, copy the DLL next to the program, or link with `-static` (Section 1.7)
the program stops with an illegal instruction on another machine	the library was built for the CPU of the machine that built it	build the library with `-DVFFT_NATIVE=OFF` (Section A.5)

B.3 A create returns `NULL`

`vfft_create()` returns `NULL` when it refuses a configuration, and `vfft_error()` then says why: print it at once, as every program of this guide does. Table 11.1 gives the reasons a program meets most often, with the change each one needs; Table B.3 gives the others, by the words they begin with.

Table B.3 More reasons a create refuses a configuration.

The reason begins	Cause	Fix
`vfft_create: invalid transform`	a value outside `vfft_transform_t`	one of the names of Table 18.3
`vfft_create: order=SCRAMBLED is only wired for C2C plans`	`VFFT_ORDER_SCRAMBLED` asked of a real plan	leave `order` at `VFFT_ORDER_NATURAL`
any other reason	a configuration this release does not serve	the reason names what is not served; Chapter 6 (complex), Chapter 7 (real) and Chapter 8 (batches) show what is served

B.4 An execute does nothing, or the numbers look wrong

`vfft_execute()` returns nothing. When it refuses a call, it writes nothing into `out` and leaves the reason in `vfft_error()`; Table 11.2 gives the reasons, with the change each one needs. A program that makes five common mistakes, two creates and three executes, prints the reason the library gives for each; every one is refused, and no refused execute writes into its output:

```
1. create with n[0] left at 0: refused
    vfft_create: n[0]=0 invalid (every transform length must be >= 1)
2. create with nthreads = -1: refused
    vfft_create: nthreads -1 is negative (0 or 1 is one thread)
3. VFFT_BACKWARD on a VFFT_R2C plan: refused
    vfft_execute: R2C plans are forward-only (real -> spectrum); the
    unnormalized inverse is a separate VFFT_C2R plan (executed with
    VFFT_BACKWARD) - nothing executed
4. two buffers for an in-place VFFT_R2C plan: refused
    vfft_execute: this R2C plan is IN-PLACE (one padded CCE plane of
    2*(N/2+1) doubles) and must be called with out == in; got distinct
    pointers -- nothing executed
5. a NULL plan: refused
    vfft_execute: NULL plan (vfft_create failed, or the plan was destroyed)
    - nothing executed

every call refused, with its reason: PASS
```

Table B.4 lists the other symptoms of an execute: a plan created in place that is given two buffers, and results that look wrong.

Table B.4 Symptoms of an execute.

Symptom	Cause	Fix
nothing written; the reason begins `vfft_execute: this R2C plan is IN-PLACE` (or `C2R`)	two buffers given to a plan created in place	pass the one buffer twice, as `in` and as `out`
the result is the expected one times N, N_1N_2 or $N_1N_2N_3$	the backward transform is not normalized, Eq. (2.1b)	divide by the product of the extents, best in a pass your program makes anyway
the upper half of a spectrum looks wrong	bin $k > N/2$ holds frequency $k - N$	read those bins as negative frequencies (Section 18.4)
bins of a `VFFT_ORDER_SCRAMBLED` plan are not where you look for them	the scrambled order is the plan's own	ask `VFFT_ORDER_NATURAL` to read bins; keep the scrambled order for a forward and backward pair of one plan
an in-place 2D or 3D real result is right in the first row only	the rows were laid out N_{last} doubles apart, or at another pitch than `stride`	lay the rows `vfft_info().stride` doubles apart
the input of an out-of-place plan changed	the plan was created with `VFFT_FLAG_DESTROY_INPUT`	drop the flag, or keep a copy of the input
the last digits of a result differ between runs, or between thread counts	different plans add in different orders: a run that measures can pick another plan than a run served from the store, and so can another `nthreads`	compare results with a tolerance, not bit for bit

Never execute one plan from two threads at once. When several threads transform at once, give each thread its own plan.

B.5 A create takes seconds, or measures on every run

A create that misses wisdom measures the candidates and saves the winner before it returns: this takes seconds, up to about a minute for some configurations, once per configuration and store. Table B.5 lists what makes it happen more often than that.

Table B.5 Symptoms of creates that measure.

Symptom	Cause	Fix
the first create of a configuration takes seconds	a miss: the create measures and saves the winner. On a CPU without shipped wisdom, every configuration misses once	let it finish once; later creates are served at once. Create plans at startup, not inside a loop
every run measures the same configuration again	nothing is saved: the library's wisdom folder lies under a prefix the program cannot write; a store you named is missing, cannot be written, or is named relative to a working directory that changes; or `VFFT_WISDOM_WRITE=0`	install the library to a prefix you can write (Section 10.2), or name a writable store by an absolute path, seeded from the folder of this CPU (Section 10.3); leave `VFFT_WISDOM_WRITE` unset

Symptom	Cause	Fix
standard error shows `[wisdom2] the store directory '<dir>' is missing or cannot be written`	`<dir>` does not exist, or the process cannot write it or its files: the folder of this CPU under a prefix the program cannot write, or a store you named	for the library's wisdom folder, install to a prefix you can write, or name a writable store seeded from the folder of this CPU; for a store you named, create it as a copy of the folder of this CPU, or make it and its files writable (Section 10.3)
standard error shows `[wisdom2] saving is off (VFFT_WISDOM_WRITE=0)`	`VFFT_WISDOM_WRITE` is set to `0`	unset it (Section 19.4)
standard error shows `[measure] waiting for another VectorFFT measurement on this machine (up to 60 s)`	another program on this machine is measuring: measurements take turns	let it finish. A create that waits longer than that measures anyway and keeps its winner for this process only
standard error shows `[measure] this thread is not pinned to a P-core`	the process may not run on a performance core, so the library keeps what it measures for this process only	let the process run on the performance cores while it creates plans
standard error says the store was measured on another CPU	a store you named holds wisdom measured on a different CPU	name no store, or seed the store from the folder of this CPU (Section 10.3); or measure again with `VFFT_FLAG_RECALIBRATE`
a plan with `nthreads` of 2 or more measures although the one-thread plan was served	each thread count is a configuration of its own	expected once for each count; later runs are served
`git status` shows changes under `src/wisdom/` of a VectorFFT clone that your project builds	a library built with `add_subdirectory()` or `FetchContent` saves new winners into the folder of this CPU under the clone's `src/wisdom/`, its wisdom folder	nothing: the saved wisdom serves your next runs. To keep the clone as it is, name a store of the application's own (Section 10.3)

B.6 A DFTI program

Table B.6 Symptoms of a program that uses the DFTI interface.

Symptom	Cause	Fix
`objdump -p app.exe` lists `Dfti` names, or the program loads an MKL DLL for its transforms	a file was compiled against MKL's `mkl_dfti.h` or `mkl.h`, not the interface's header	put `src/shims/dfti/` ahead of MKL's include folder, include `mkl_dfti.h` in every file that calls DFTI, and compile `vfft_dfti.c` with your files (Chapter 16)
a DFTI call returns a status other than 0	the descriptor asks for something the interface does not serve, or a call comes in the wrong order	`DftiErrorMessage()` names the kind of error; where the interface refuses a setting or a layout, it also prints the reason on standard error, on a line that begins `vfft dfti:` (Chapter 17)
every run of a DFTI program measures again	the program cannot write the library's wisdom folder, and the DFTI interface has no setting that names a store	install the library to a prefix you can write, or set `VFFT_WISDOM_DIR` to a writable store seeded from the folder of this CPU (Section 10.3)